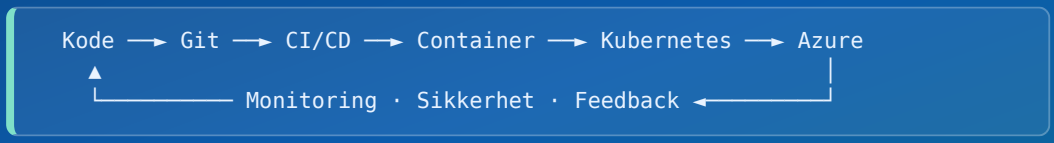
LÆREBOK · NOROFF DEVOPS ENGINEERING

DEVOPS, CLOUD & MICROSOFT AZURE

Den komplette guiden fra nybegynner til profesjonell

Visuell utgave – lær med bilder



IT-grunnlag · Nettverk · Linux · Python · Git · Cloud · Azure (AZ-900/AZ-104)
DevOps · Docker · Kubernetes · Terraform · CI/CD · Sikkerhet · MLOps
12 praktiske prosjekter · Quiz med fasit · Ordliste · 12-måneders læringsplan

Utgave oktober 2026 · Skrevet på norsk med engelske fagbegreper

Innholdsfortegnelse

Slik bruker du boken
- Symbolene i boken
- Læringsreisen
- Tre regler

1 · Forstå IT fra bunnen
- Hvordan fungerer en datamaskin?
- Operativsystemet
- Server vs. vanlig PC
- Applikasjonen: frontend, backend og database
- Databaser
- Internett, IP og DNS
- HTTP og API
- Hva skjer når du åpner en nettside?
- Klient–server og arkitekturer
- Virtualisering
- Slik henger alt sammen
- Test deg selv

2 · Nettverk for DevOps
- Lagmodellen (OSI / TCP-IP)
- IP-adresser
- Subnetting og CIDR
- DNS – internettets telefonkatalog
- DHCP – automatiske adresser
- TCP, UDP og porter
- Ruter, svitsj, brannmur og NAT
- VPN
- Lastbalansering og reverse proxy
- TLS og sertifikater
- HTTP-statuskoder
- SSH med nøkler
- Feilsøking – nedenfra og opp
- Test deg selv

3 · Linux
- Hvorfor Linux?
- Terminalen
- Filsystemet
- Kommandoene du må kunne
- Brukere og rettigheter
- Prosesser og tjenester
- Pakkehåndtering

Miljøvariabler
Bash-skript
Cron – planlagte jobber
Logger og overvåking
Sikkerhet på 6 punkter
Feilsøking
Test deg selv

4 · Programmering for DevOps
Hva er programmering?
Kom i gang
Variabler og datatyper
Betingelser og løkker
Funksjoner
Lister, ordbøker og JSON
API-kall, feil og logging
Python eller Bash?
Python mot Azure
Profesjonelle vaner
Test deg selv

5 · Git og GitHub
Hvorfor versjonskontroll?
De tre områdene – det viktigste å forstå
Branch og merge
GitHub: clone, push og pull
Pull requests
Angre trygt
Arbeidsflyter
GitHub Actions – første smakebit
Test deg selv

6 · Cloud Computing
Hva er skyen?
Slik ser skyen ut fysisk
IaaS, PaaS og SaaS
Public, private og hybrid
Skalering
Høy tilgjengelighet og katastrofer
Skytjenestene i kategorier
Delt ansvar
Kostnader
Azure, AWS og Google Cloud
Test deg selv

7 · Microsoft Azure
7.1 Grunnmur, identitet og styring

7.2 Nettverk og compute
7.3 Lagring, data, sikkerhet og drift
7.4 Azure DevOps og Bicep
Del 7 – test deg selv
7.5 AZ-900 og AZ-104

8 · DevOps Engineering
Muren som DevOps rev
DevOps-kultur – CALMS
Agile og Scrum
DevOps-livssyklusen
CI, Continuous Delivery og Continuous Deployment
Infrastruktur som buskap
Deploy-strategier
Monitoring og observability
Hendelser og SRE
Platform Engineering og DevSecOps
En dag som DevOps Engineer
Test deg selv

9 · Docker og containere
Hvorfor containere?
Container vs. virtuell maskin
Image og container
Dockerfile og lag
Volumes og nettverk
Docker Compose
Sikkerhet
Feilsøking
Prosjekt i korte trekk
Test deg selv

10 · Kubernetes
Hvorfor Kubernetes?
Arkitekturen
Pod, Deployment og Service
Ingress og Gateway API
ConfigMap, Secret og Namespace
Lagring
Autoskalering
Helm
Sikkerhet
AKS i praksis
Feilsøking
Test deg selv

11 · Infrastructure as Code

Hvorfor IaC?
Terraform
Bicep vs. Terraform
Ansible
GitOps
Test deg selv

12 · CI/CD og automatisering
Pipelinen
Testpyramiden
GitHub Actions
Artefakter
Hemmeligheter og OIDC
Rollback, blue-green og canary
Hele pipelinen – kode til produksjon
Test deg selv

13 · Sikkerhet og monitoring
Del A – Sikkerhet
Del B – Monitoring og observability
Del C – Hendelser og feilsøking
Test deg selv

14 · AI-infrastruktur og MLOps
Hva er annerledes med AI?
MLOps
Trening vs. inferens
CPU vs. GPU
Servering av modeller
RAG – svar fra dine egne dokumenter
Pipelines, versjonering og overvåking
Azure ML og Microsoft Foundry
AI-sikkerhet og kostnader
Fra DevOps til MLOps
Test deg selv

15 · Tolv prosjekter
1 · Linux-server
2 · Kostnadsjeger i Python
3 · Azure-VM med kode
4 · GitHub-portefølje
5 · Docker-app
6 · Nettside på Azure Static Web Apps
7 · CI/CD til App Service
8 · Terraform med pipeline
9 · Kubernetes med Helm
10 · Komplett plattform – sluttprosjektet

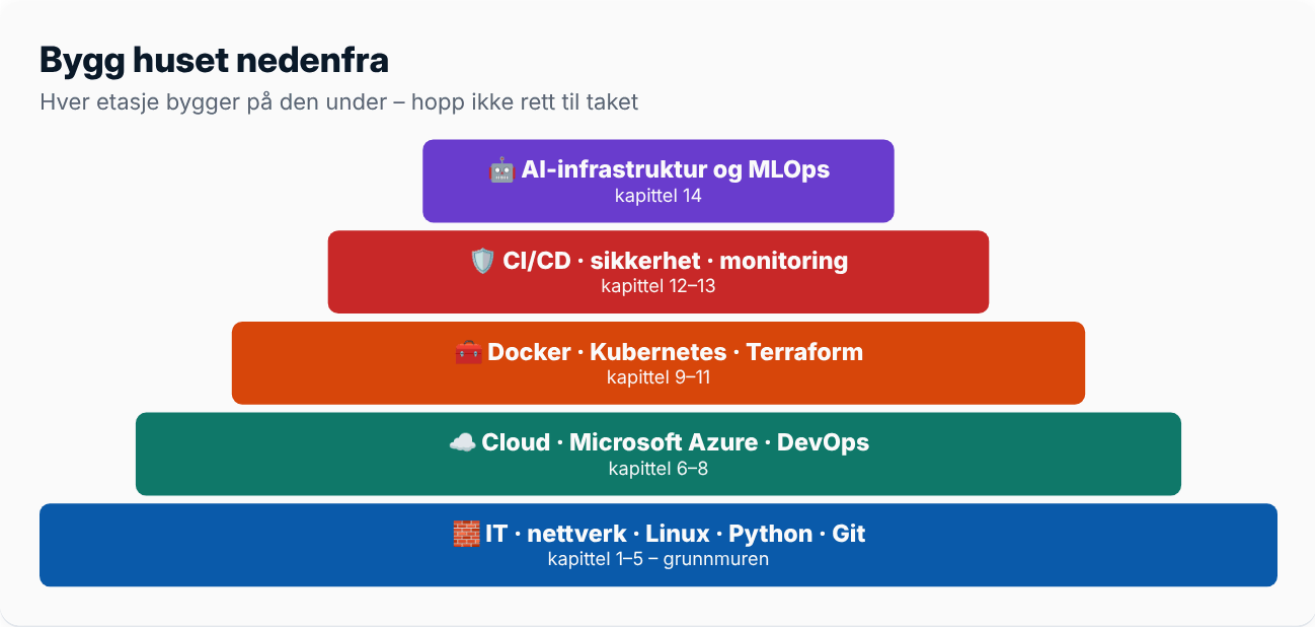
11 · MLOps
12 · Observability og game day
Slik presenterer du prosjektene

Vedlegg · Hurtigreferanser, sertifiseringer og læringsplan

A · De viktigste begrepene på ett oppslag
B · Hurtigreferanser
C · Sertifiseringer
D · Kilder
E · 12-måneders læringsplan

Slik bruker du boken

Dette er den **visuelle utgaven**: hvert tema bygges rundt en illustrasjon, med korte punkter i stedet for lange avsnitt. Vil du ha alle detaljer, finnes **fullversjonen** (samme kapitler, all tekst) som eget oppslagsverk.

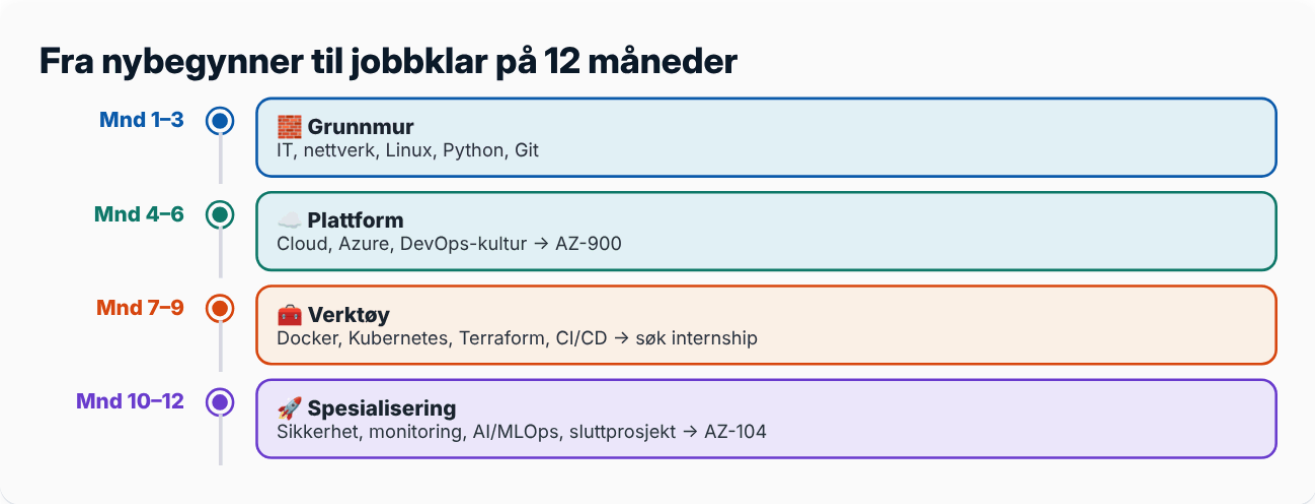


Symbolene i boken

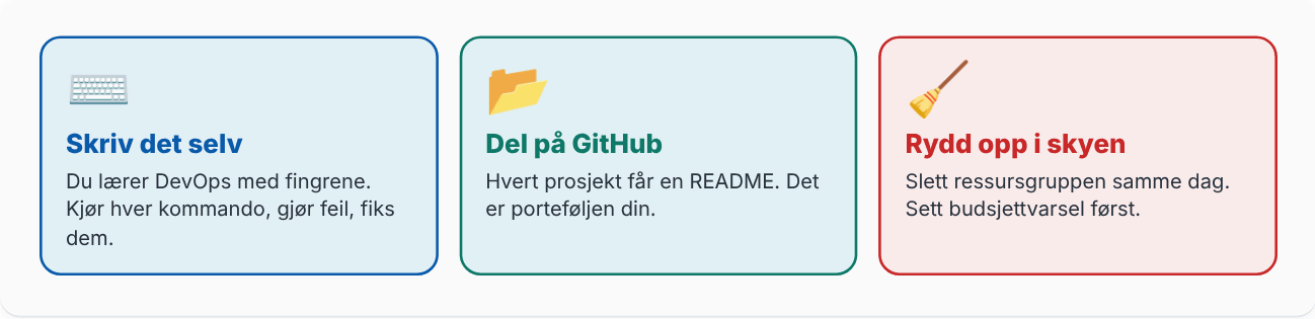


Kodeblokker: `$` betyr «skriv dette i terminalen» (ikke skriv selve `$`). Linjer uten `$` er svaret du får.

Læringsreisen



Tre regler



💰 **Kostnad:** Cloud koster ekte penger. Bruk Azure for Students eller gratiskonto, lag et **budsjettvarsel** før du starter (kapittel 7), og slett ressursene når du er ferdig.

ℹ️ **Om oppdatert informasjon:** Boken bygger på offisiell dokumentasjon slik den var kjent per **oktober 2026**. Priser, tjenestenavn og eksamensinnhold endres – sjekk alltid Microsoft Learn og annen offisiell dokumentasjon (kildeliste bakerst).

1 · Forstå IT fra bunnen

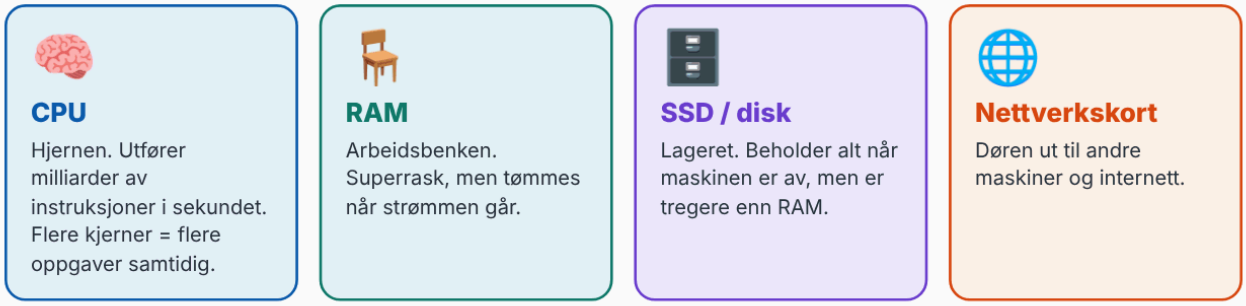
Kapittelet på én side

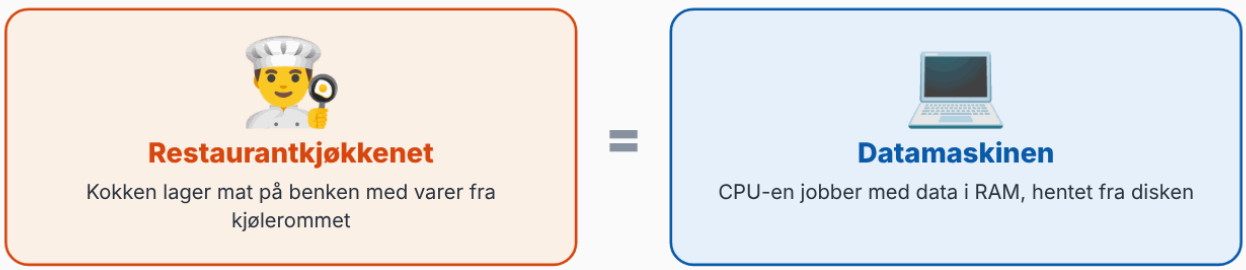


Hvordan fungerer en datamaskin?

⚡ **Kort sagt:** En datamaskin tar imot data, behandler dem, lagrer dem og gir et resultat.

De fire viktigste delene



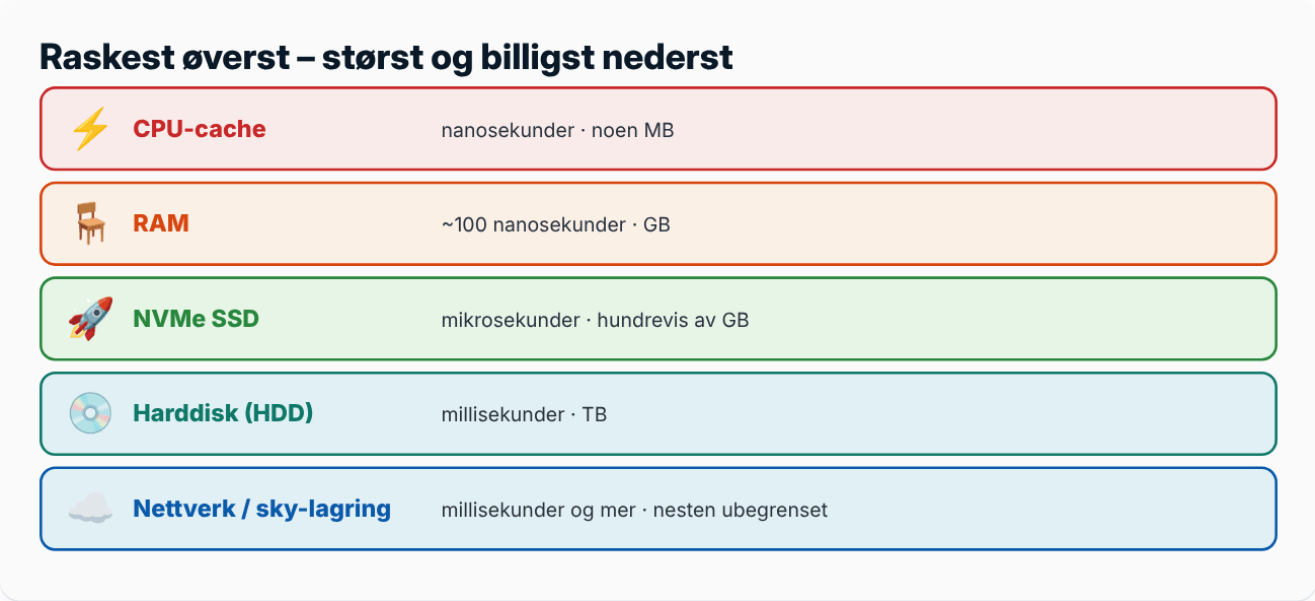


Kokk → CPU

Benken → RAM

Flere kokker → Flere kjerner

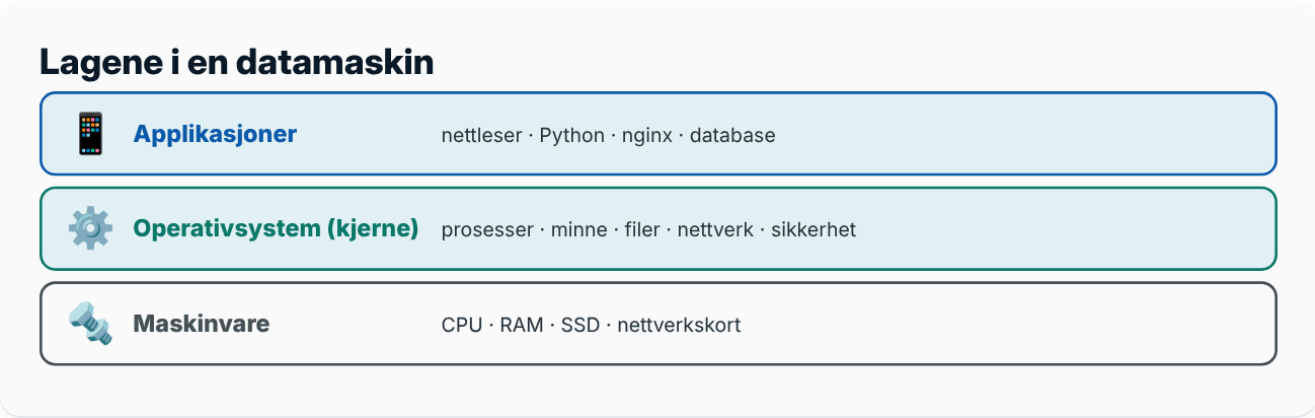
Kjølerommet → SSD / disk



🏢 **Fra arbeidslivet:** Når du lager en VM i Azure, velger du antall **vCPU** og GB **RAM**. Går en server tom for RAM, blir den treg – eller programmet drepes (`OOMKilled`).

Operativsystemet

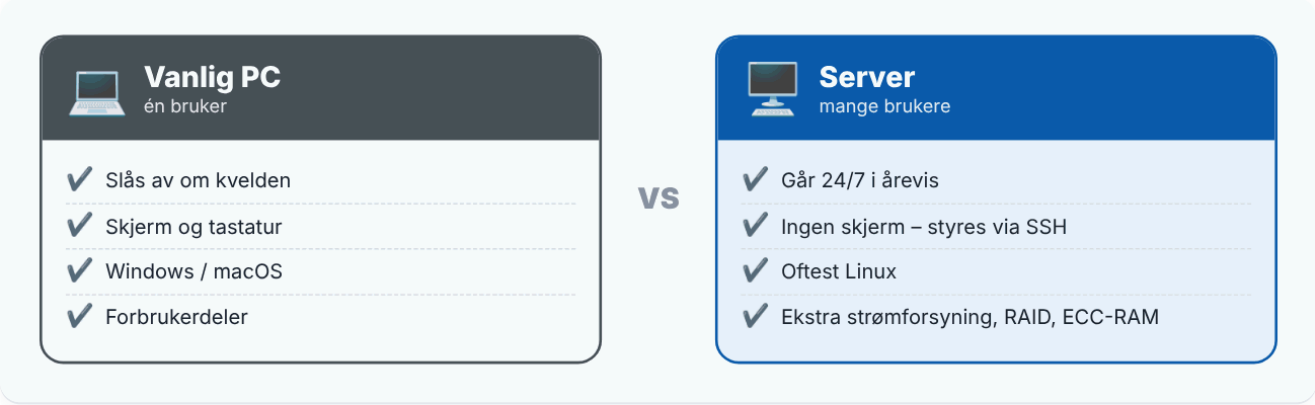
⚡ **Kort sagt:** Operativsystemet (OS) er sjefen som lar programmene dele maskinvaren trygt.



- Eksempler: **Linux** (de fleste servere), Windows, macOS.
- **Kjernen** (*kernel*) er hjertet i OS-et. Linux er egentlig navnet på en kjerne.

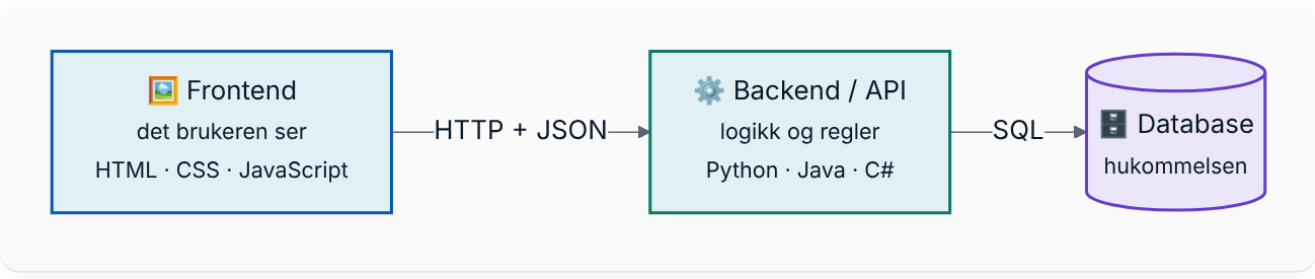
Server vs. vanlig PC

⚡ **Kort sagt:** En server er en *rolle* – en maskin som leverer tjenester til andre over nettverket.

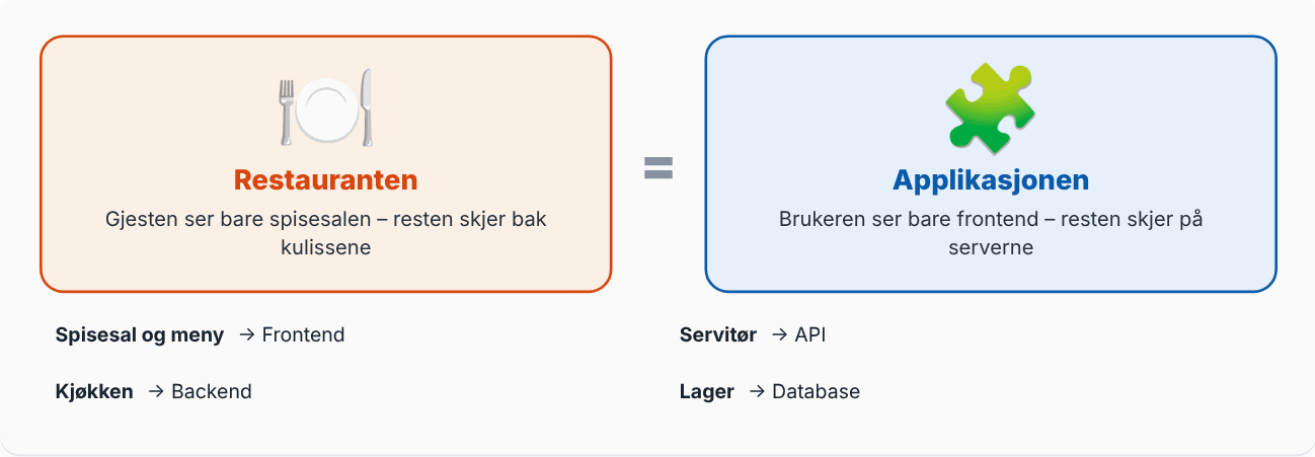




Applikasjonen: frontend, backend og database

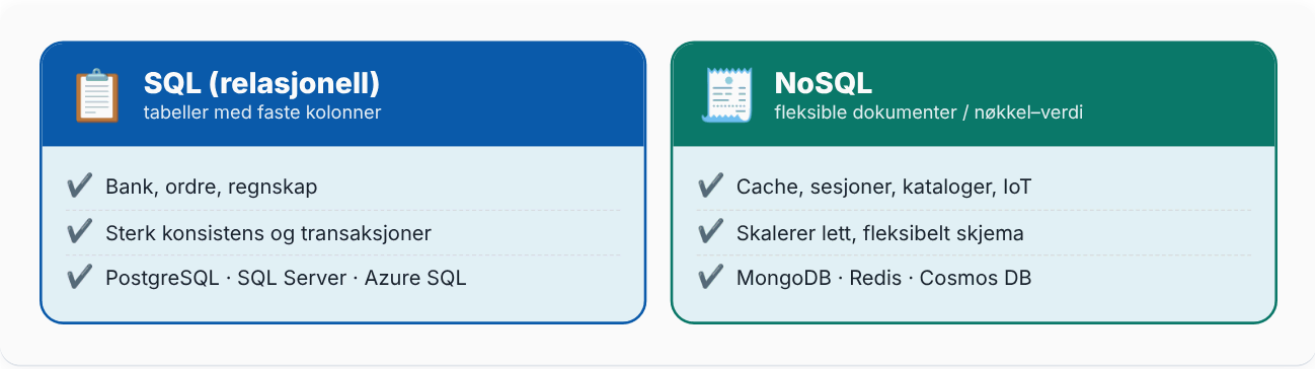


En moderne applikasjon i tre deler



Databaser

⚡ **Kort sagt:** En database lagrer og henter data trygt og raskt – også når tusenvis skriver samtidig.

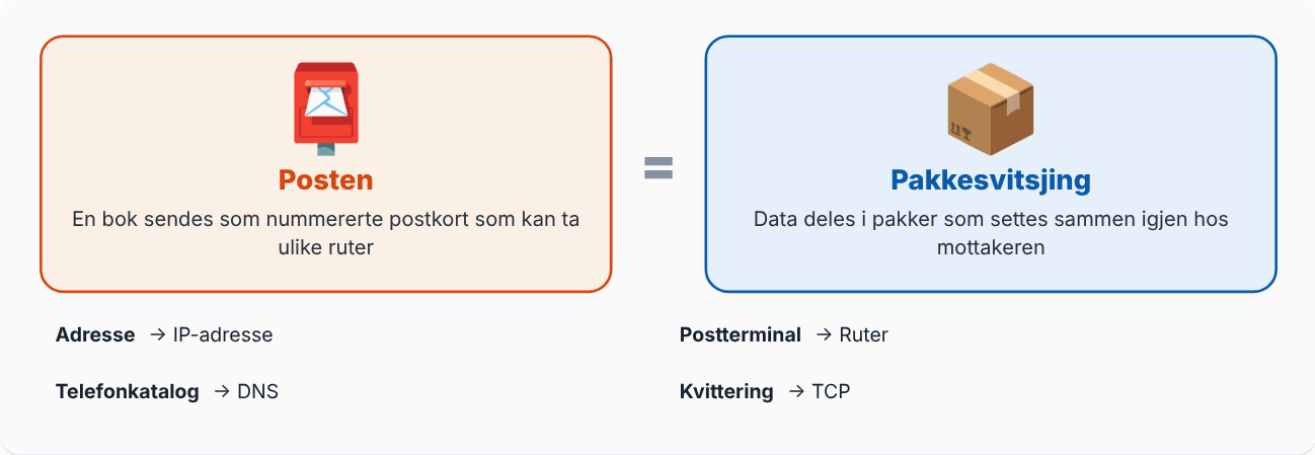


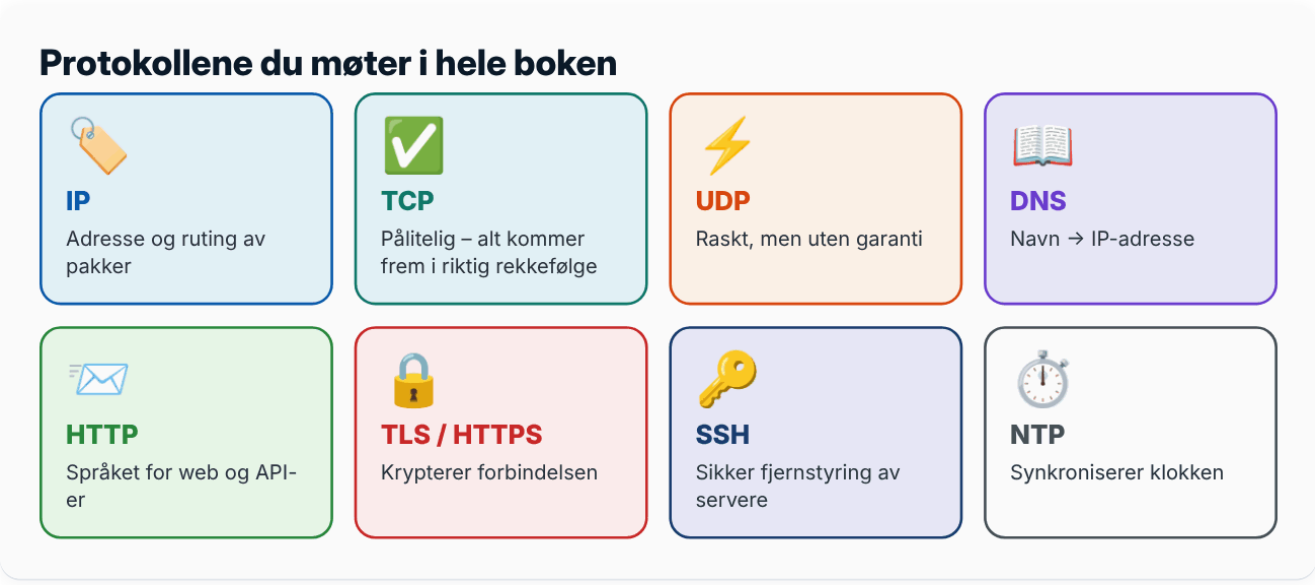

```sql
-- Hent navn og beløp for alle ordrer over 1000 kroner
SELECT kunder.navn, ordrer.beløp
FROM ordrer JOIN kunder ON kunder.id = ordrer.kunde_id
WHERE ordrer.beløp > 1000;
```

> 💡 **Husk:** En server kan bygges på nytt på minutter. Tapte data er ofte borte for alltid. **Backup – og test at du kan gjenopprette.**

Internett, IP og DNS

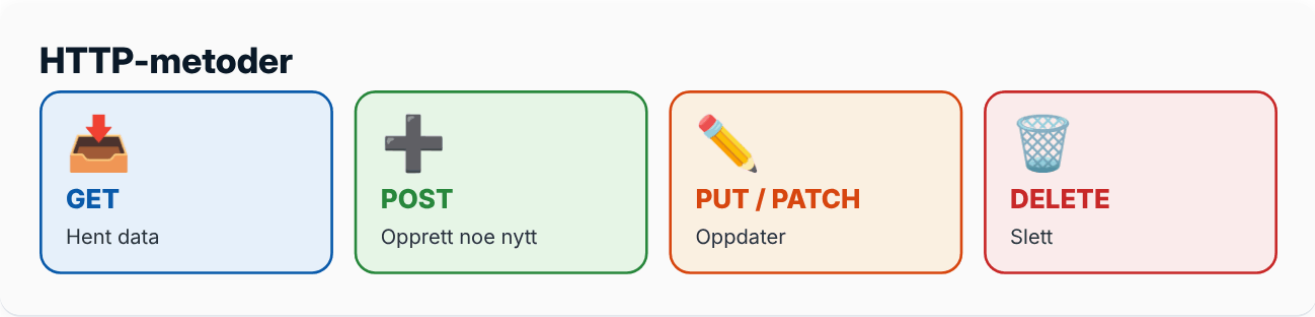
⚡ **Kort sagt:** Data deles i små **pakker** med avsender- og mottakeradresse (IP). Rutere sender dem videre som postterminaler.

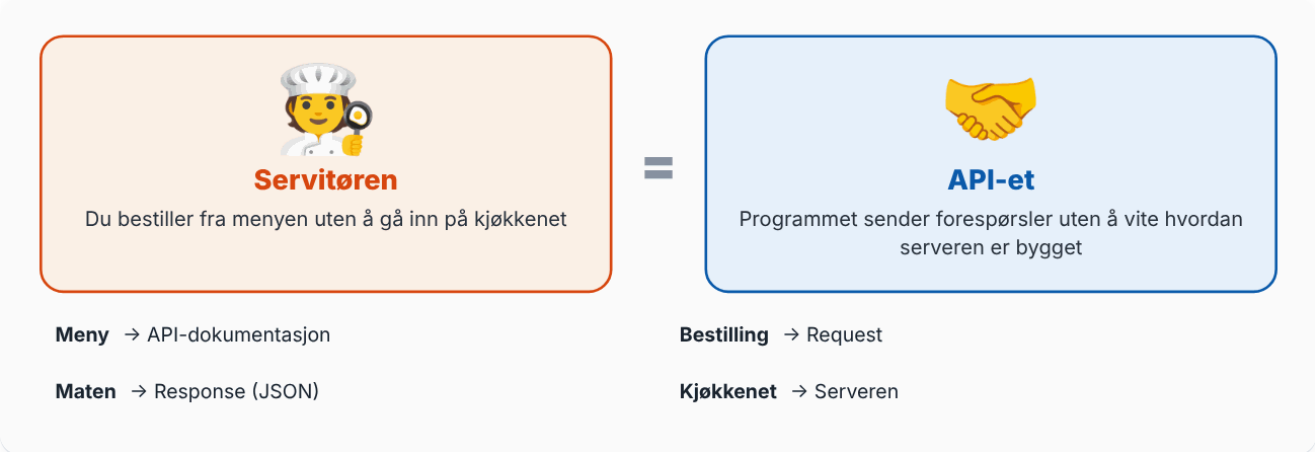




HTTP og API

⚡ **Kort sagt:** Klienten sender en **forespørsel**, serveren svarer med en **statuskode** og data.



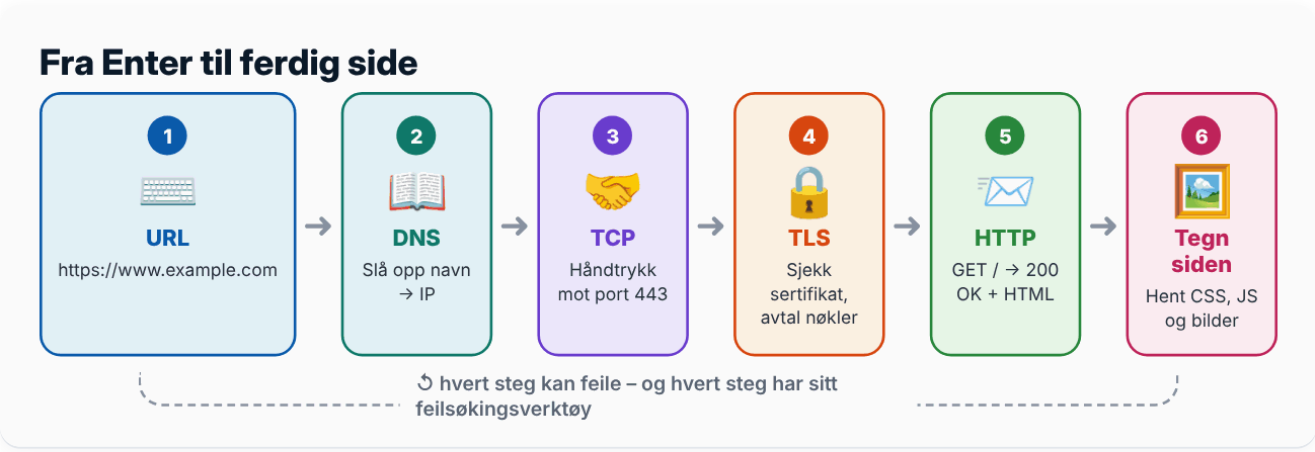


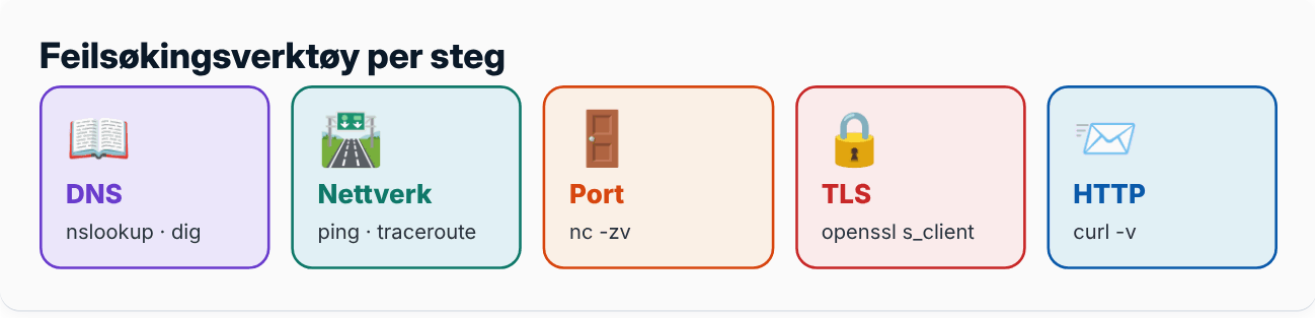

```
$ curl https://api.github.com/users/octocat
{ "login": "octocat", "name": "The Octocat", "public_repos": 8, ... }
```

🏢 **Fra arbeidslivet:** Alt i skyen er API-kall. Klikker du «Create» i Azure Portal, sender portalen et API-kall. Derfor kan **alt** i skyen automatiseres.

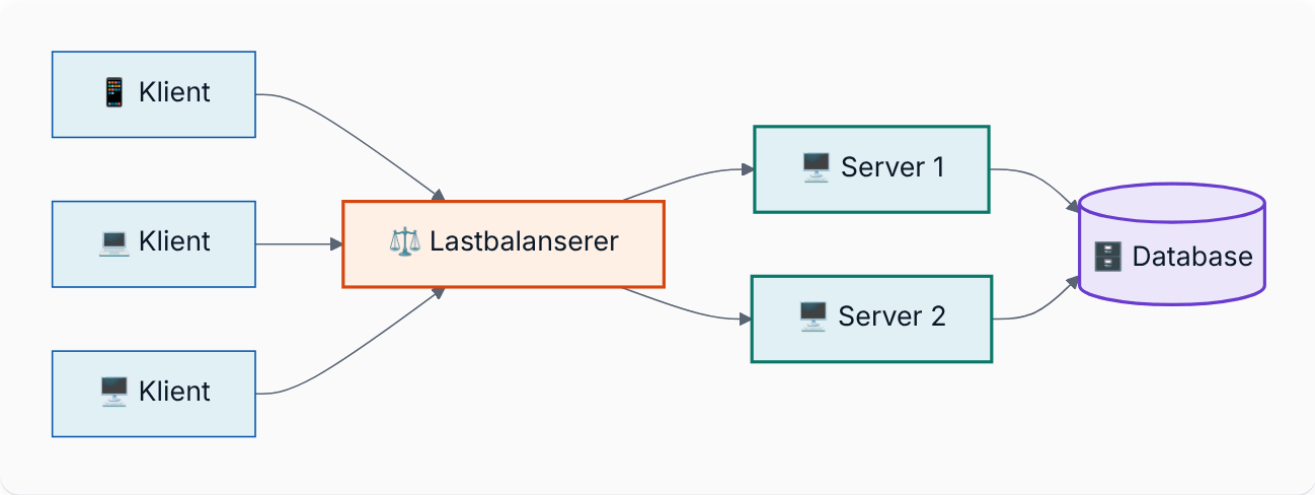
Hva skjer når du åpner en nettside?

Et klassisk intervjuspørsmål. Lær denne figuren utenat:





Klient–server og arkitekturer



Klient–server med lastbalansering – ingen enkelt server kan stoppe alt

Fra enkelt til avansert

Monolitt
Alt i ett program. Enkelt å starte.

Tre lag
Presentasjon · logikk · data

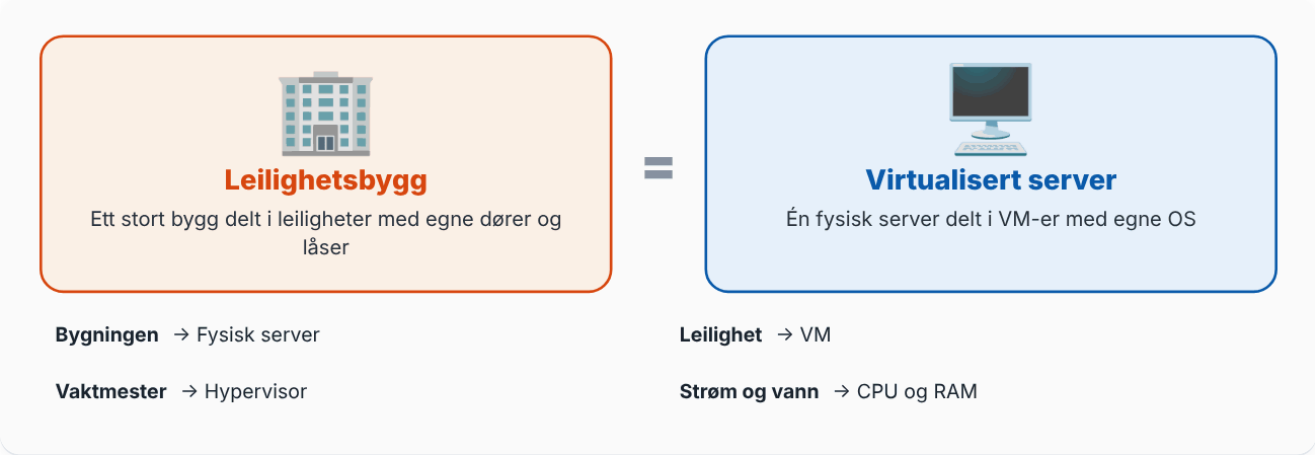
Mikrotjenester
Mange små tjenester via API. Fleksibelt, men mer å drifte – her skinner Docker og Kubernetes.

Virtualisering

⚡ **Kort sagt:** En **hypervisor** deler én fysisk server i mange **virtuelle maskiner** (VM-er), hver med eget operativsystem.

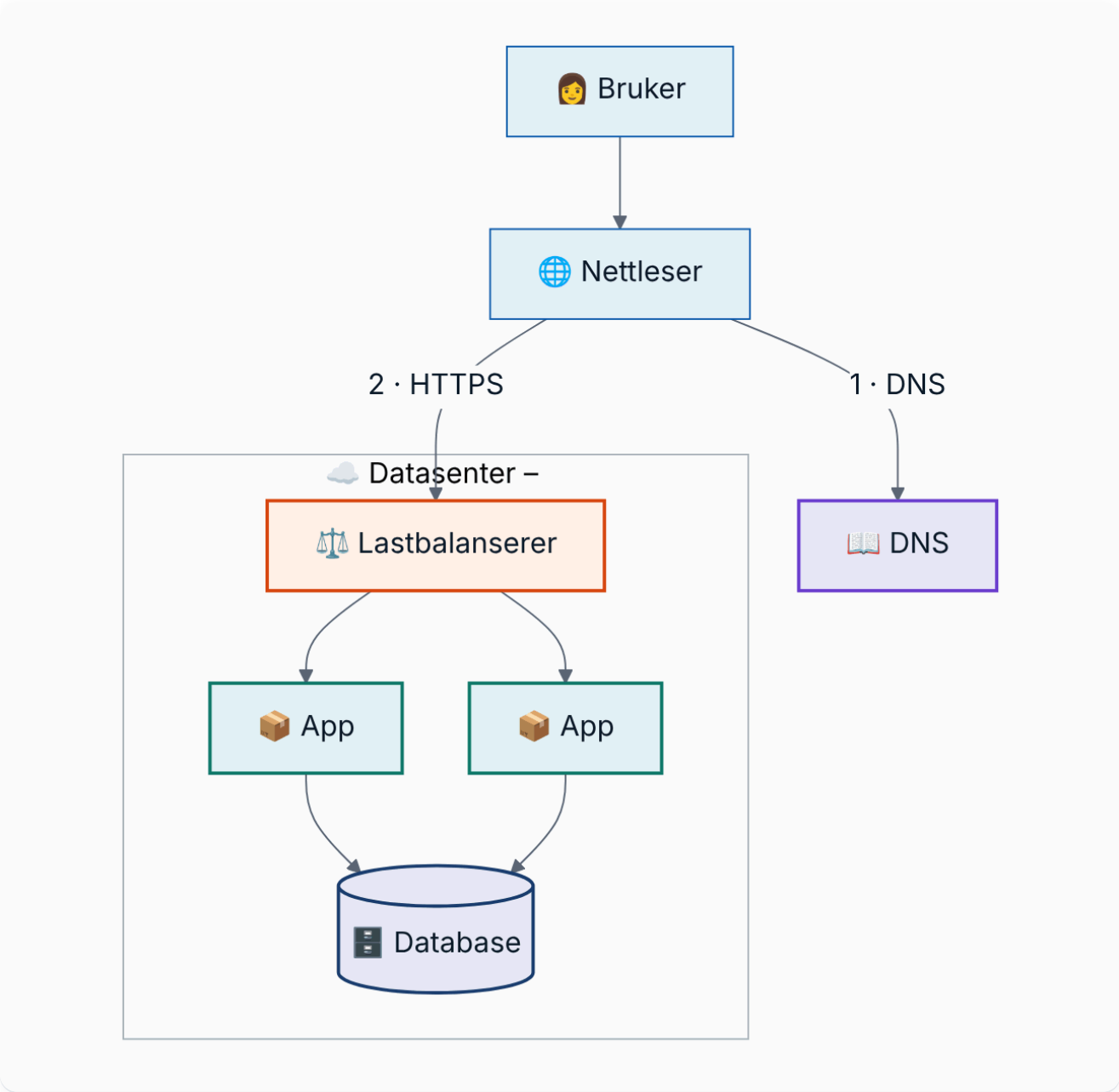
Virtualisering

VM 1 · VM 2 · VM 3	hver med egen app og eget OS (Ubuntu, Windows, Red Hat)
Hypervisor	Hyper-V · KVM · VMware ESXi – fordeler ressursene
Fysisk server	CPU · RAM · disk · nettverk



🏢 **Fra arbeidslivet:** Hele Azure er bygget på virtualisering. En Azure-VM er en virtuell maskin på en fysisk server i et Microsoft-datasenter.

Slik henger alt sammen



Hele kjeden – alt i boken handler om å bygge, automatisere, sikre og overvåke denne

Test deg selv

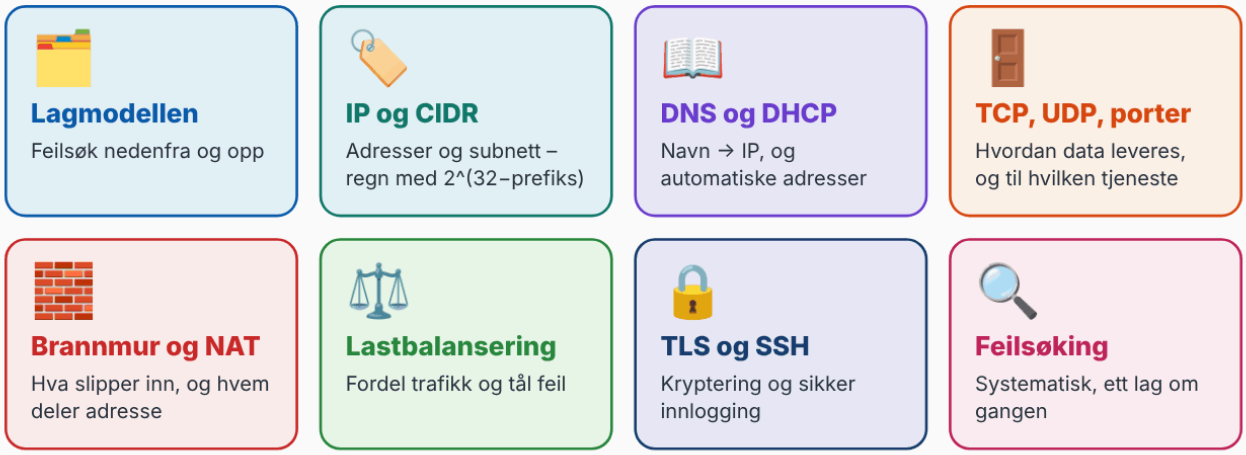
1. Hva er forskjellen på RAM og SSD? → *RAM er raskt korttidsminne som tømmes; SSD beholder data, men er tregere.*
2. Hva gjør DNS? → *Oversetter navn til IP-adresser.*
3. Hva betyr statuskode 404? → *Ressursen finnes ikke.*
4. Hva er et API? → *En avtalt måte for programmer å snakke sammen – som en servitør.*
5. Hva er en hypervisor? → *Programvaren som kjører virtuelle maskiner på en fysisk server.*

6. Hvorfor kan alt i skyen automatiseres? → *Fordi alt – også klikk i portalen – er API-kall.*

Prøv selv: `nslookup www.noroff.no` · `curl -I https://www.noroff.no` · installer Ubuntu i VirtualBox eller WSL.

2 · Nettverk for DevOps

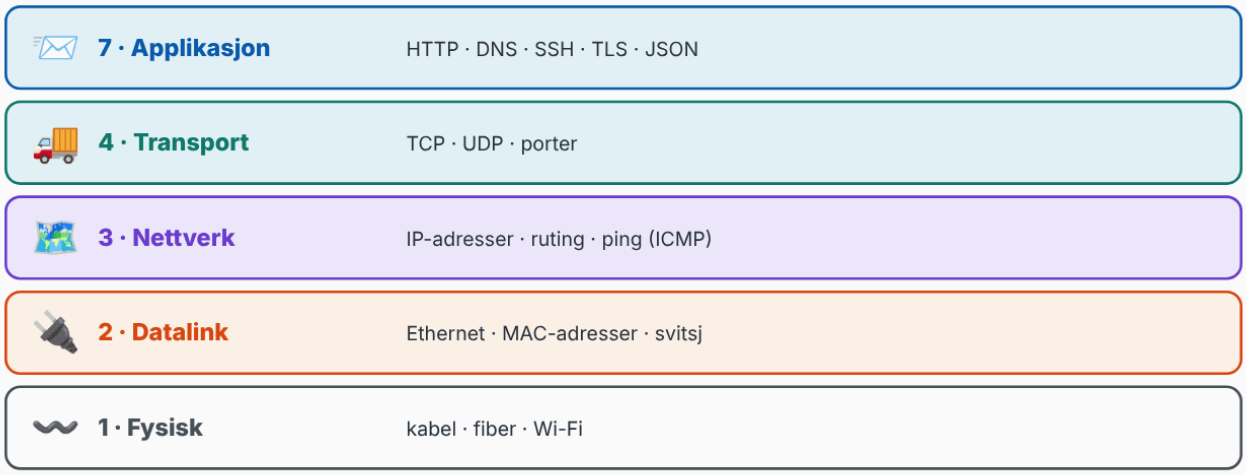
Kapittelet på én side



🏢 **Hvorfor dette er viktig:** «Det virker ikke» betyr veldig ofte at noe i nettverket er feil – en brannmurregel, DNS, en port, et subnett. I Azure designer du dette selv.

Lagmodellen (OSI / TCP-IP)

Nettverket i lag – hvert lag har én jobb



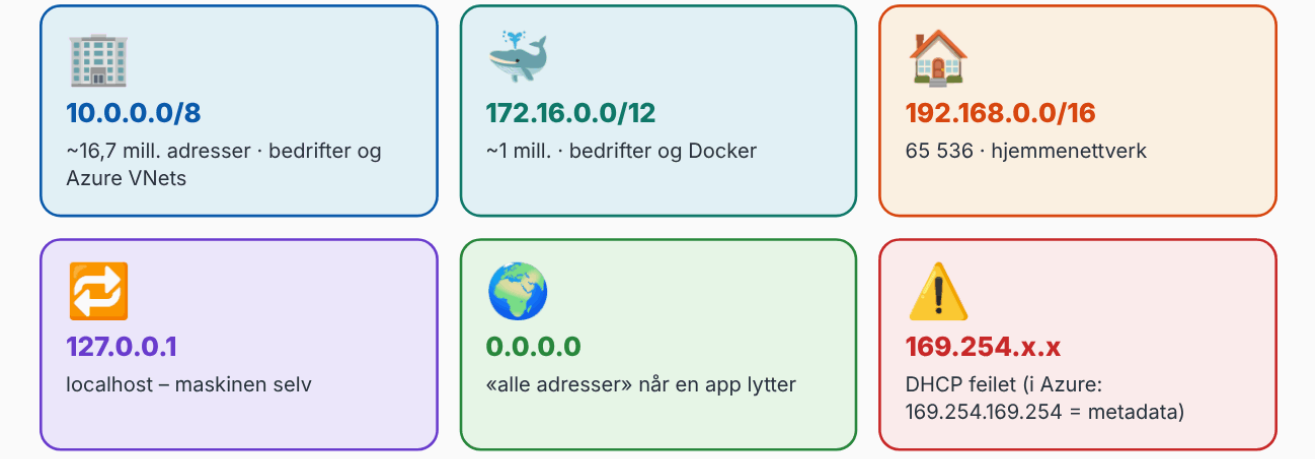


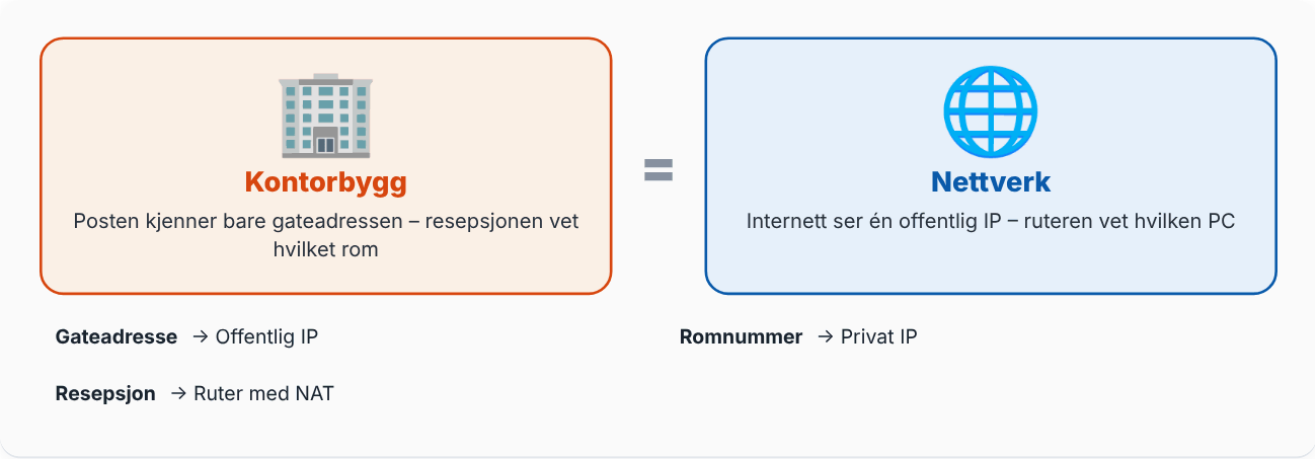
🏢 **Fra arbeidslivet:** «Lag 4-lastbalanserer» = ser bare IP og port (Azure Load Balancer). «Lag 7» = forstår HTTP og URL-er (Application Gateway).

IP-adresser



Private adresser (brukes inne i nett – rutes aldri på internett)

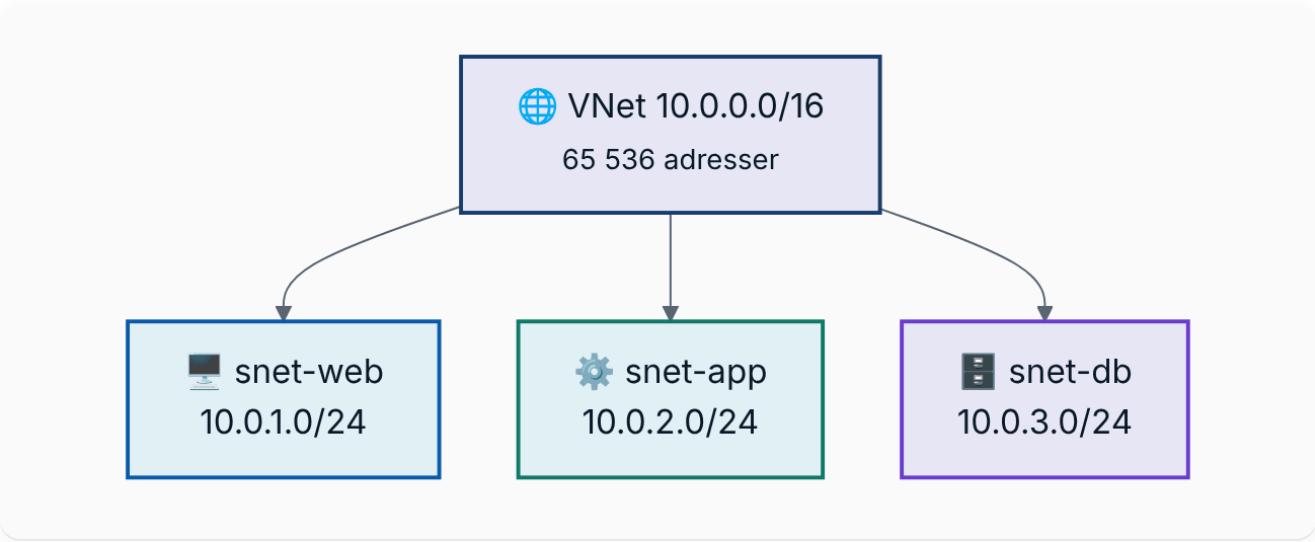




Subnetting og CIDR

⚡ **Kort sagt:** Tallet etter `/` sier hvor mange bit som er nettverksdelen. Resten er til maskiner.





Ett VNet delt i tre subnett – web, app og database

Azure reserverer 5 adresser i hvert subnett (eksempel 10.0.1.0/24)

0️⃣ **.0**
Nettverksadresse

🚪 **.1**
Standard gateway

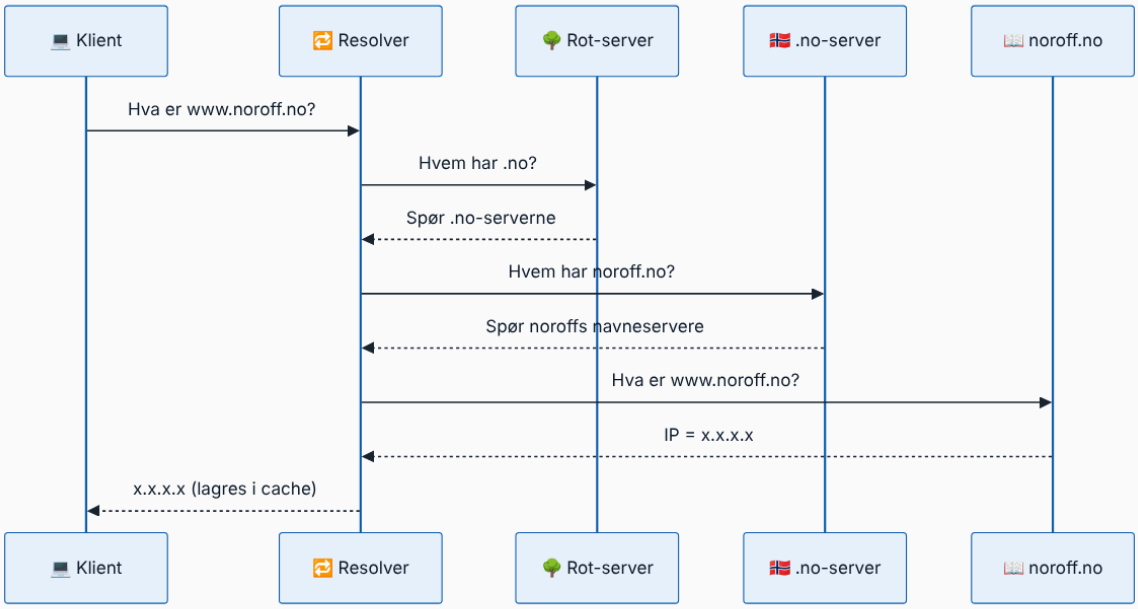
📖 **.2 og .3**
Azure DNS

📢 **.255**
Broadcast

✅ **251**
brukbare adresser igjen

⚠️ **Vanlig feil:** VNets som overlapper (f.eks. `10.0.0.0/16` og `10.0.128.0/17`) kan ikke kobles sammen. Planlegg adresser romslig fra start – særlig for Kubernetes.

DNS – internettets telefonkatalog



Et DNS-oppslag – svaret caches i TTL sekunder

De viktigste DNS-postene

🅰️ **A / AAAA**
Navn → IPv4 / IPv6

🔗 **CNAME**
Navn → annet navn (alias)

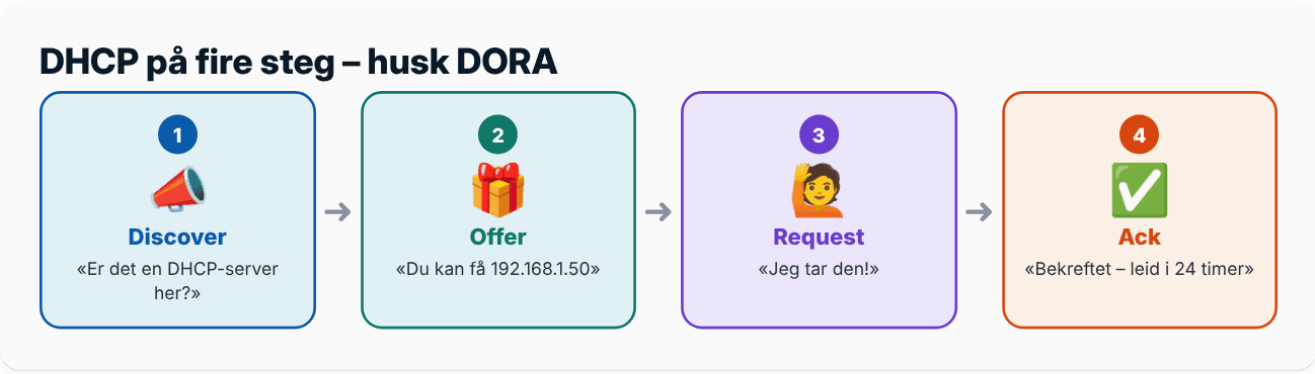
✉️ **MX**
E-postserver for domenet

📝 **TXT**
Tekst – ofte verifisering

```
$ dig +short www.example.com      # bare svaret
$ dig example.com MX              # hvilken e-postserver?
```

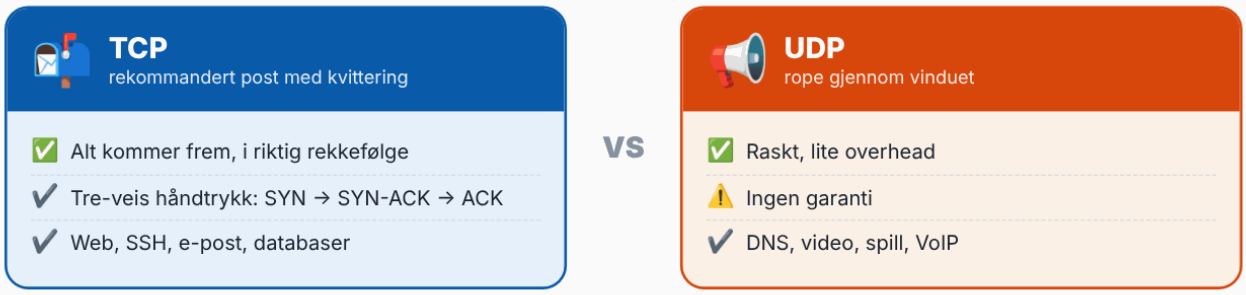
⚠️ **Vanlig feil:** «Jeg endret DNS, men ingenting skjer!» – gamle svar lever i cache til **TTL** går ut. «It's always DNS» er et ekte ordtak.

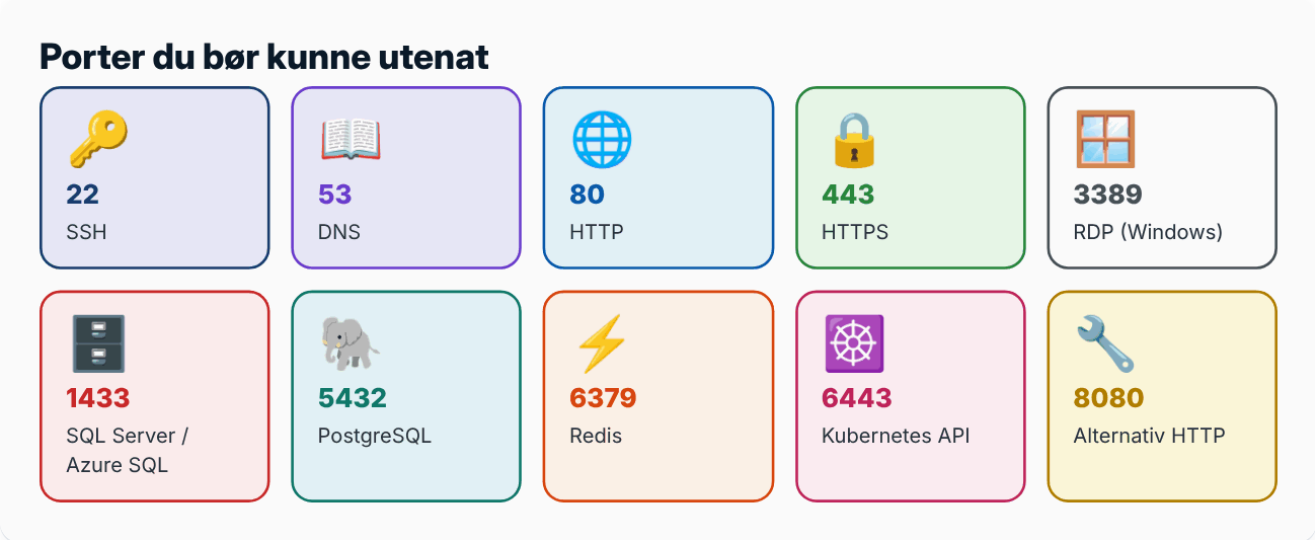
DHCP – automatiske adresser



🍕 **Hverdagsanalogi:** Hotellresepsjonen gir deg et rom (adresse) for et visst antall netter (*lease*).

TCP, UDP og porter

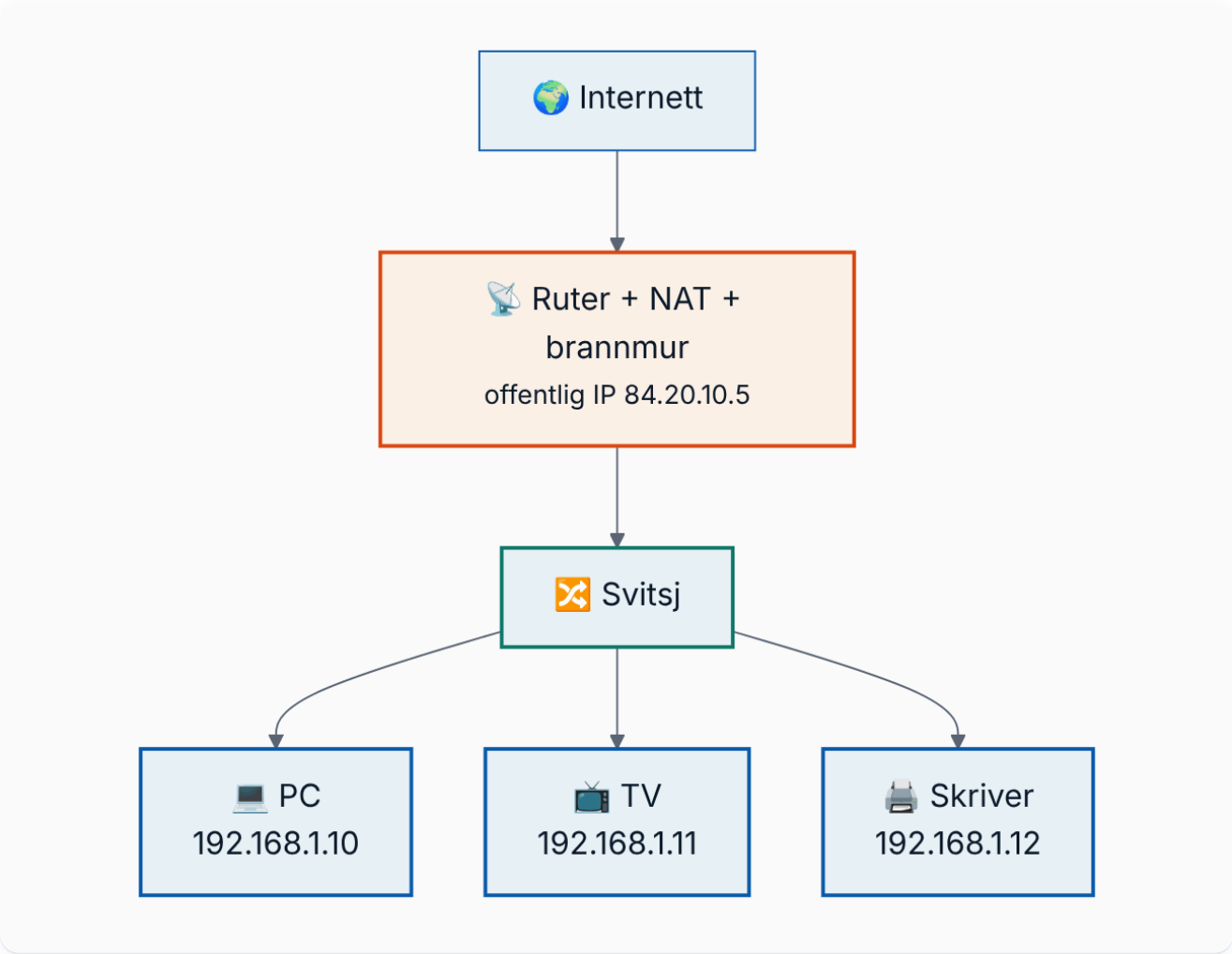




🍕 **Hverdagsanalogi:** IP-adressen er **bygget**, porten er **døren**.

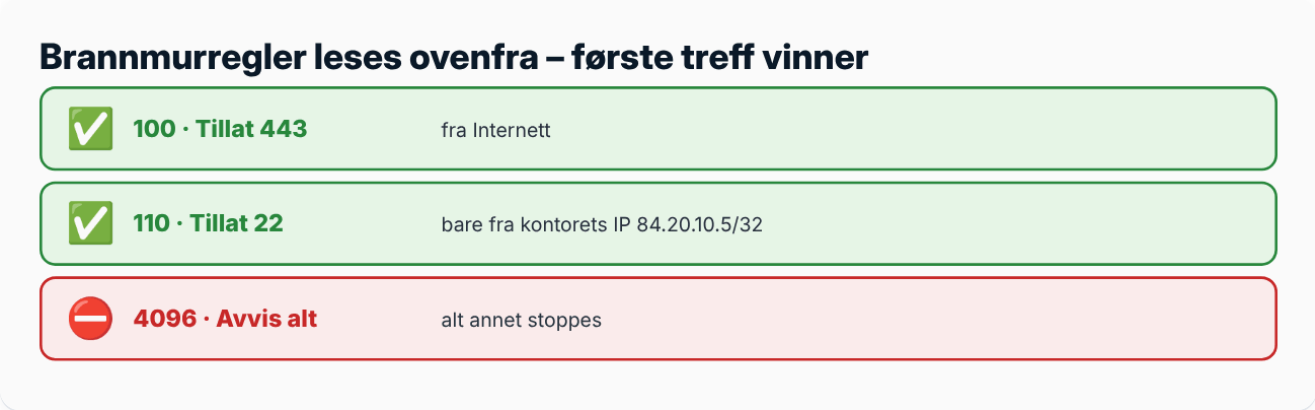
⚠️ **Vanlig feil:** En app som lytter på `127.0.0.1:8080` kan bare nås fra maskinen selv. Den må lytte på `0.0.0.0:8080` for å nås utenfra – klassiker i Docker.

Ruter, svitsj, brannmur og NAT



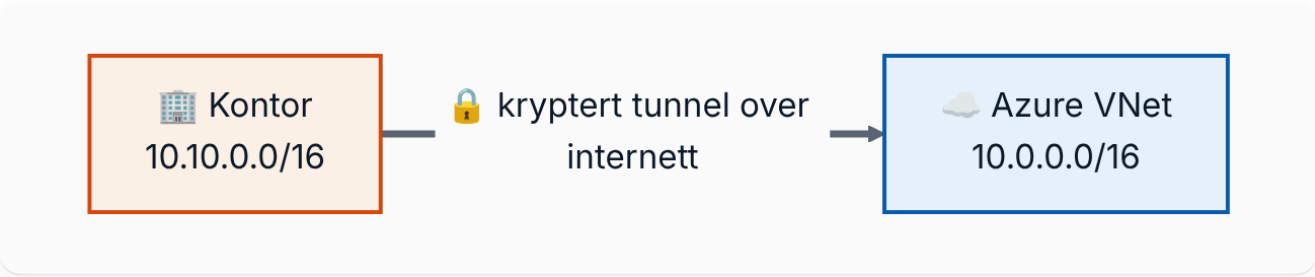
Hjemmenettet – svitsj kobler enheter, ruter kobler nettverk





💰 **Sikkerhet:** Åpne **aldri** SSH (22) eller RDP (3389) mot hele internett. Roboter prøver å bryte seg inn hele døgnet.

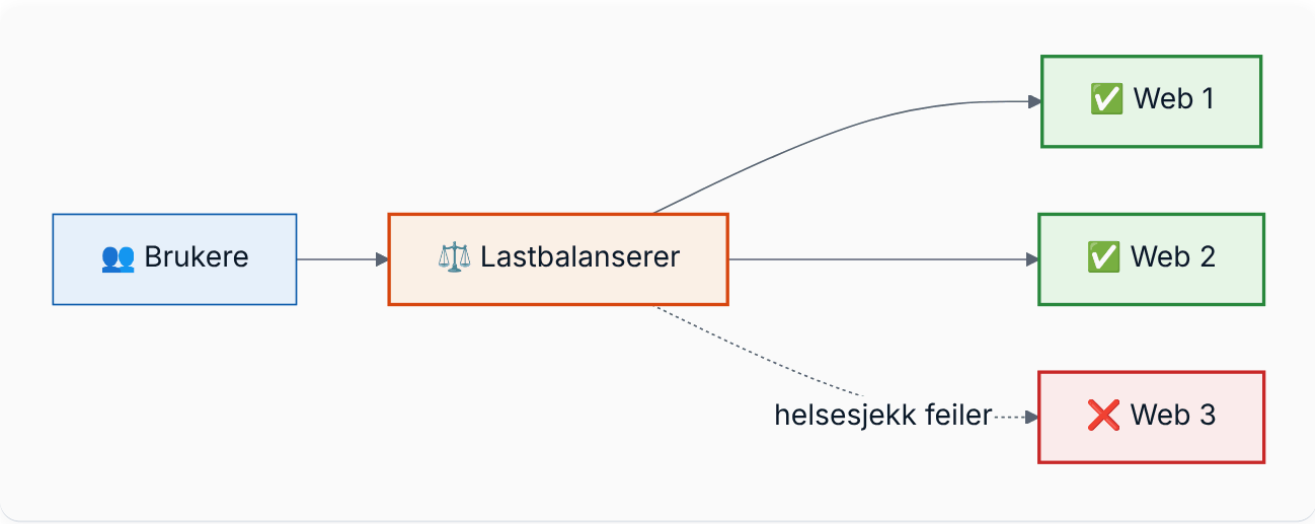
VPN



Site-to-site VPN – kryptert tunnel mellom kontoret og Azure (adresseområdene må ikke overlappe)

- **Site-to-site:** hele kontoret ↔ Azure (Azure VPN Gateway).
- **Point-to-site:** én PC ↔ nettverket.
- **ExpressRoute:** privat linje til Azure – ikke over internett.

Lastbalansering og reverse proxy



Lastbalansereren sjekker helsen og sender bare trafikk til friske servere

4 Lag 4
ser IP og port

- ✔ Rask fordeling av TCP/UDP
- ✔ Azure Load Balancer

7 Lag 7
forstår HTTP

- ✔ Ruting på URL og vertsnavn
- ✔ TLS og WAF
- ✔ Application Gateway · Front Door

Forward proxy
står foran klientene

- ✔ Sender forespørsler ut for dem
- ✔ Filtrering og logging i bedrifter

vs

Reverse proxy
står foran serverne

- ✔ Tar imot for dem: TLS, ruting, cache
- ✔ nginx · Traefik · Application Gateway · Ingress

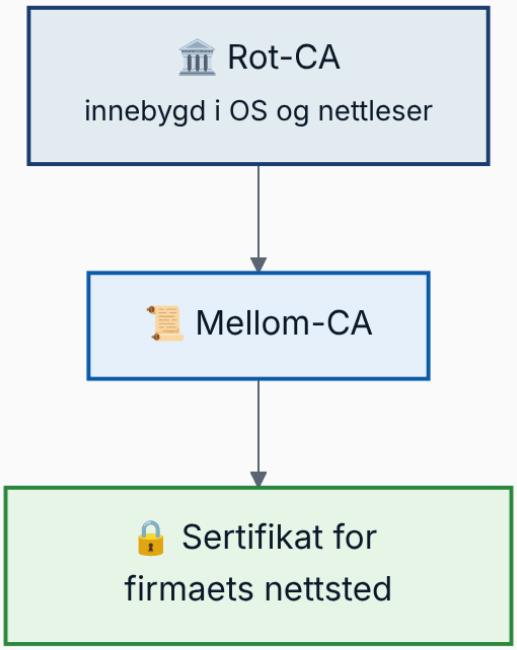
TLS og sertifikater

TLS gir tre ting

Konfidensialitet
Ingen kan lese trafikken

Integritet
Ingen kan endre den i det skjulte

Autentisering
Sertifikatet beviser at det er riktig server



Tillitskjeden – nettleseren stoler på rot-CA-en, som går god for resten

⚠️ **Vanlig feil: Utløpte sertifikater** er en av de vanligste årsakene til nedetid. Automatiser fornyelse og overvåk utløpsdatoer.

HTTP-statuskoder

Første siffer forteller hvem som har feilen

✅ **2xx – OK**
200 OK · 201 Created · 204 No Content

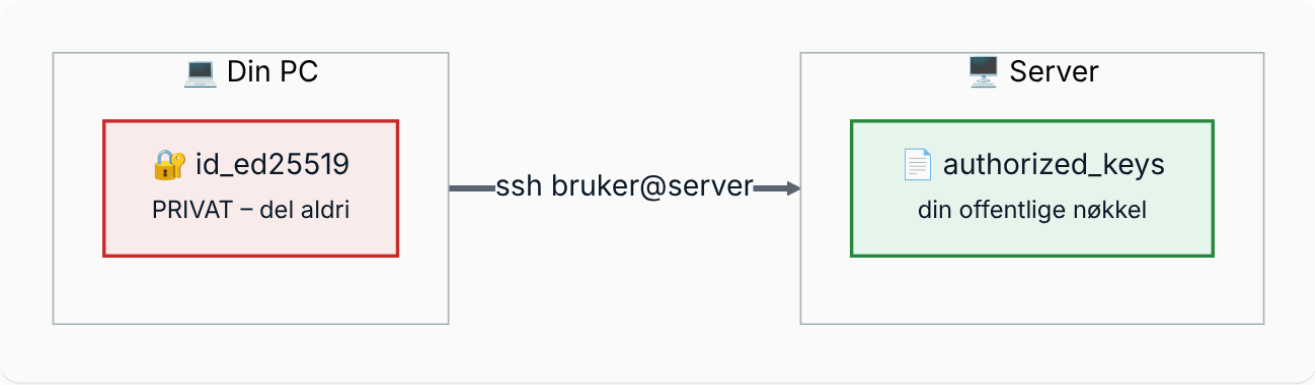
↪️ **3xx – Videre**
301 flyttet · 302/307 midlertidig · 304 bruk cache

🙋 **4xx – Klientfeil**
400 feil format · 401 logg inn · 403 ikke lov · 404 finnes ikke · 429 for mange

🔥 **5xx – Serverfeil**
500 krasj · 502 backend svarer ikke · 503 overbelastet · 504 timeout

💡 **Husk: 401** = «hvem er du?» · **403** = «jeg vet hvem du er, men nei». **502** fra en proxy = appen *bak* svarer ikke.

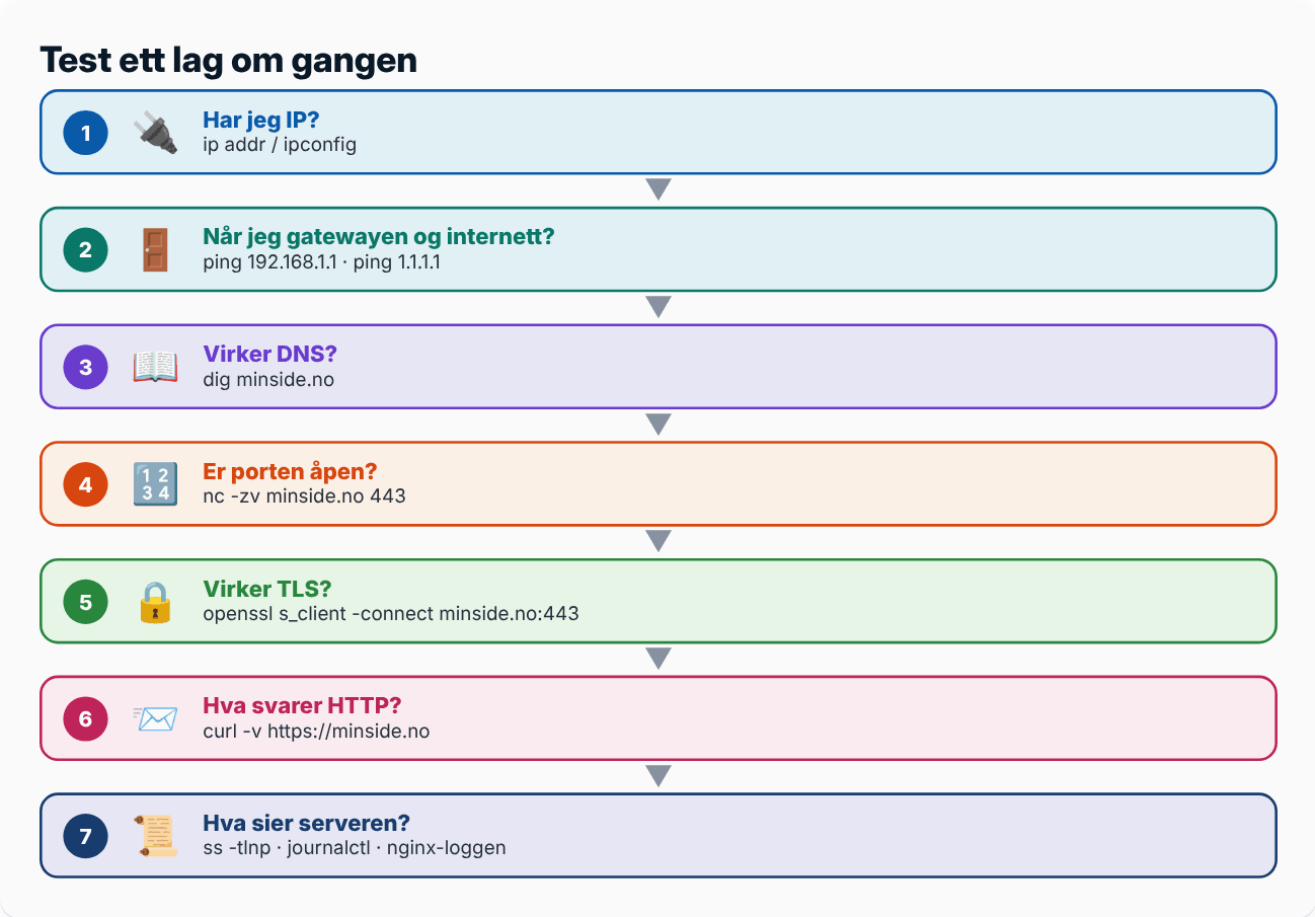
SSH med nøkler

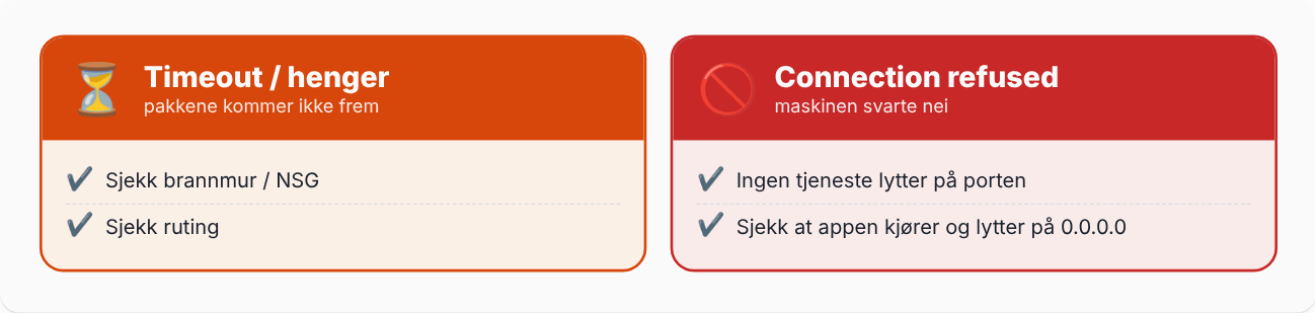


SSH-nøkler – den private nøkkelen forlater aldri PC-en din

```
$ ssh-keygen -t ed25519 -C "student@noroff"   # lag nøkkelpar
$ ssh-copy-id student@203.0.113.10             # legg offentlig nøkkel på serveren
$ ssh student@203.0.113.10                     # logg inn
```

Feilsøking – nedenfra og opp





Test deg selv

1. Hvor mange adresser er det i `/26`? → *$2^6 = 64$ (59 brukbare i Azure).*
2. Er `172.32.0.1` privat? → *Nei – 172.16.0.0/12 slutter på 172.31.255.255.*
3. A-post vs. CNAME? → *A peker til IP; CNAME peker til et annet navn.*
4. Hvilke porter bruker HTTPS, SSH og PostgreSQL? → *443, 22, 5432.*
5. «Refused» vs. «timeout»? → *Refused: ingen lytter. Timeout: blokkert/droppet.*
6. Kan `10.0.0.0/16` og `10.0.128.0/17` peeres? → *Nei – de overlapper.*

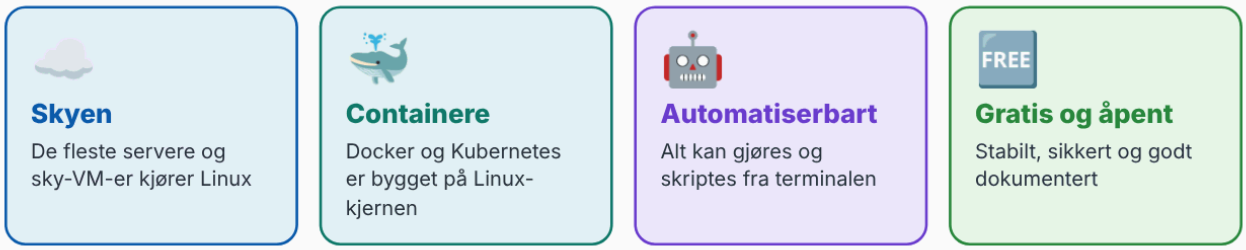
Prøv selv: `dig noroff.no NS` · `traceroute vg.no` · `ss -tlnp` · lag et SSH-nøkkelpar.

3 · Linux

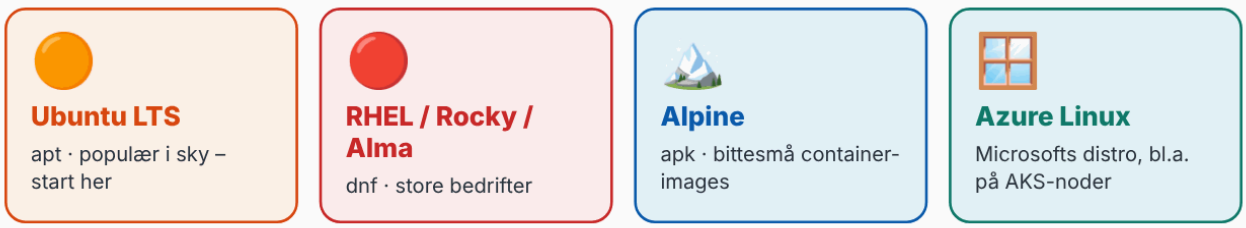
Kapittelet på én side



Hvorfor Linux?



Vanlige distribusjoner



> 💡 **Øv gratis:** WSL på Windows (`wsl --install`), en VM i VirtualBox, eller **Azure Cloud Shell** i nettleseren.

Terminalen

Anatomien til en kommando: ls -la /var/log

ls
kommandoen – hva skal gjøres

-la
flagg – endrer oppførselen (l = lang liste, a = alle)

/var/log
argument – hva kommandoen jobber med

Overlevelsestaster

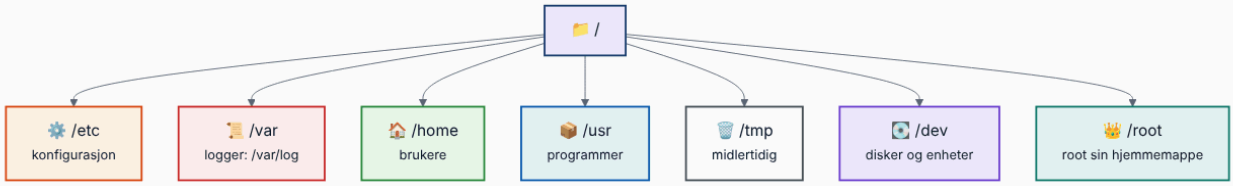
Tab
Fullfør navn – bruk hele tiden!

Pil opp
Forrige kommando

Ctrl + C
Avbryt

Ctrl + R
Søk i historikken

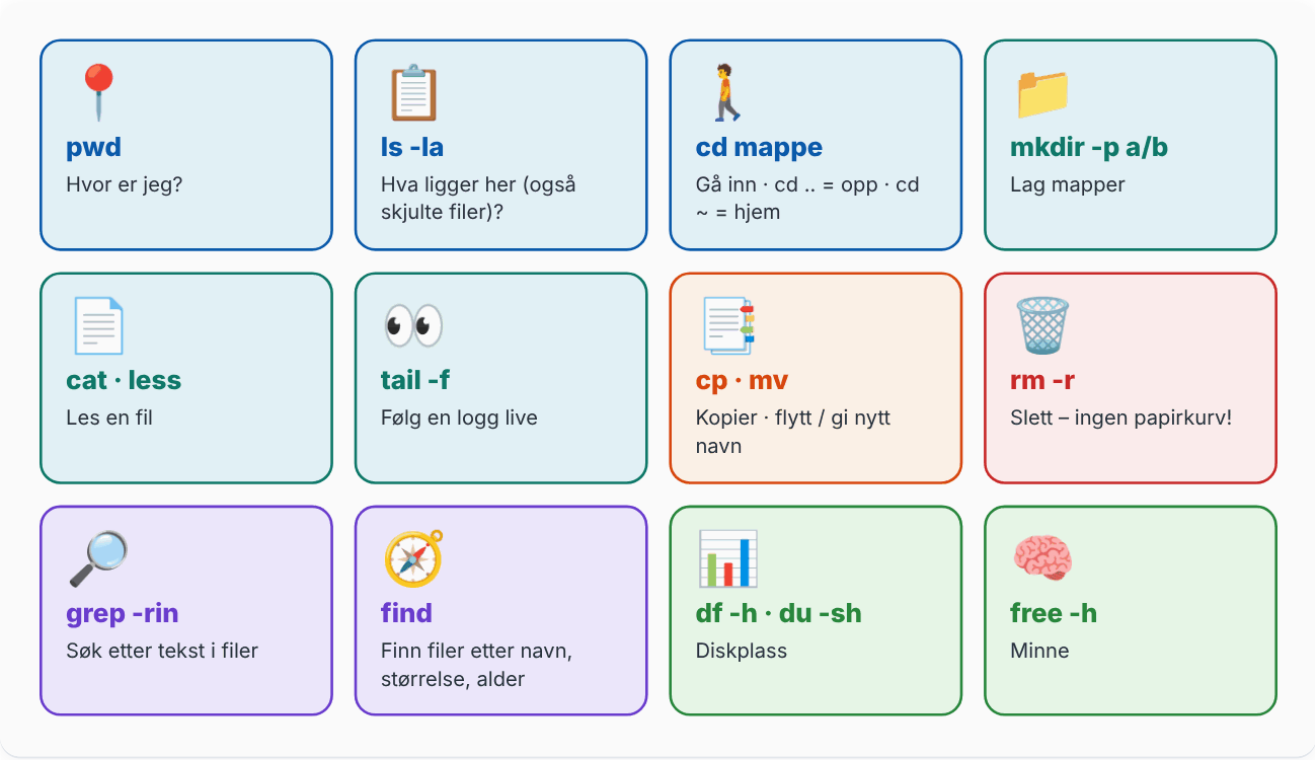
Filsystemet



Alt starter i roten /

> 💡 **Husk tre mapper:** `/etc` = konfigurasjon · `/var/log` = logger · `/home` = brukerfiler.

Kommandoene du må kunne



Pipes – superkraften




```
$ cat access.log | awk '{print $1}' | sort | uniq -c | sort -rn | head -5
$ echo "hei" > fil.txt        # > overskriver
$ echo "mer" >> fil.txt       # >> legger til
$ kommando > logg.txt 2>&1    # både utskrift og feil til fil
```

⚠️ **Vanlig feil:** `rm -rf` med en tom variabel kan slette alt. Dobbeltsjekk alltid stien.

Brukere og rettigheter

Les rettighetene: -rwxr-xr-- student utviklere deploy.sh



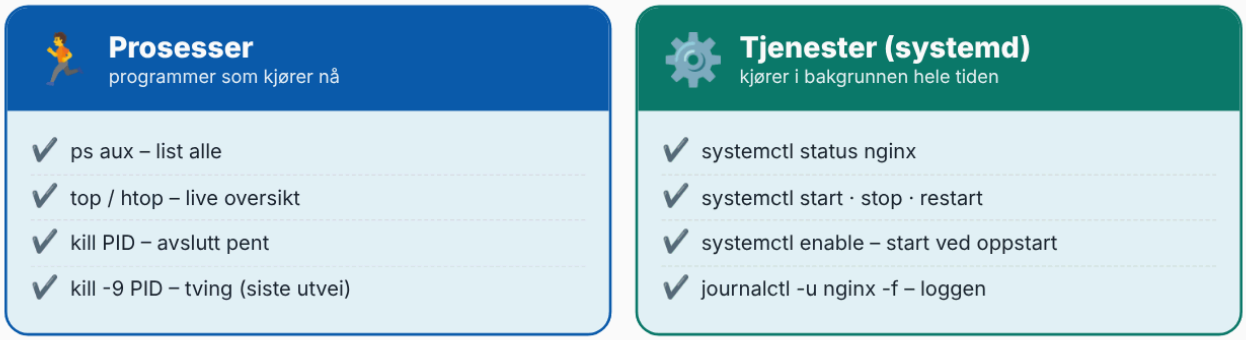
Tallene: r = 4 · w = 2 · x = 1 – legg sammen per gruppe




```
$ chmod +x deploy.sh                    # gjør kjørbar
$ chmod 600 ~/.ssh/id_ed25519           # bare eier
$ sudo chown www-data:www-data -R /var/www/side
$ sudo useradd -m -s /bin/bash kari     # ny bruker
$ sudo usermod -aG sudo kari            # legg i gruppe (husk -a!)
```

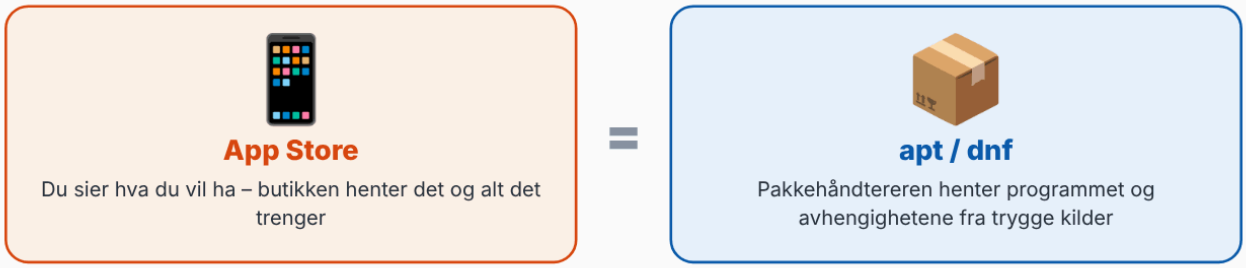
💡 **Husk:** Bruk en vanlig bruker + `sudo`, ikke root. Og aldri `chmod 777` for å «få det til å virke».

Prosesser og tjenester



⚠️ **Vanlig feil:** `start` uten `enable` – tjenesten forsvinner ved neste omstart. Bruk `systemctl enable --now`.

Pakkehåndtering



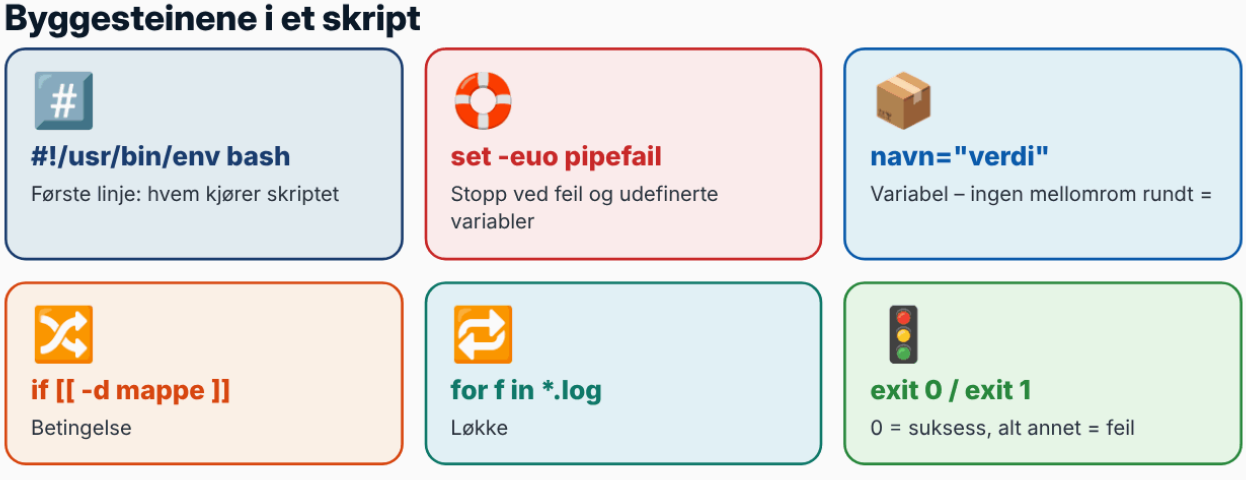

```
$ sudo apt update                # oppdater pakkelisten (glem ikke!)
$ sudo apt install -y nginx      # installer
$ sudo apt upgrade               # oppgrader alt
```

Miljøvariabler

```
$ export APP_ENV=production      # sett
$ echo $APP_ENV                  # les
$ echo $PATH                     # mappene shellet leter etter programmer i
```

🏢 **Fra arbeidslivet:** Samme app kjører i test og produksjon – bare med forskjellige miljøvariabler. Hemmeligheter skal **ikke** ligge i `.bashrc` eller Git.

Bash-skript



```bash
#!/usr/bin/env bash
set -euo pipefail
MAL="/var/backups/side"
mkdir -p "$MAL"
tar -czf "$MAL/side-$(date +%F).tar.gz" /var/www/side     # lag backup
find "$MAL" -name "*.tar.gz" -mtime +7 -delete            # slett eldre enn 7 dager
```

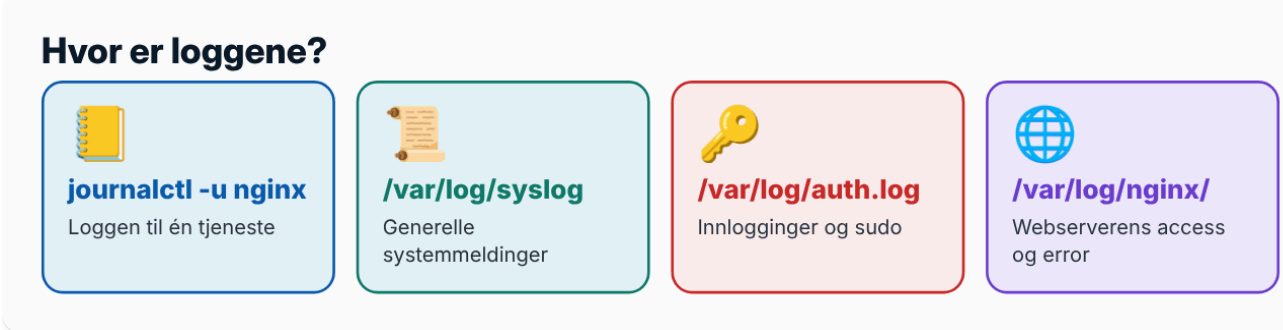
💡 **Husk:** Alltid `"$variabel"` med anførselstegn – og kjør **ShellCheck** på skriptene dine.

Cron – planlagte jobber

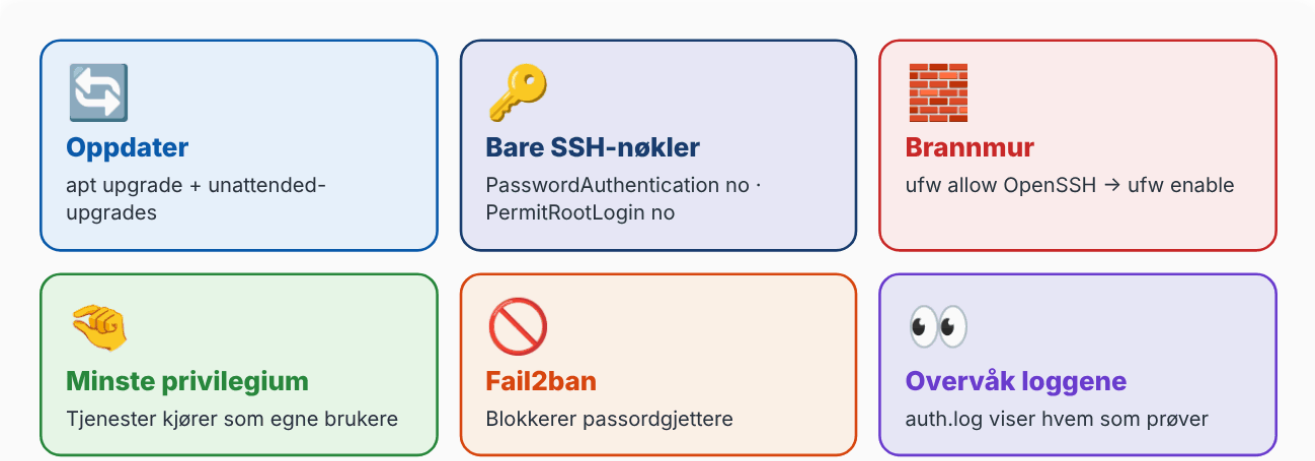


🏢 **Fra arbeidslivet:** Samme syntaks brukes i GitHub Actions, Kubernetes CronJobs og Azure Functions.

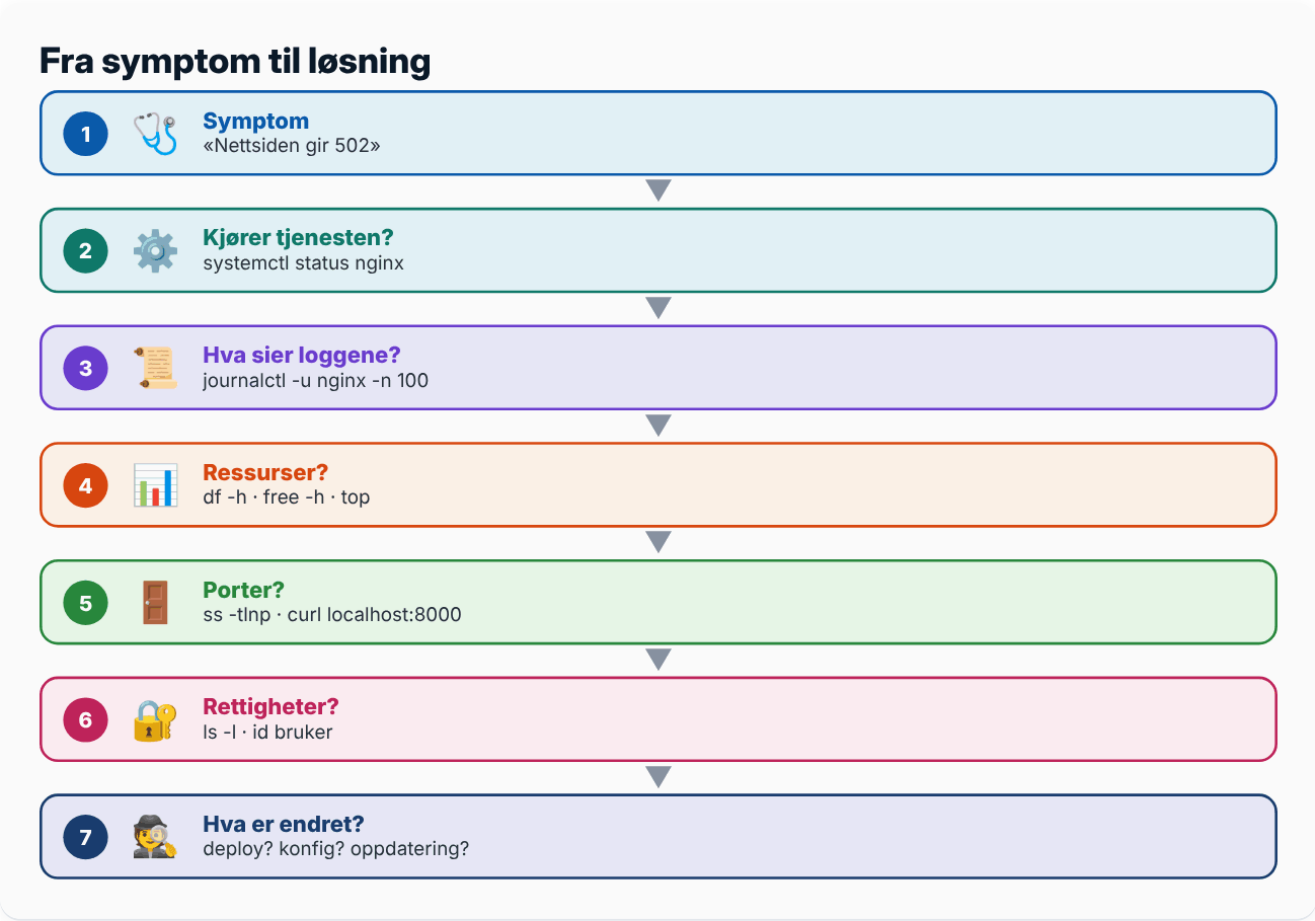
Logger og overvåking



Sikkerhet på 6 punkter



Feilsøking



Test deg selv

1. Hvor ligger konfig og logger? → */etc og /var/log*.

2. Hva betyr `chmod 640`? → *Eier rw, gruppe r, andre ingenting.*
3. `>` vs. `>>`? → *Overskriv vs. legg til.*
4. `start` vs. `enable`? → *Nå vs. ved oppstart.*
5. Hva betyr `0 */6 * * *`? → *Hver sjette time.*
6. Hva betyr returkode 0? → *Suksess.*

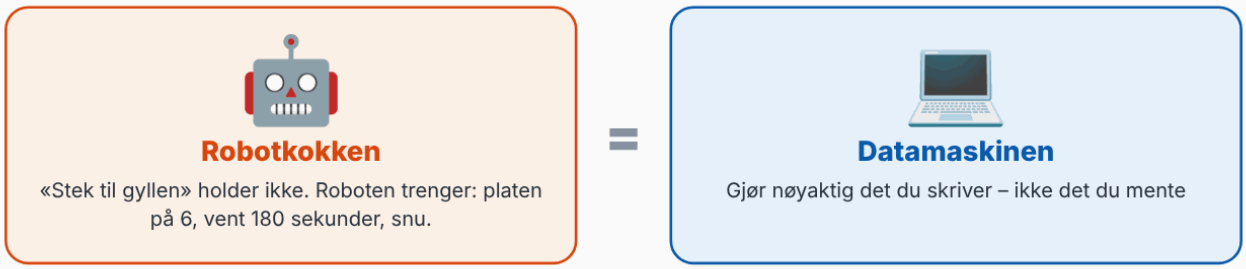
Prøv selv: Installer nginx, start og enable den, og skriv et backup-skript som kjører med cron.

4 · Programmering for DevOps

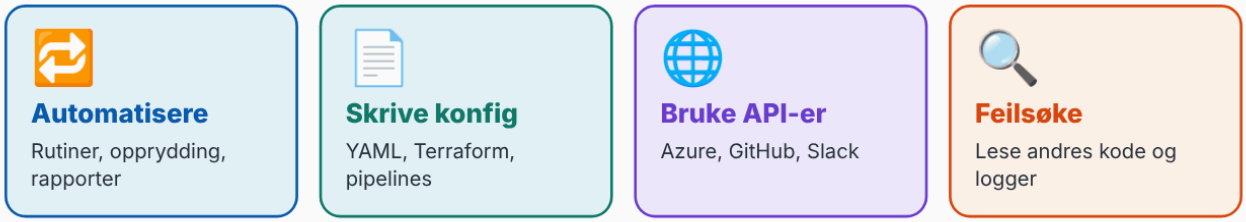
Kapittelet på én side



Hva er programmering?



Hvorfor en DevOps Engineer koder

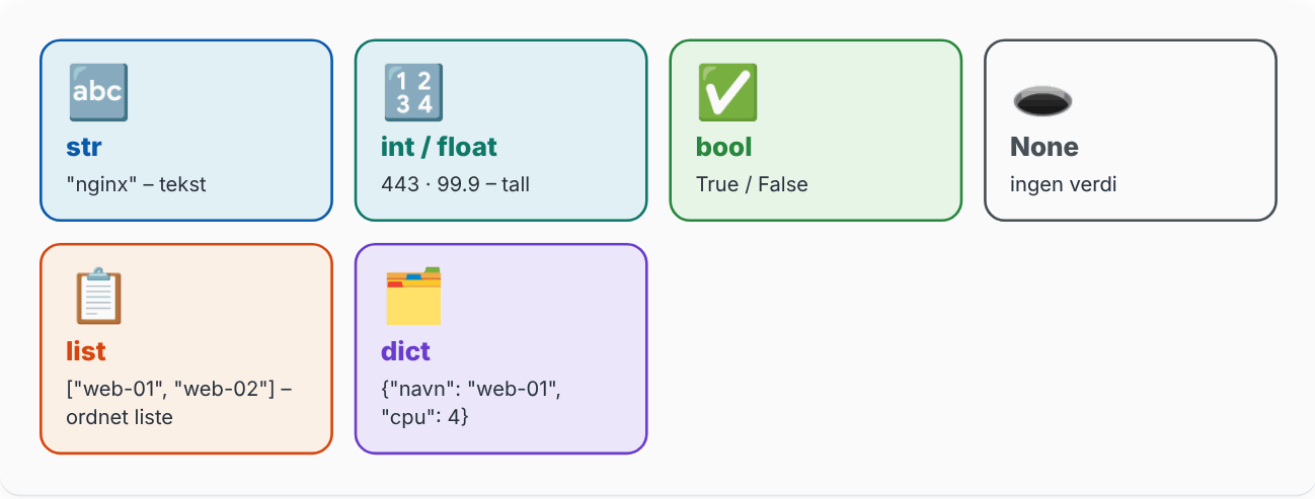


💡 **Start med Python + Bash.** Senere: Go (Docker, Kubernetes og Terraform er skrevet i Go) og PowerShell.

Kom i gang

```
$ python3 --version
$ python3 -m venv .venv && source .venv/bin/activate   # eget miljø per prosjekt
(.venv) $ pip install requests
(.venv) $ pip freeze > requirements.txt                 # lagre pakkelisten
```

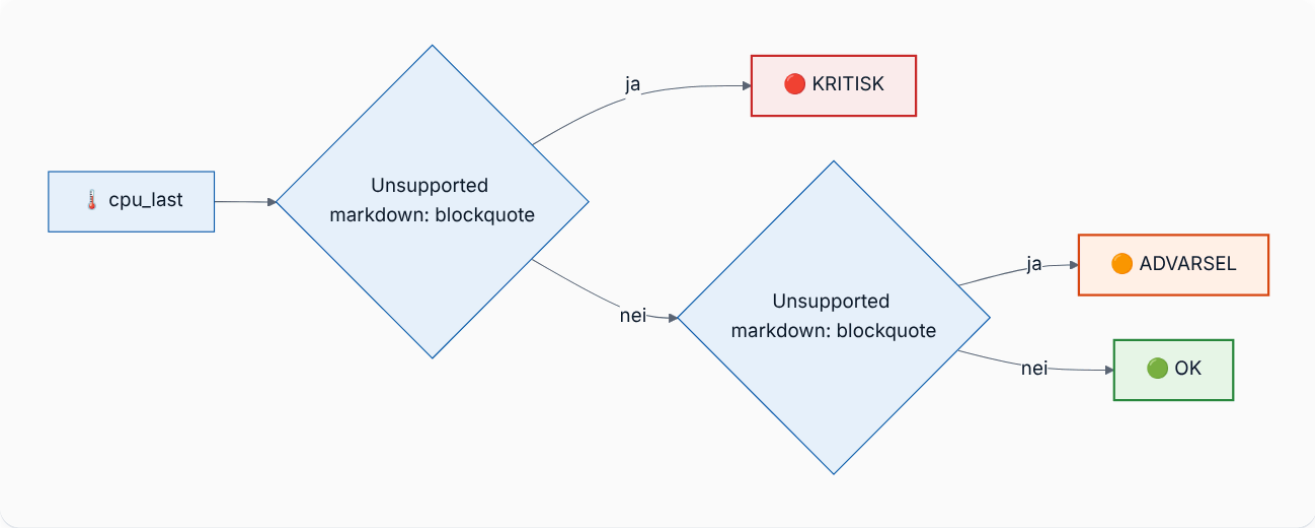
Variabler og datatyper




```
servernavn = "web-01"                          # tekst
antall_cpu = 4                                 # heltall
print(f"{servernavn} har {antall_cpu} CPU-er") # f-streng setter inn verdier
port = int("8080")                             # tekst → tall
```

⚠️ **Vanlig feil:** Verdier fra miljøvariabler og filer er alltid **tekst**. `"5" + 5` gir feil – bruk `int("5")`.

Betingelser og løkker

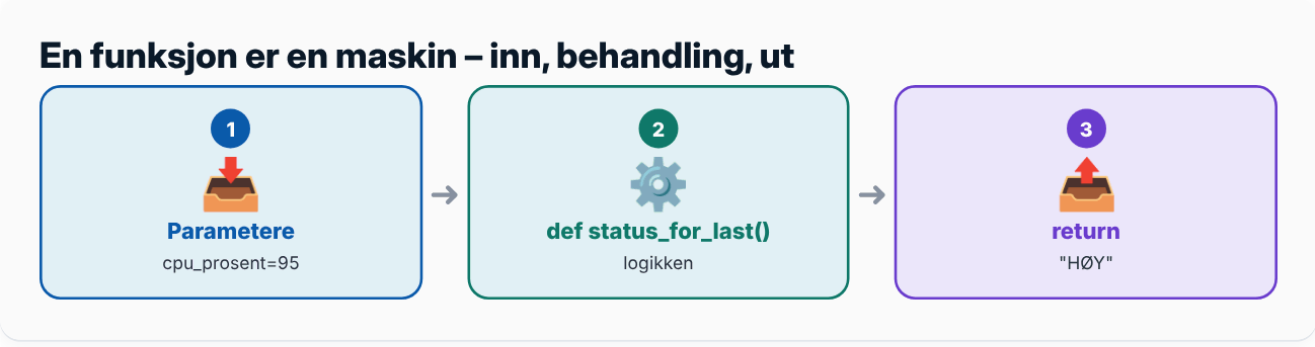


if / elif / else – velg vei basert på en verdi

```python
if cpu_last > 90:
    print("KRITISK")            # innrykk på 4 mellomrom = hører til if
elif cpu_last > 70:
    print("ADVARSEL")
else:
    print("OK")

for server in ["web-01", "web-02", "db-01"]:   # gjør noe for hver
    print(f"Sjekker {server}")
```

Funksjoner




```python
def status_for_last(cpu_prosent: float, grense: float = 80.0) -> str:
    """Returnerer HØY eller NORMAL."""
    return "HØY" if cpu_prosent >= grense else "NORMAL"
```

Lister, ordbøker og JSON

Liste
porter[0] → 22

- ✔ porter = [22, 80, 443]
- ✔ porter.append(8080)
- ✔ Indeks starter på 0!

Ordbok (dict)
vm["navn"] → web-01

- ✔ vm = {"navn": "web-01"}
- ✔ vm.get("ip", "ingen") – trygt
- ✔ JSON fra API-er blir ordbøker

JSON
API-er

- ✔ {"porter": [80, 443]}
- ✔ Doble anførselstegn
- ✔ Ingen kommentarer

vs

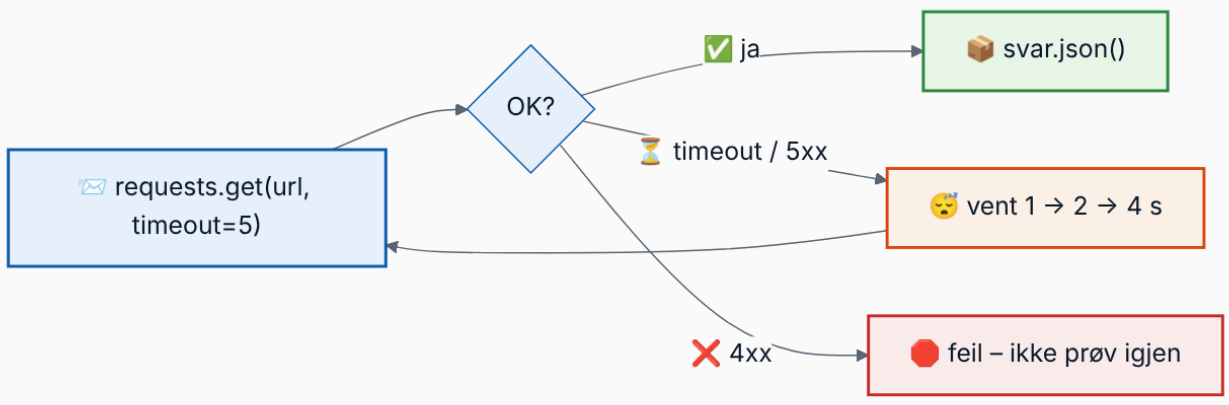
YAML
konfigurasjon

- ✔ porter:
  - 80
  - 443
- ✔ Innrykk med MELLOMROM
- ✔ Kubernetes · Actions · Compose

```
import json, yaml
data = json.loads('{"porter": [80, 443]}')     # tekst → dict
config = yaml.safe_load(open("config.yaml"))   # aldri yaml.load på ukjente filer
```

⚠️ **Vanlig feil:** Tab i YAML. Bruk alltid mellomrom – og sett tekst som `"1.10"` i anførselstegn.

API-kall, feil og logging



Robust API-kall – prøv igjen med økende ventetid ved midlertidige feil

```python
import logging, os, requests
logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")
log = logging.getLogger("app")

token = os.environ["GITHUB_TOKEN"]                      # hemmelighet fra miljøet – aldri i koden
try:
    svar = requests.get("https://api.github.com/user",
                        headers={"Authorization": f"Bearer {token}"}, timeout=10)
    svar.raise_for_status()                             # kast feil ved 4xx/5xx
    log.info("Hei, %s", svar.json()["login"])
except requests.exceptions.RequestException as e:
    log.error("API-kall feilet: %s", e)
```

Loggnivåer

🔥 **CRITICAL**	programmet kan ikke fortsette
❌ **ERROR**	en operasjon feilet
⚠️ **WARNING**	noe uventet, men vi fortsetter
ℹ️ **INFO**	normale hendelser
🔬 **DEBUG**	detaljer for feilsøking

Python eller Bash?

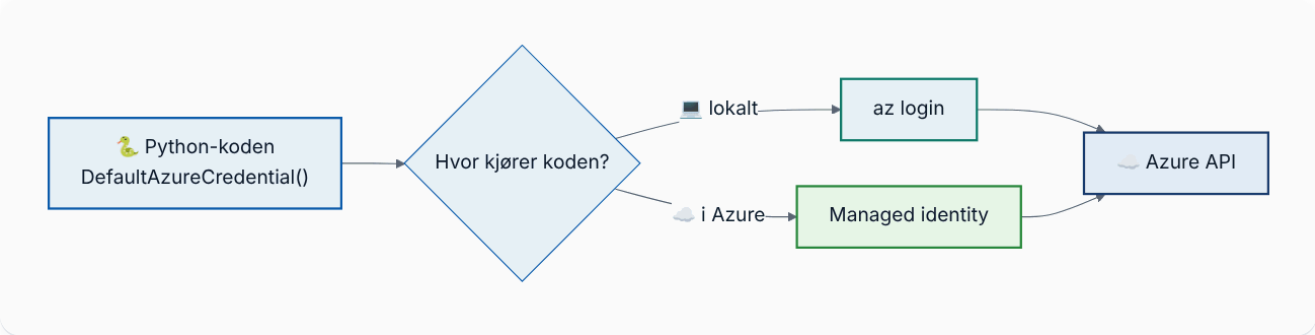
🐚 Bash

- ✔️ Lime sammen Linux-kommandoer
- ✔️ Korte skript
- ✔️ Installere, kopiere, starte

🐍 Python

- ✔️ Logikk og datastrukturer
- ✔️ JSON og API-er
- ✔️ Feilhåndtering og tester
- ✔️ Skal vedlikeholdes over tid

Python mot Azure



DefaultAzureCredential finner riktig innlogging – samme kode lokalt og i Azure

```python
from azure.identity import DefaultAzureCredential
from azure.mgmt.resource import ResourceManagementClient
klient = ResourceManagementClient(DefaultAzureCredential(), "<subscription-id>")
for rg in klient.resource_groups.list():
    print(rg.name, rg.location, rg.tags)
```

Profesjonelle vaner



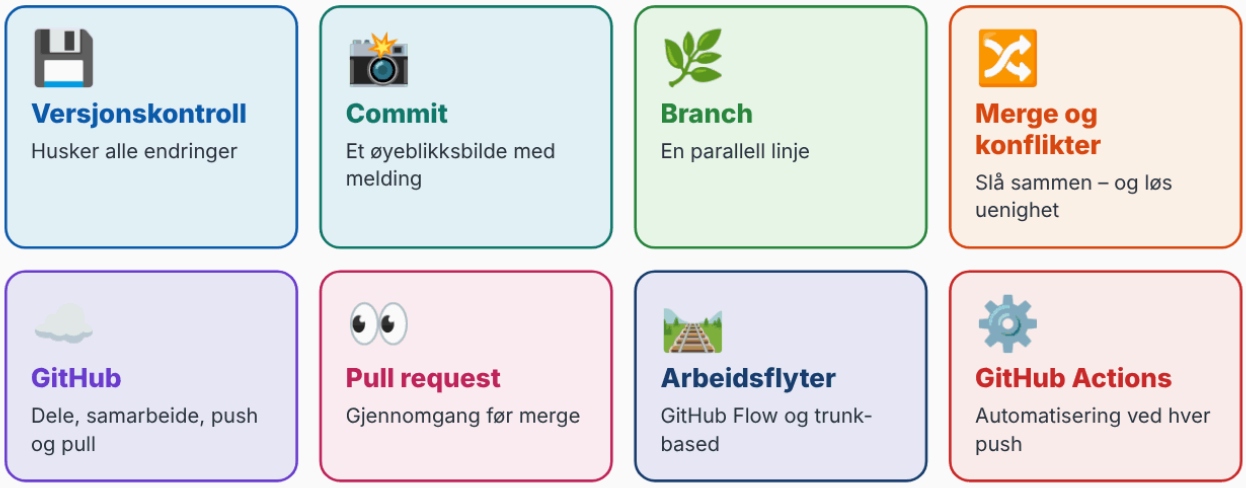
Test deg selv

1. Liste vs. ordbok? → *Liste: indeks. Ordbok: nøkkel → verdi.*
2. `["a","b","c"][1]`? → *"b".*
3. Hva er galt med tab i YAML? → *YAML tillater bare mellomrom.*
4. Hvorfor timeout? → *Ellers kan skriptet henge for alltid.*
5. Hvorfor er `except: pass` dumt? → *Skjuler alle feil.*
6. Hvor skal et token ligge? → *Miljøvariabel eller hemmelighetshvelv.*

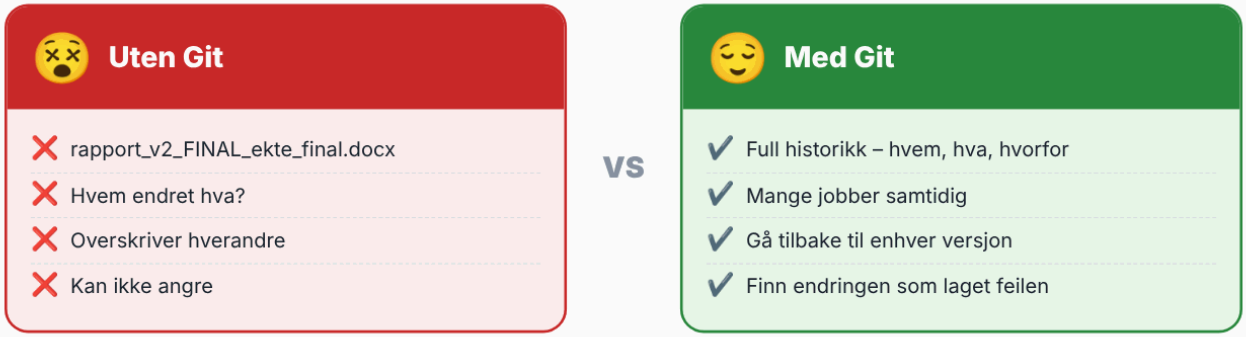
Prøv selv: Skriv et skript som sjekker om en liste nettsider svarer, og skriver en JSON-rapport (mal i fullversjonen).

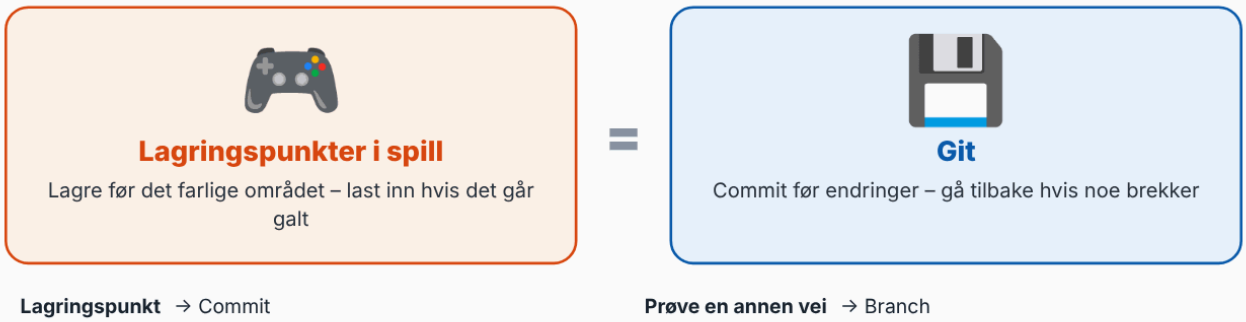
5 · Git og GitHub

Kapittelet på én side



Hvorfor versjonskontroll?





Lagringspunkt → Commit

Prøve en annen vei → Branch

Git
verktøyet – lokalt

- ✔ Commits, branches, historikk
- ✔ Laget av Linus Torvalds (2005)

GitHub
nettjenesten – i skyen

- ✔ Deling og samarbeid
- ✔ Pull requests, issues, Actions
- ✔ Alternativer: GitLab, Azure Repos

De tre områdene – det viktigste å forstå

Fra endring til lagret historikk



git add flytter til staging · git commit lagrer i historikken

Flyttekassen
Velg hva som skal oppi kassen, teip igjen og skriv etikett

=

Commit
git add velger, git commit lagrer med melding

```
$ git init
$ git add README.md                    # legg i staging
$ git commit -m "Legg til README"      # lagre
$ git status · git diff · git log --oneline --graph
```

Dårlige meldinger

- ✘ fix
- ✘ endringer
- ✘ asdf

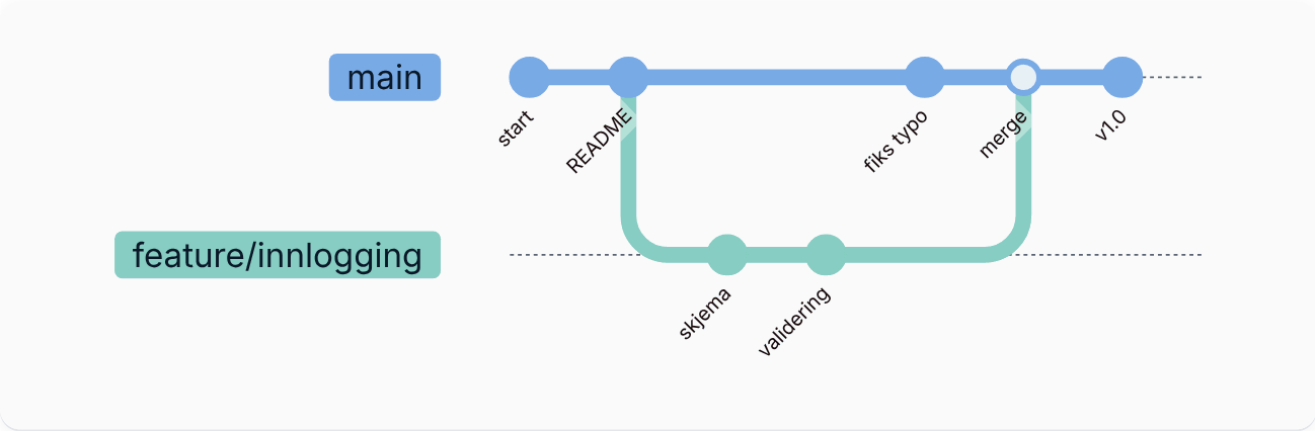
vs

Gode meldinger

- ✔ fix: rett nullpeker ved tomt passord
- ✔ feat: legg til /health-endepunkt
- ✔ Sier hva OG hvorfor

> ⚠️ **Vanlig feil:** `.env` med passord havner i Git. Bruk `.gitignore` – og har det skjedd: **bytt passordet først**.

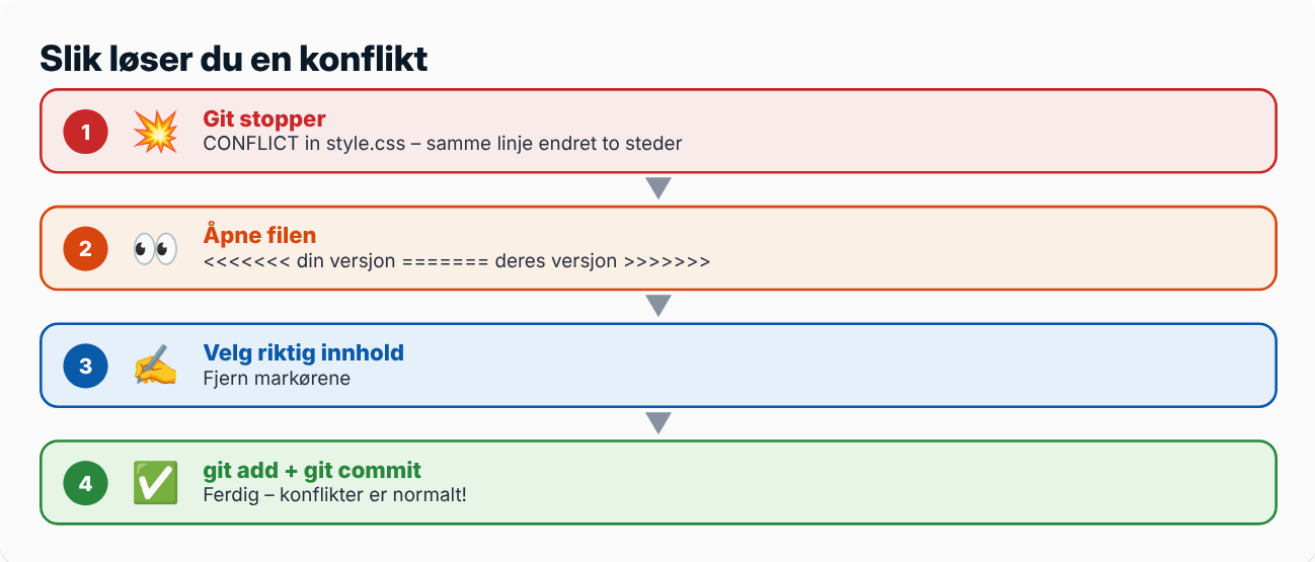
Branch og merge



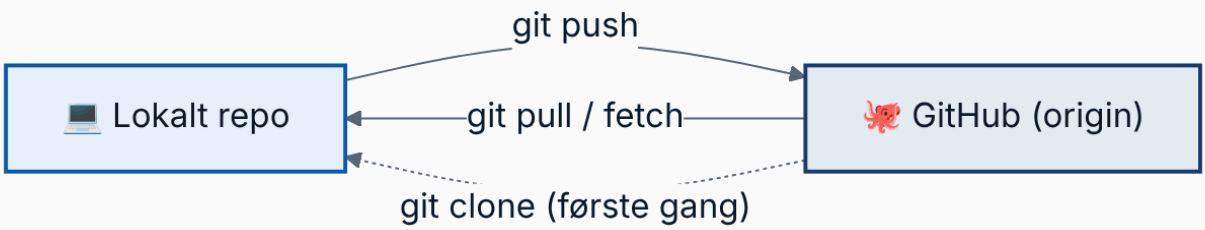
En feature-branch blir laget, får commits og merges tilbake til main

```
$ git switch -c feature/innlogging    # ny branch
$ git switch main && git merge feature/innlogging
```

Merge-konflikter



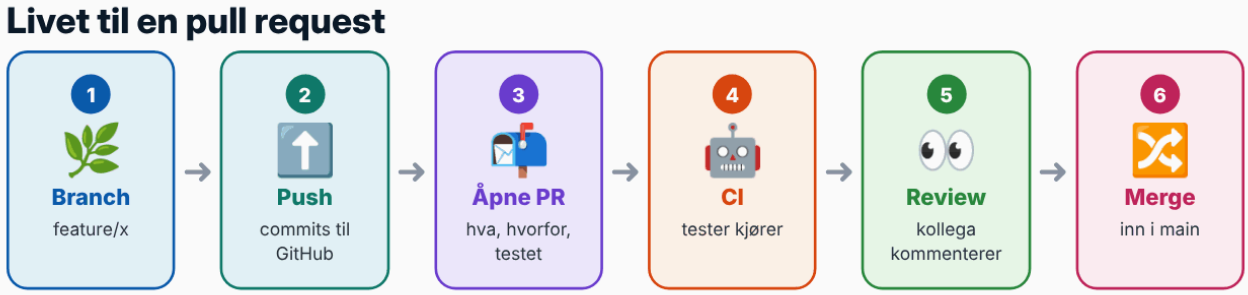
GitHub: clone, push og pull



Synkronisering mellom din PC og GitHub

⚠️ **Vanlig feil:** `git push --force` på en delt branch sletter kollegaers arbeid. Hent først med `git pull`.

Pull requests



💡 **Husk:** Små PR-er som gjør én ting. `Closes #12` lukker issue 12 automatisk ved merge. Beskytt `main` med rulesets: krev PR og grønne tester.

Angre trygt

git restore fil
Forkast endringer i en fil

git restore --staged
Ta ut av staging

git revert HASH
Ny commit som opphever – trygt på delte brancher

git reset --hard
Skriver om historikk – bare lokalt

git stash
Legg arbeid til side

git reflog
Finn «tapte» commits

Arbeidsflyter

GitHub Flow
anbefalt

- ✔ main er alltid deploybar
- ✔ Kortlevde brancher + PR
- ✔ Merge → deploy

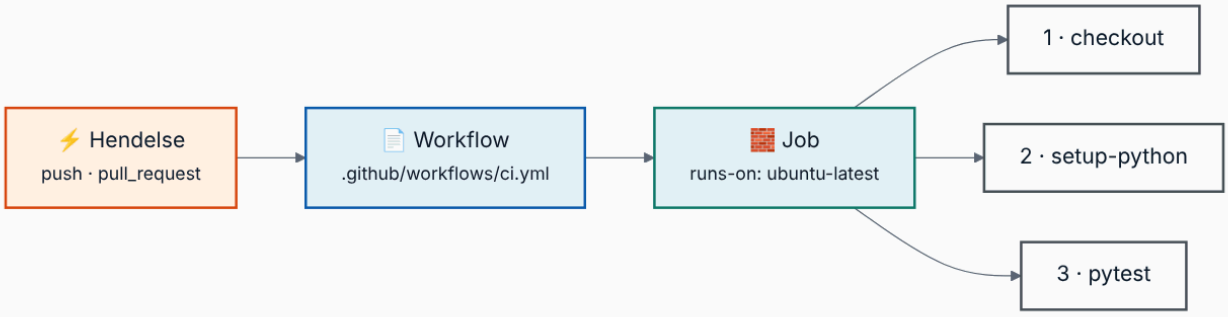
Trunk-based
høytytende team

- ✔ Integrer til main minst daglig
- ✔ Feature flags skjuler uferdig kode

Git Flow
eldre, tyngre

- ✔ develop, release, hotfix …
- ✔ Passer planlagte versjoner

GitHub Actions – første smakebit



En workflow – hendelse starter jobber som kjører steg på en runner

```yaml
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install -r requirements.txt && pytest -q
```

Test deg selv

1. Git vs. GitHub? → *Verktøyet vs. nettjenesten.*
2. Hva er staging? → *Det som skal med i neste commit.*
3. Når oppstår konflikt? → *Samme linjer endret i to brancher.*
4. fetch vs. pull? → *pull = fetch + merge.*
5. Passord pushet – første steg? → *Bytt passordet.*
6. revert vs. reset? → *revert er trygt på delte brancher.*

Prøv selv: Lag et repo, fremprovoser en konflikt, løs den, og legg til en workflow med en grønn ✓.

6 · Cloud Computing

Kapittelet på én side

Hva er skyen?
Lei IT-ressurser over internett, betal for bruk

Datasentre
Regioner og tilgjengelighetssoner

IaaS · PaaS · SaaS
Hvor mye drifter du selv?

Public · private · hybrid
Hvem eier maskinvaren?

Skalering
Opp, ut og automatisk

HA og DR
Tål feil – og katastrofer

Delt ansvar
Leverandøren sikrer skyen, du sikrer innholdet

Kostnader
Betal for bruk – og rydd opp

Hva er skyen?

Strømnettet
Ingen fabrikk har egen generator lenger – du betaler for kWh

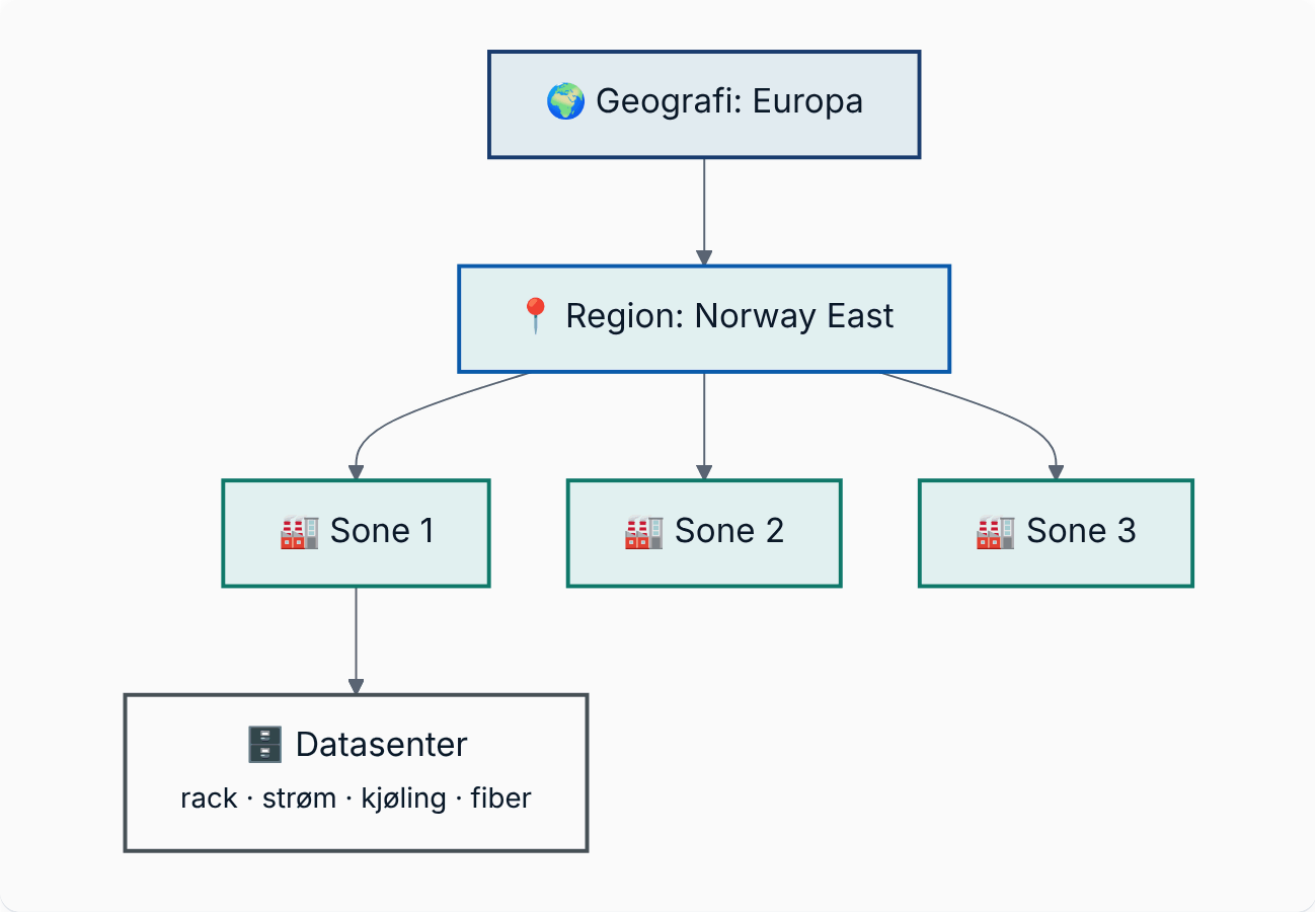
=

Skyen
Du eier ikke serverne – du betaler for timene og gigabytene du bruker

Eget datasenter (on-prem)		Sky
⚠️ Kjøpe maskinvare på forhånd (CapEx)	vs	✅ Betal løpende for bruk (OpEx)
⚠️ Uker å få ny server		✅ Ny server på minutter
⚠️ Må kjøpe for toppen (julehandel)		✅ Skaler opp og ned
✅ Full kontroll		⚠️ Kostnader må styres aktivt



Slik ser skyen ut fysisk



Geografi → region → tilgjengelighetssoner → datasentre med rack

🏢 **Fra arbeidslivet:** Norske virksomheter bruker ofte **Norway East** (Oslo-området) for lav forsinkelse og data i Norge. Ikke alle tjenester finnes i alle regioner – sjekk først.

IaaS, PaaS og SaaS

Hvem drifter hva? (blått = du · grått = leverandøren)

On-premises	DU: alt – fra strøm og maskinvare til app og data
IaaS	DU: OS, runtime, app, data · LEVERANDØR: maskinvare, nettverk, virtualisering
PaaS	DU: app og data · LEVERANDØR: alt under, inkl. OS og patching
SaaS	DU: data og brukere · LEVERANDØR: hele programvaren

Pizza som tjeneste 🍕

On-prem
Lag pizza hjemme fra bunnen – kjøp ovn, lag deig

IaaS
Lei et kjøkken med ovn – ta med ingrediensene selv

PaaS
Bestill levert – du dekker bordet

SaaS
Gå på restaurant – bare spis

Eksempler i Azure

IaaS
Virtual Machines · VNet · Managed Disks

PaaS
App Service · Azure SQL · Functions · Container Apps

SaaS
Microsoft 365 · GitHub · Salesforce

💡 **Husk:** Velg det høyeste nivået som dekker behovet. «Hvorfor trenger vi en VM, når App Service kunne gjort jobben?»

Public, private og hybrid

Public cloud
Azure, AWS, Google – delt maskinvare, logisk isolert

Private cloud
Skyteknologi bare for én organisasjon

Hybrid
Eget datasenter + sky, koblet med VPN/ExpressRoute

Multi-cloud
Flere offentlige skyer samtidig

Skalering

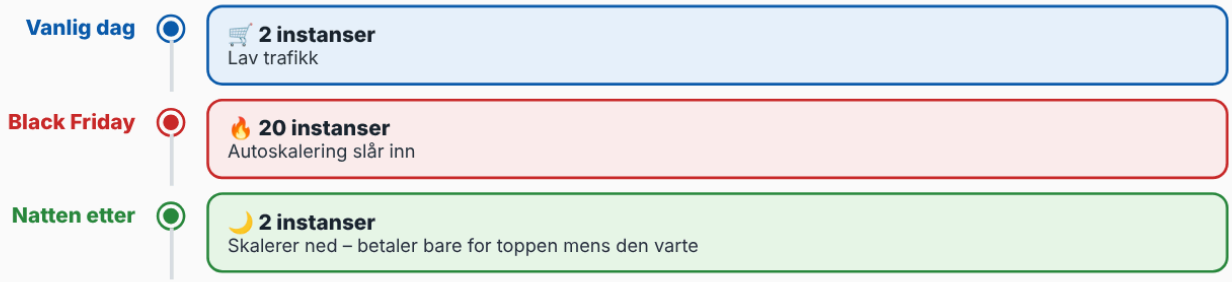
Vertikal (scale up)
større maskin

- ✅ Enkelt
- ⚠️ Har et tak
- ⚠️ Krever ofte omstart

Horisontal (scale out)
flere maskiner

- ✅ Nesten ubegrenset
- ✅ Tåler at én dør
- ⚠️ Appen må tåle flere kopier (stateless)

Elastisitet – automatisk opp og ned (nettbutikk)

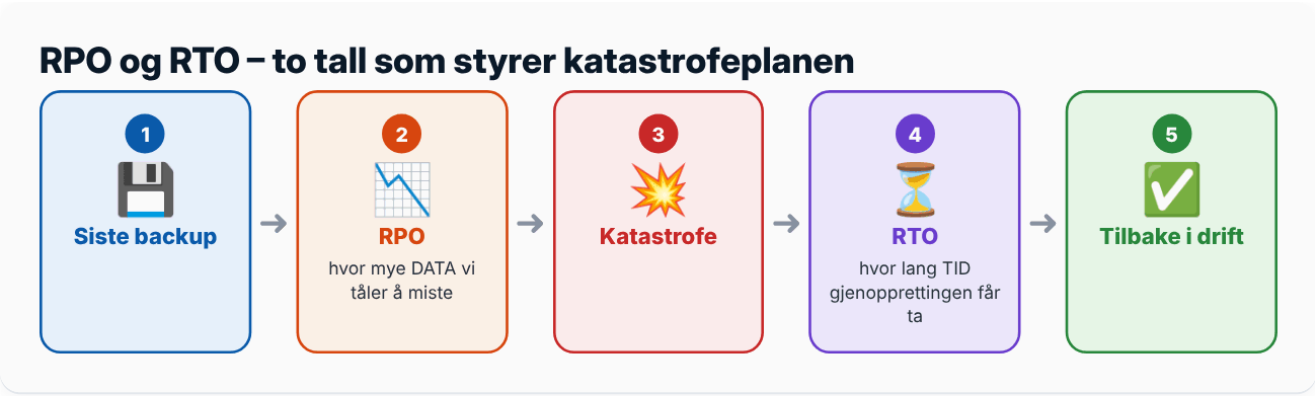


Høy tilgjengelighet og katastrofer

Hva betyr «niere»? Maks nedetid per år

99 %	99,9 %	99,99 %	99,999 %
3,65 dager	8,8 timer	53 minutter	5 minutter

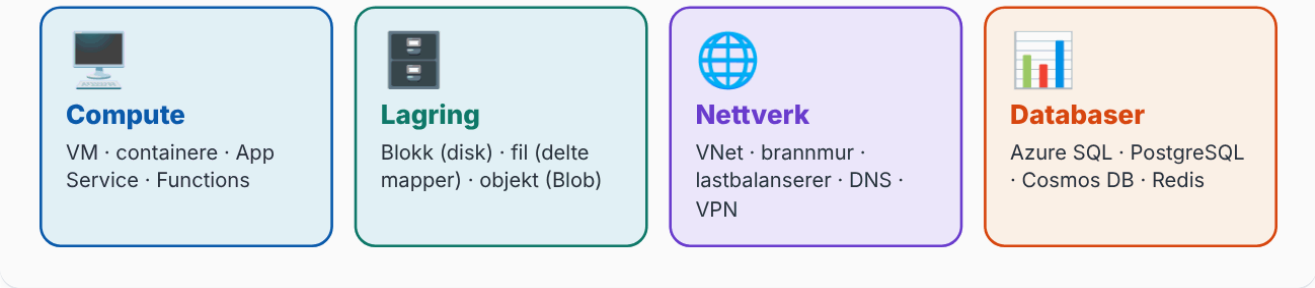
Avhengigheter i serie senker totalen: 3 × 99,9 % ≈ 99,7 %





💡 **Husk:** En backup du aldri har gjenopprettet, er ikke en backup – det er et håp.

Skytjenestene i kategorier



⚡ Serverless
kode som kjører ved hendelser

- ✅ Null serverdrift
- ✅ Skalerer til null – billig ved lav bruk
- ⚠️ Kaldstart
- ⚠️ Begrenset kjøretid

Delt ansvar

🏢 Leverandøren sikrer SKYEN

- ✔️ Fysiske datasentre
- ✔️ Maskinvare og nettverk
- ✔️ Hypervisor
- ✔️ (PaaS/SaaS: også OS og plattform)

🙋 Du sikrer det du legger I skyen

- ❗ DATA – alltid
- ❗ Brukere, tilganger, MFA – alltid
- ❗ Enhetene som brukes – alltid
- ❗ Konfigurasjon (IaaS: også OS og patching)

⚠️ **Vanlig feil:** De fleste sikkerhetsbrudd i skyen skyldes **kundens** feilkonfigurasjon – åpne lagringskontoer, svake passord, manglende MFA.

Kostnader

Hva koster penger?

⏱️ Compute
Per sekund en VM kjører

🗄️ Lagring
Per GB per måned + transaksjoner

📤 Trafikk ut
Data UT av skyen koster ofte

🧱 Tjenester som står på
Gatewayer, brannmurer, offentlige IP-er

Spar penger – sjekkliste



💰 **Kostnad:** «Jeg rydder opp i morgen» er den dyreste setningen for en student. Slett ressursgruppen **samme dag**.

Azure, AWS og Google Cloud



Well-Architected – fem søyler å sjekke alt mot

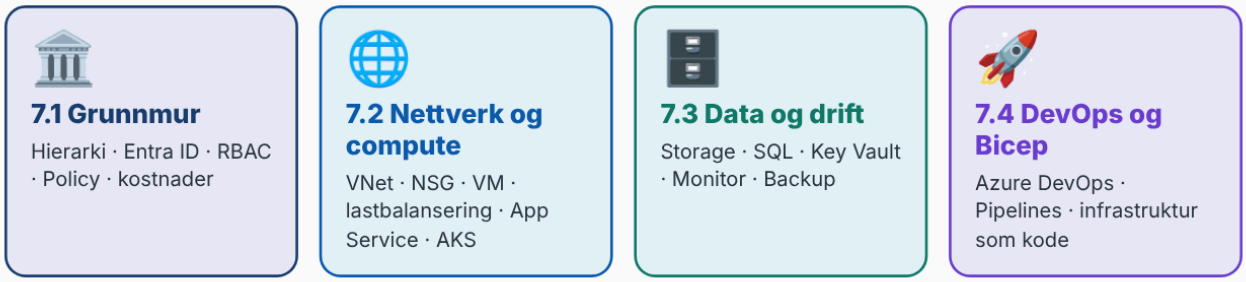


Test deg selv

1. CapEx vs. OpEx? → *Investering på forhånd vs. løpende utgift.*
2. Hva er en tilgjengelighetssone? → *Fysisk adskilt datasenter-lokasjon i en region.*
3. Hvem patcher OS-et i PaaS? → *Leverandøren.*
4. Hva er du alltid ansvarlig for? → *Data, identiteter, enheter.*
5. RPO vs. RTO? → *Tåler å miste data vs. tid til gjenoppretting.*
6. Blob Storage heter … hos AWS? → *S3.*

7 · Microsoft Azure

Azure-delen på én side



💰 **Før du lager noe i Azure:** 1) Bruk Azure for Students eller gratiskonto. 2) Lag **budsjettvarsel**. 3) Én ressursgruppe per øvelse – **slett den etterpå**.

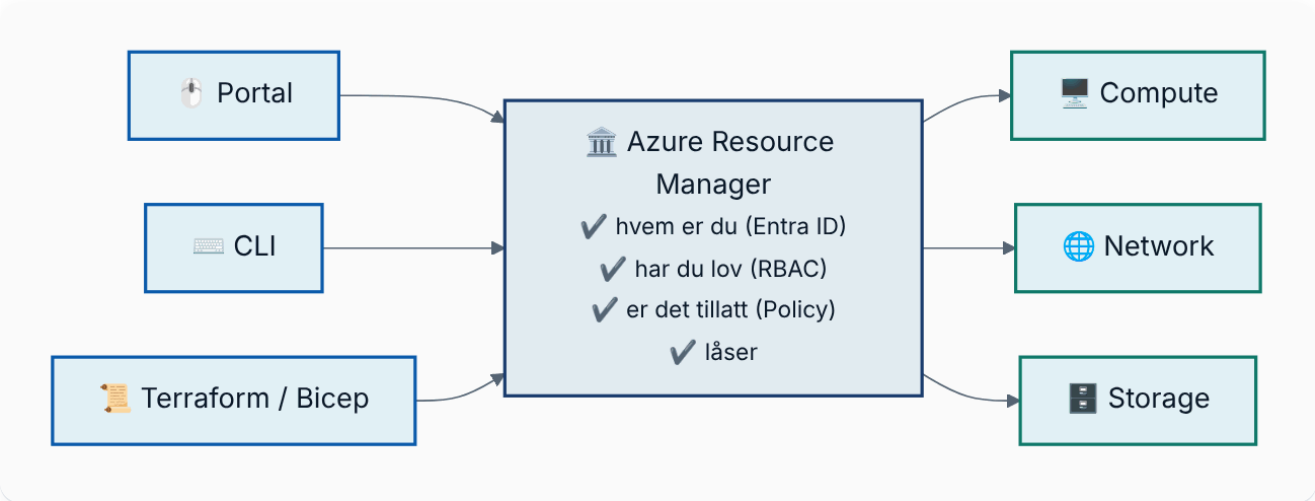
7.1 Grunnmur, identitet og styring

Verktøyene



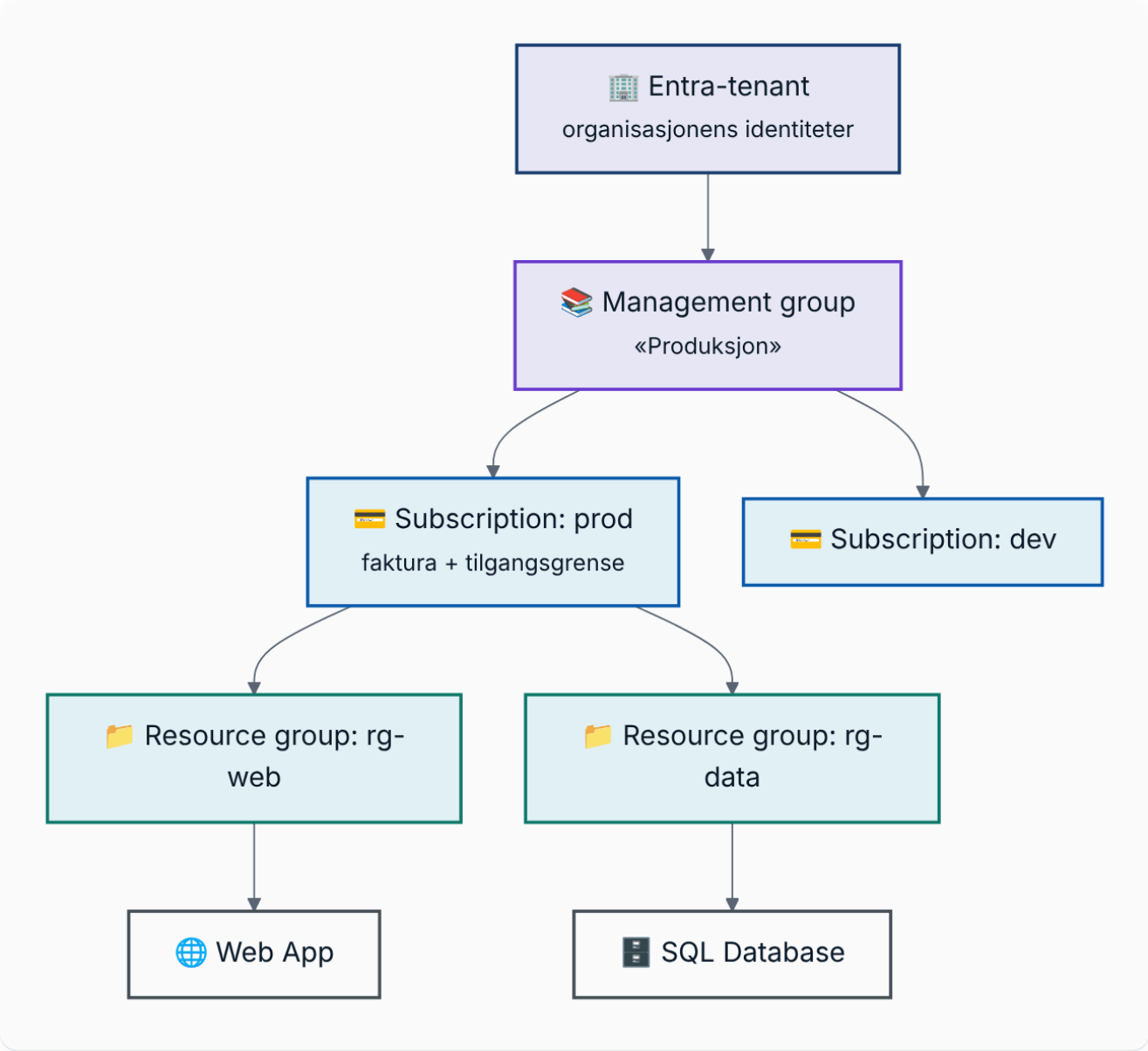

```
$ az login
$ az account show -o table
$ az group create -n rg-lab-01 -l norwayeast --tags miljø=lab eier=student
$ az vm list --query "[].name" -o tsv          # filtrer utdata
```

Azure Resource Manager – alt går gjennom samme dør

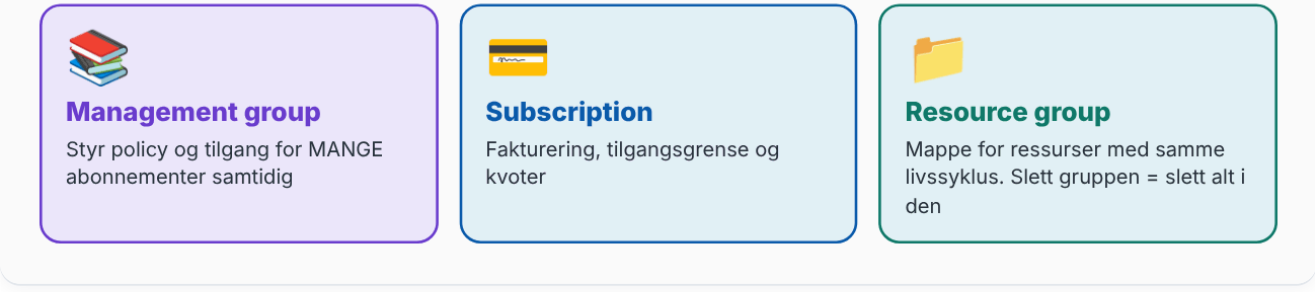


Portal, CLI og kode bruker samme API – derfor er sikkerhet og policy alltid lik

Hierarkiet



Tilganger og policyer arves nedover

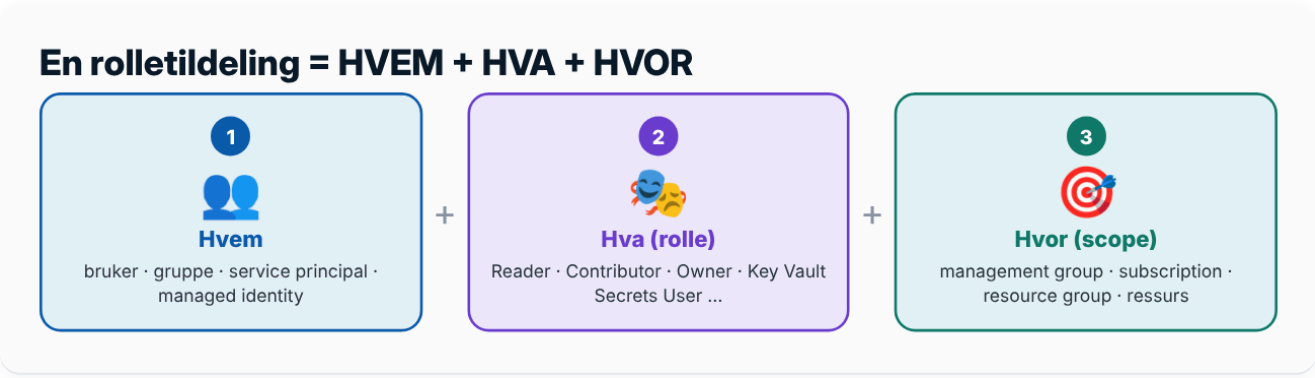


💡 **Husk:** En ressurs ligger i **nøyaktig én** ressursgruppe. Ressursgrupper kan **ikke** nøstes. Navngi konsekvent: `rg-`, `vnet-`, `vm-`, `kv-`, `st…`.

Microsoft Entra ID – hvem er du?



Azure RBAC – hva får du gjøre?



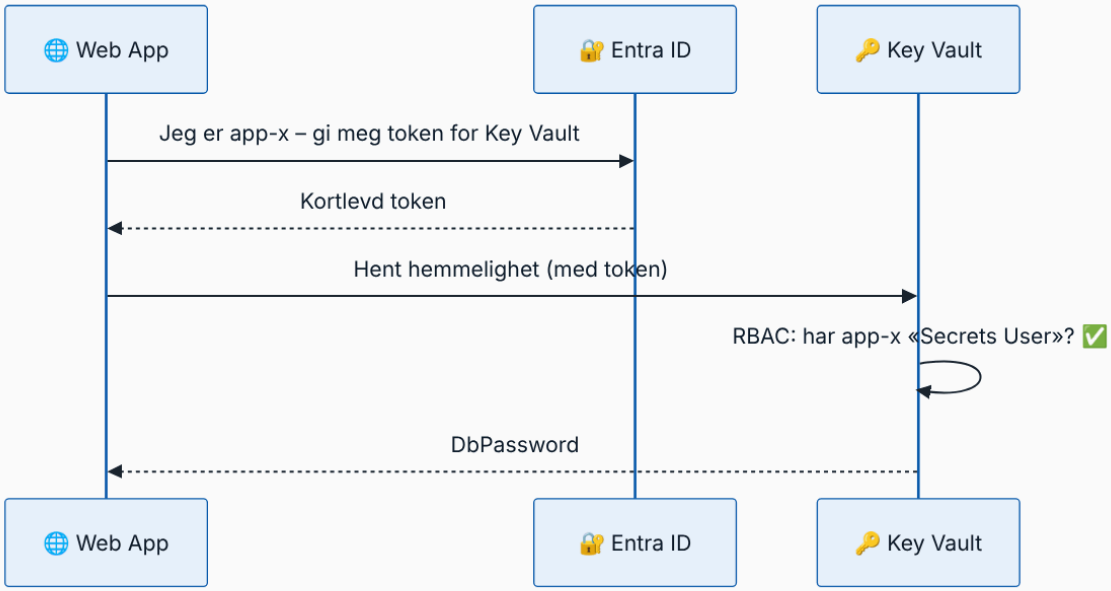
De viktigste rollene

👑 **Owner**	Alt – også gi andre tilgang. Nesten aldri.
🛠️ **Contributor**	Lage og endre alt – men ikke gi tilgang
👀 **Reader**	Se alt, endre ingenting – trygg standard
🎯 **Tjenesteroller**	AcrPull · Storage Blob Data Reader · Key Vault Secrets User

```
$ az role assignment create --assignee-object-id <gruppe-id> --assignee-principal-type
Group \
    --role "Reader" --scope $(az group show -n rg-lab-01 --query id -o tsv)
```

⚠️ **Vanlig feil:** Entra-roller (Global Administrator – styrer brukere) ≠ Azure RBAC-roller (Owner – styrer ressurser). To ulike systemer.

Managed identities – ingen passord



Appen får et kortlevd token – det finnes ingen hemmelighet å lekke

🔗 System-assigned

- ✔ Lages med ressursen
- ✔ Slettes med ressursen
- ✔ Én ressurs

📌 User-assigned

- ✔ Egen ressurs
- ✔ Kan deles av mange
- ✔ Lever uavhengig

💡 **Husk:** «Hvordan skal appen logge inn på en annen Azure-tjeneste?» → nesten alltid **managed identity**.

Azure Policy – hva er lov å lage?

🎭 RBAC
HVEM får gjøre noe

- ✔ «Kari får lage VM-er»

vs

📏 Policy
HVA er tillatt

- ✔ «Ingen – heller ikke Kari – får lage VM-er utenfor Norway East»

Policy-effekter

⛔ Deny
Stopper opprettelsen

📝 Audit
Tillater, men markerer avvik

✏️ Modify / Append
Legger til f.eks. en tag

🏗️ DeployIfNotExists
Lager det som mangler

Tags og låser

🏷️ Tags
miljø=prod · eier=team-x · kostnadssted=4410. Arves IKKE automatisk.

🔒 CanNotDelete
Kan endres, ikke slettes – også for Owner

🧊 ReadOnly
Kan verken endres eller slettes

Kostnadsstyring

Gjør dette første dag – budsjett med varsel

1. 🔎 **Cost Management → Budgets → Add**
   Scope: abonnementet ditt
2. 💵 **Månedlig beløp**
   f.eks. 20 i din valuta
3. 🔔 **Varsler**
   50 % og 90 % faktisk · 100 % prognose
4. 📧 **E-post**
   Din adresse → Create

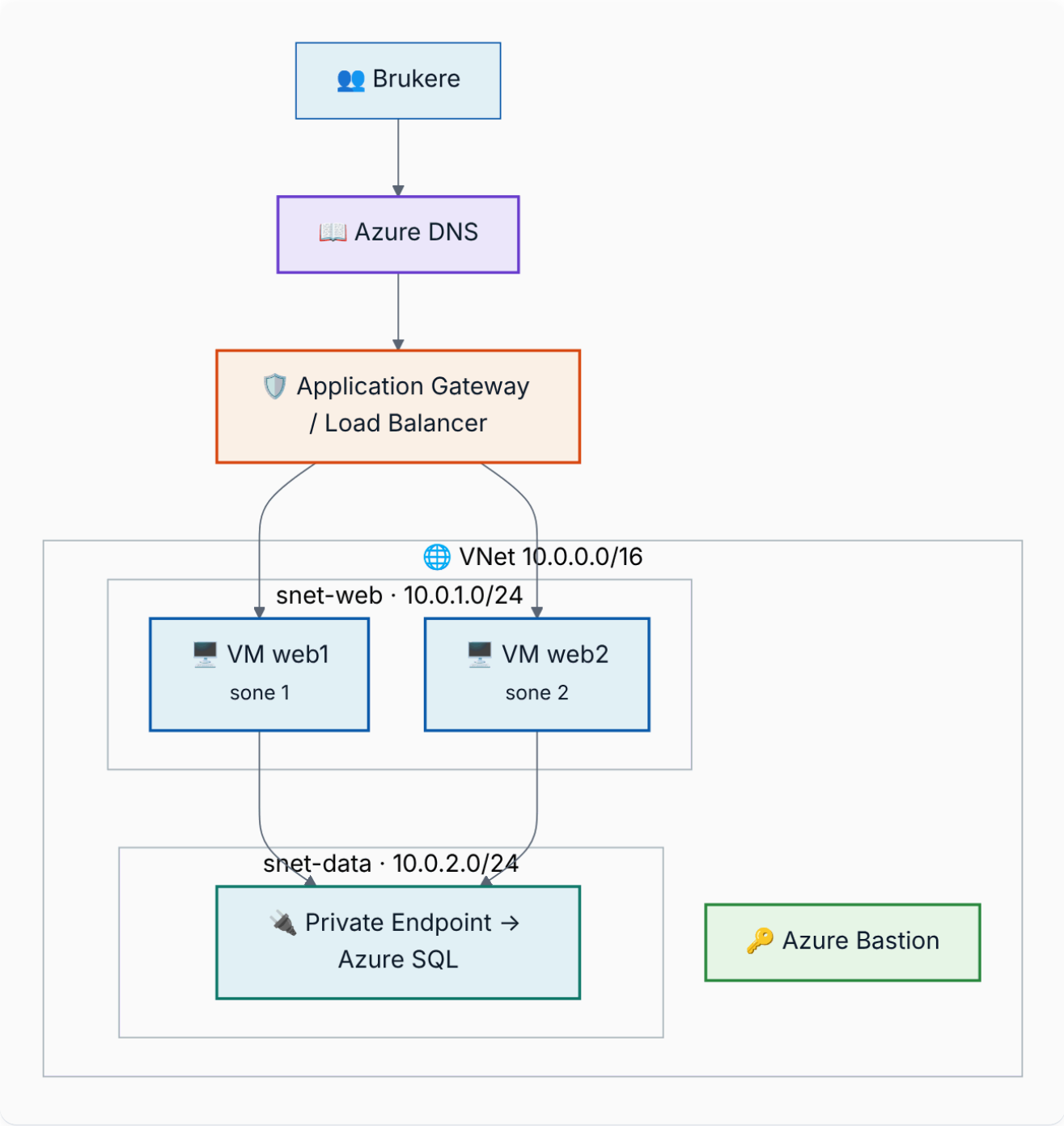
⚠️ **Vanlig feil:** Et budsjett **varsler** – det **stopper ikke** forbruket. Rydd opp selv.

🧮 **Pricing Calculator**
Estimer FØR du bygger

📊 **Cost analysis**
Se faktisk forbruk per gruppe, tjeneste eller tag

💡 **Azure Advisor**
Anbefalinger: «denne VM-en er nesten ubrukt»

7.2 Nettverk og compute



Arkitekturen i dette kapitlet – NSG på snet-web tillater 80/443, NSG på snet-data tillater 1433 bare fra web

Virtual Network og subnett

VNet
Ditt private nettverk i én region. Gratis i seg selv.

Subnett
Del av VNet-et. 5 adresser reservert. Noen tjenester krever eget subnett.

Peering
Kobler VNets. Ingen overlapp! Ikke transitivt (A–B–C ≠ A–C).



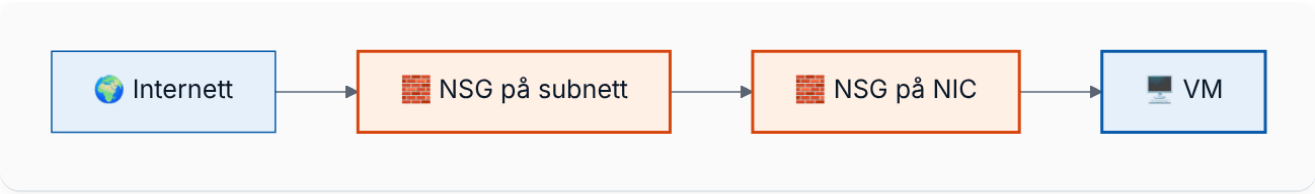
Hub-and-spoke – det vanligste bedriftsoppsettet

```
az network vnet create -g $RG -n vnet-lab --address-prefix 10.0.0.0/16 \
  --subnet-name snet-web --subnet-prefixes 10.0.1.0/24
```

Network Security Groups (NSG)

NSG-regler: lavest tall først – første treff vinner

- **100 · Tillat 80, 443** fra Internet
- **110 · Tillat 22** bare fra din IP /32
- **65000 · AllowVnetInBound** standard: trafikk innen VNet
- **65001 · AllowAzureLoadBalancer** standard: helsesjekker
- **65500 · DenyAllInBound** standard: alt annet avvises



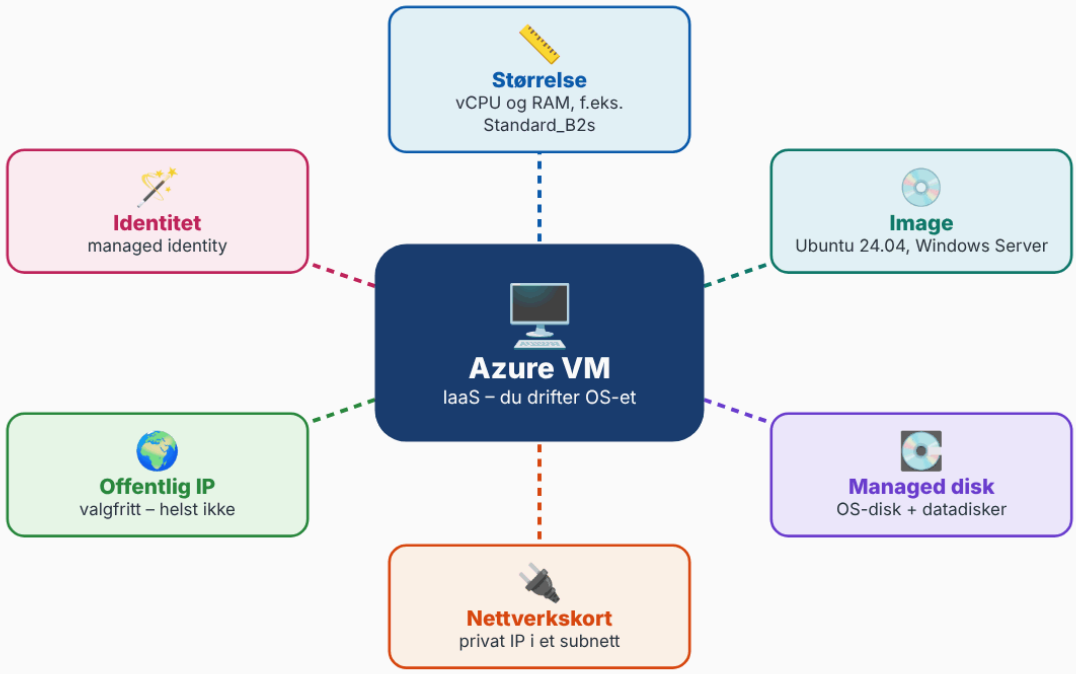
Finnes NSG både på subnett og nettverkskort, må BEGGE tillate trafikken

```
az network nsg rule create -g $RG --nsg-name nsg-web -n allow-web --priority 100 \
  --direction Inbound --access Allow --protocol Tcp \
  --source-address-prefixes Internet --destination-port-ranges 80 443
```

🔍 **Feilsøk:** *Network Watcher → IP flow verify* forteller hvilken regel som tillot eller blokkerte.

Virtual Machines

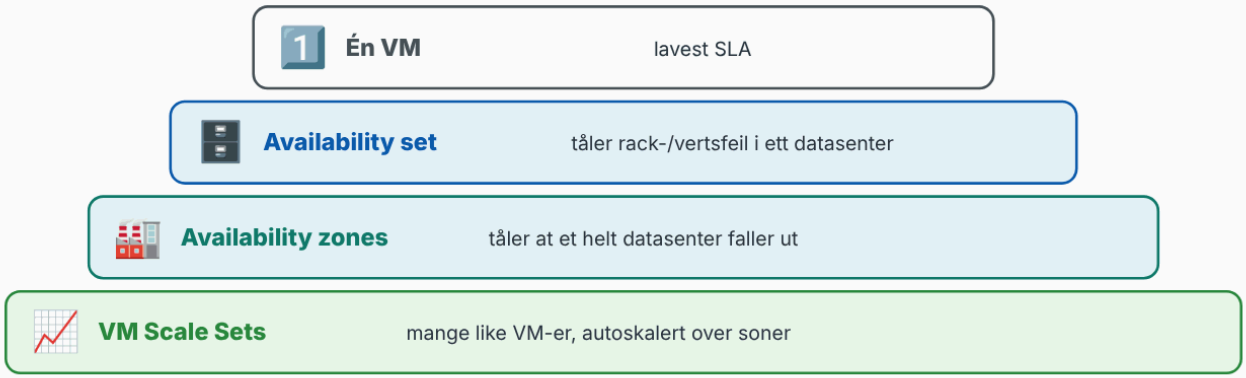
En VM består av



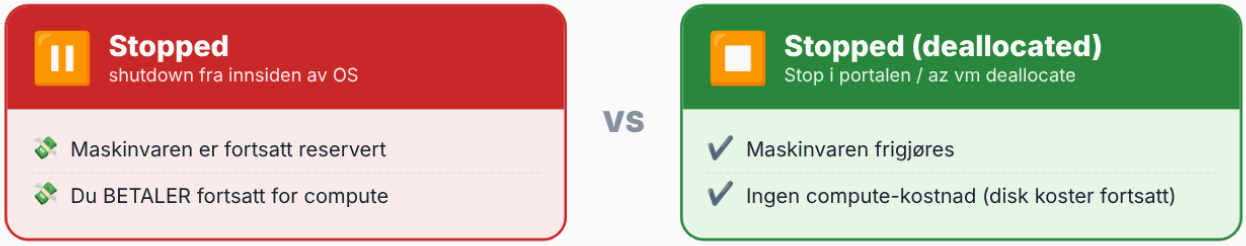
VM-familier



Tilgjengelighet for VM-er



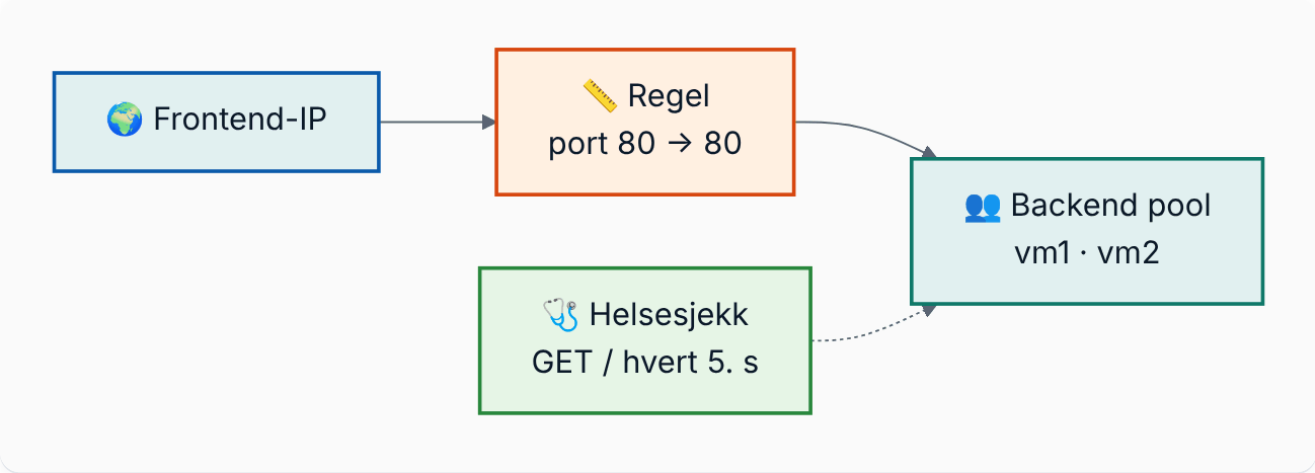

```
az vm create -g $RG -n vm-web01 --image Ubuntu2404 --size Standard_B1s \
  --vnet-name vnet-lab --subnet snet-web --admin-username azureuser \
  --generate-ssh-keys --public-ip-sku Standard --custom-data cloud-init.yaml
az vm deallocate -g $RG -n vm-web01        # stopp OG slutt å betale for compute
az vm auto-shutdown -g $RG -n vm-web01 --time 1900
```



🔑 **Sikker innlogging:** Azure **Bastion** gir SSH/RDP uten offentlig IP på VM-en.

Lastbalansering

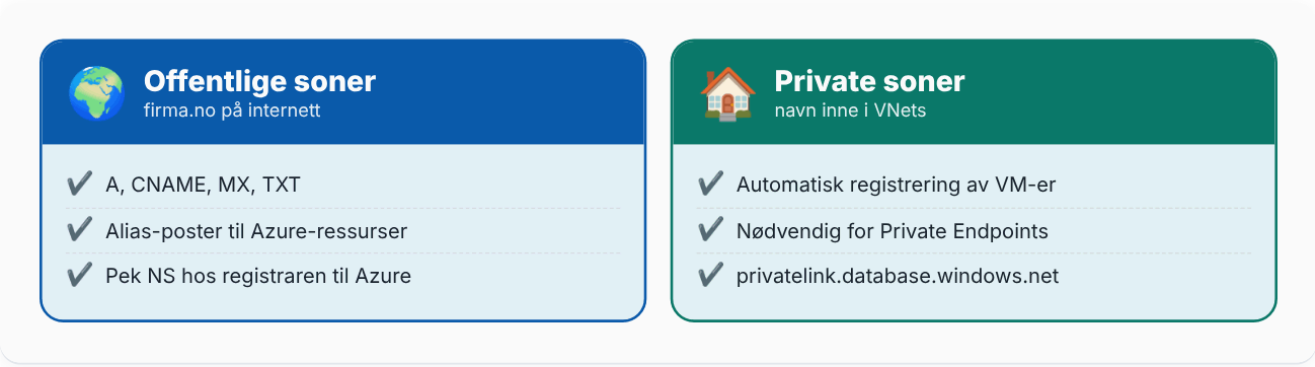




Bestanddelene i en Azure Load Balancer

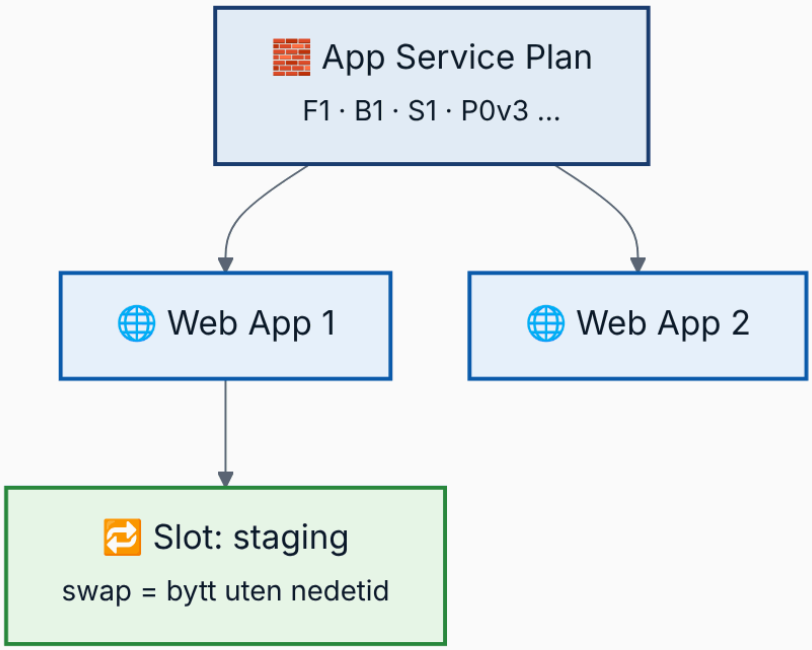
⚠️ **502 fra Application Gateway** = backend er usunn: appen kjører ikke, NSG blokkerer, feil port eller feil helsesjekk-sti.

Azure DNS



App Service





Du betaler for PLANEN – flere apper kan dele den

```
az webapp up --name app-lab-<unik> -g $RG --runtime "PYTHON:3.12" --sku F1
az webapp log tail -g $RG -n app-lab-<unik>
```

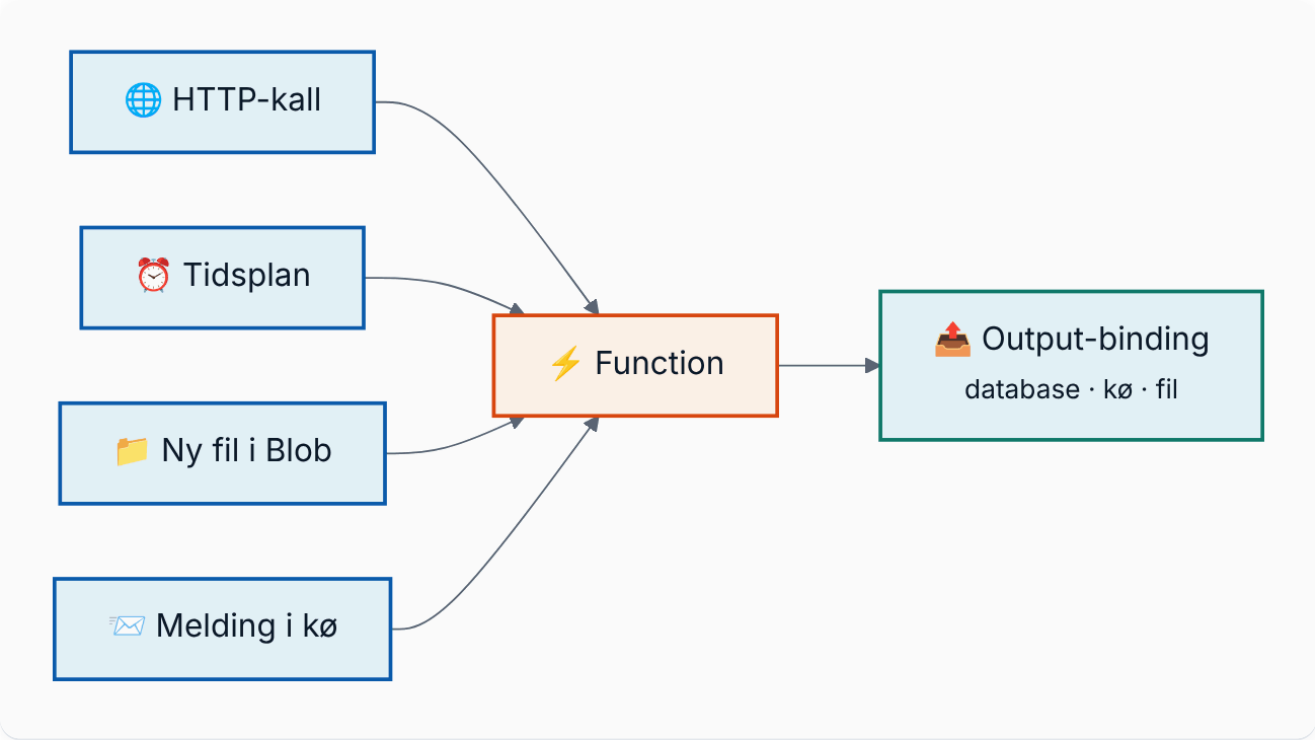
F1 Free
Læring – begrenset, ingen slots

Basic
Små apper og test

Standard+
Produksjon – autoskalering og slots

Key Vault-referanser
@Microsoft.KeyVault(...) i app settings

Azure Functions – serverless

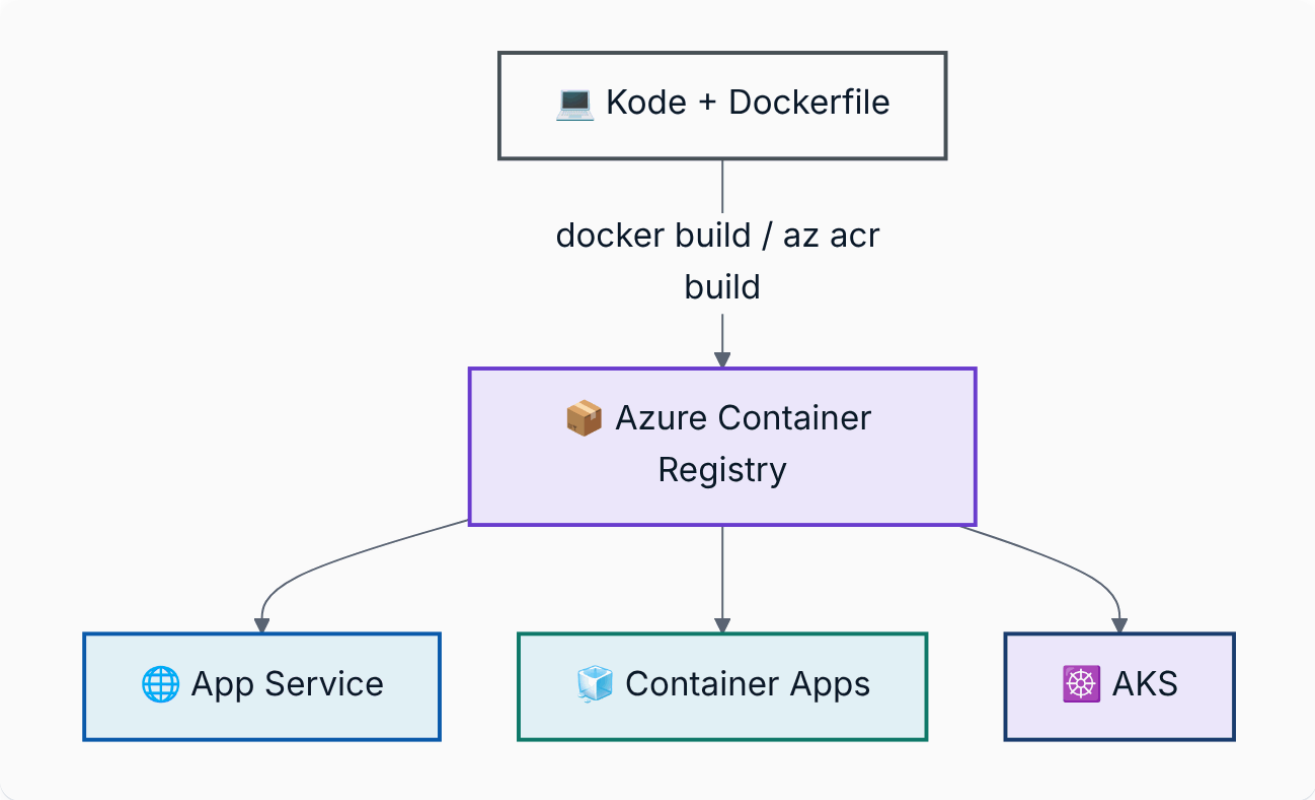


En funksjon kjører bare når en trigger skjer

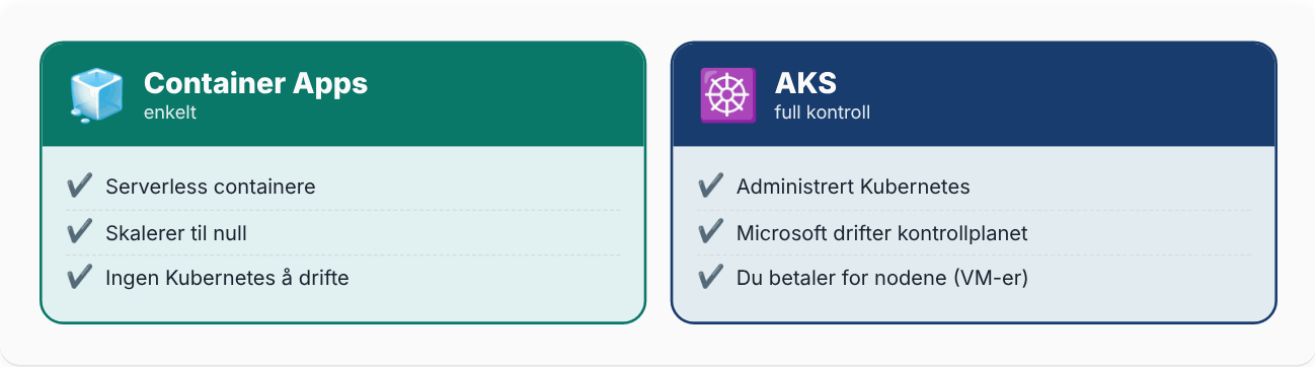
```python
@app.route(route="hei")                                    # HTTP-trigger
def hei(req: func.HttpRequest) -> func.HttpResponse:
    return func.HttpResponse(f"Hei, {req.params.get('navn', 'verden')}!")
```

⚠️ Timer-triggere har **6 felt** (sekunder først): `0 */5 * * * *` = hvert 5. minutt. Anbefalt plan for nye apper: **Flex Consumption**.

Container Registry (ACR) og AKS



Fra kode til kjørende container i Azure – tjenestene henter images med rollen AcrPull

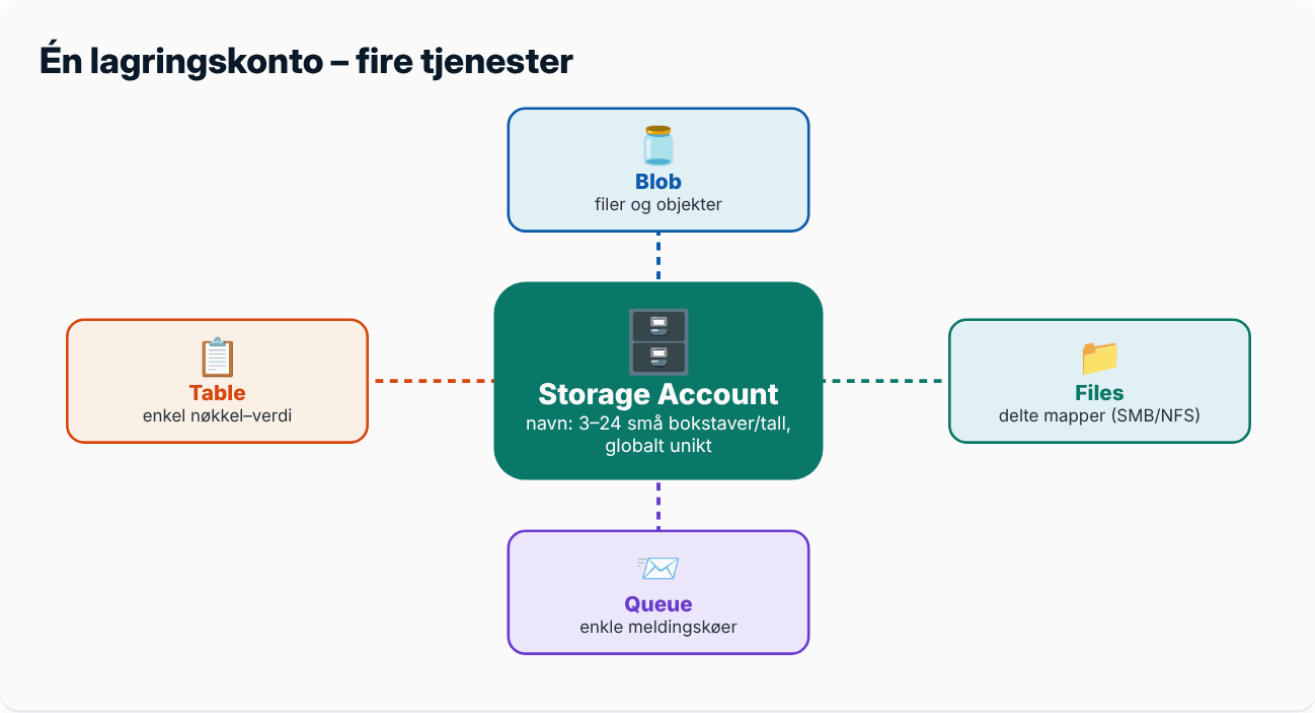


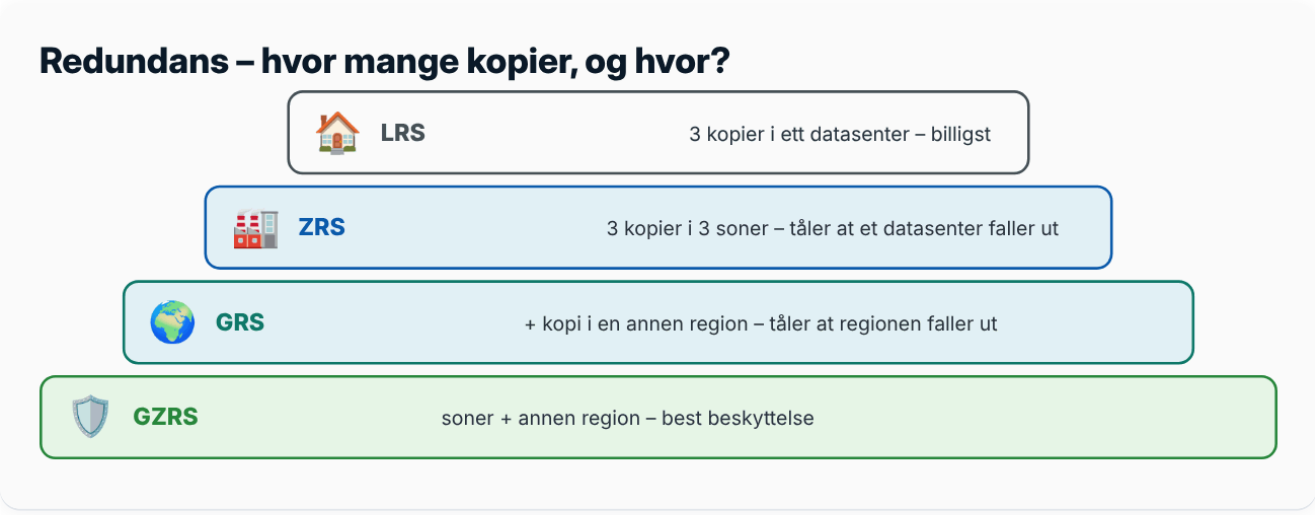
> 💰 **Kostnad:** AKS-noder er vanlige VM-er. `az aks stop` når du ikke bruker clusteret – og slett det samme dag.

```
az acr create -g $RG -n acrlab<unik> --sku Basic
az aks create -g $RG -n aks-lab --tier free --node-count 1 --node-vm-size Standard_B2s \
  --attach-acr acrlab<unik> --generate-ssh-keys
az aks get-credentials -g $RG -n aks-lab && kubectl get nodes
```

7.3 Lagring, data, sikkerhet og drift

Storage Account





Tilgang til data – fra best til verst

🏅 **Entra ID + RBAC**	managed identity med «Storage Blob Data Reader» – anbefalt
⏱️ **SAS-token**	tidsbegrenset lenke – bra for midlertidig deling
🗝️ **Kontonøkler**	full tilgang til alt – unngå, kan slås av
🌍 **Anonym offentlig**	alle kan lese – av som standard

Blob Storage

Tilgangsnivåer – billig lagring eller billig lesing?

🔥 **Hot**	brukes ofte · dyr lagring · billig lesing
❄️ **Cool**	sjelden · min. 30 dager
🧊 **Cold**	svært sjelden · min. 90 dager
🗄️ **Archive**	nesten aldri · billigst · må hentes frem (timer)



Livssyklusregler flytter data automatisk og sparer penger

```
az storage account create -n st$RANDOM -g $RG -l norwayeast --sku Standard_LRS \
  --min-tls-version TLS1_2 --allow-blob-public-access false
az storage blob upload --account-name <konto> -c bilder -f bilde.png --auth-mode login
```

💡 **Ekstra beskyttelse:** soft delete, versjonering og *immutable storage* (kan ikke endres eller slettes i en periode – vern mot løsepengevirus).

Azure SQL Database

Azure SQL Database
PaaS

- ✔ Nye skyapper
- ✔ Microsoft patcher og tar backup
- ✔ Serverless kan pause

SQL Managed Instance
PaaS

- ✔ Flytte eksisterende SQL Server
- ✔ Nesten 100 % kompatibel

SQL Server på VM
IaaS

- ✔ Full kontroll
- ✔ Du drifter alt

Innebygd robusthet og sikkerhet

Point-in-time restore
Tilbake til et hvilket som helst tidspunkt (7–35 dager)

Geo-replikering
Failover til annen region

Entra-autentisering
Appen logger inn med managed identity

Private Endpoint
Ikke eksponert mot internett

⚠️ **Vanlig feil:** «Allow Azure services to access this server» slipper inn **alle** Azure-kunders ressurser, ikke bare dine.

Key Vault

Hvelvet for hemmeligheter

Secrets
Passord, tilkoblingsstrenger, API-nøkler

Keys
Krypteringsnøkler – også i HSM

Certificates
TLS-sertifikater med automatisk fornyelse

Key Vault Administrator
Alt

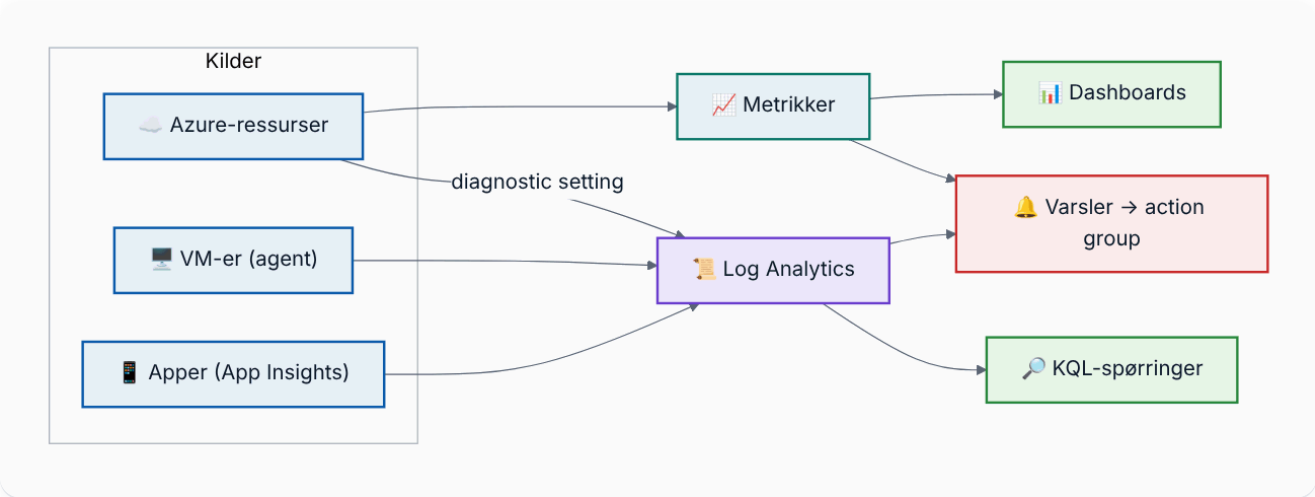
Secrets Officer
Lage og endre hemmeligheter

Secrets User
Lese – det appen trenger

Soft delete + purge protection
Slettet ≠ borte

```
az keyvault create -n kv-lab-$RANDOM -g $RG -l norwayeast --enable-rbac-authorization true
az keyvault secret set --vault-name <kv> -n DbPassword --value '<verdi>'
```

Azure Monitor og Log Analytics



Kilder → datalager → bruk

⚠️ **Vanlig feil:** Ressurslogger samles **ikke** inn før du lager en *diagnostic setting*.

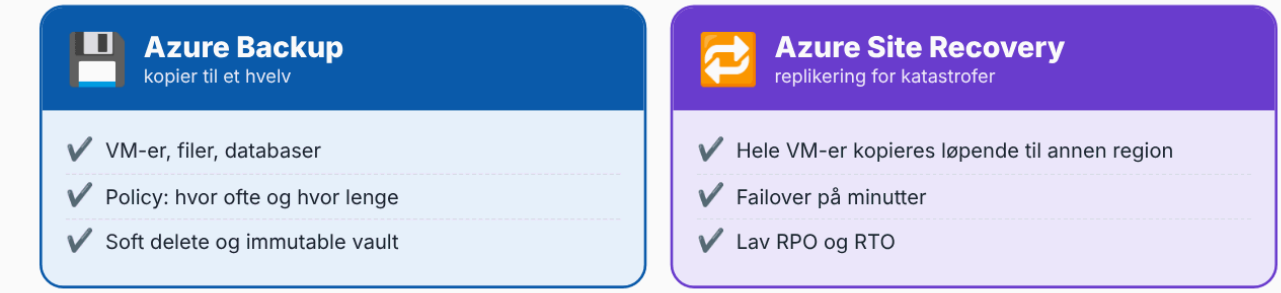
```
AzureActivity                                    // hvem gjorde hva siste døgn?
| where TimeGenerated > ago(1d)
| summarize antall = count() by Caller, OperationNameValue
| order by antall desc
```

KQL på 30 sekunder



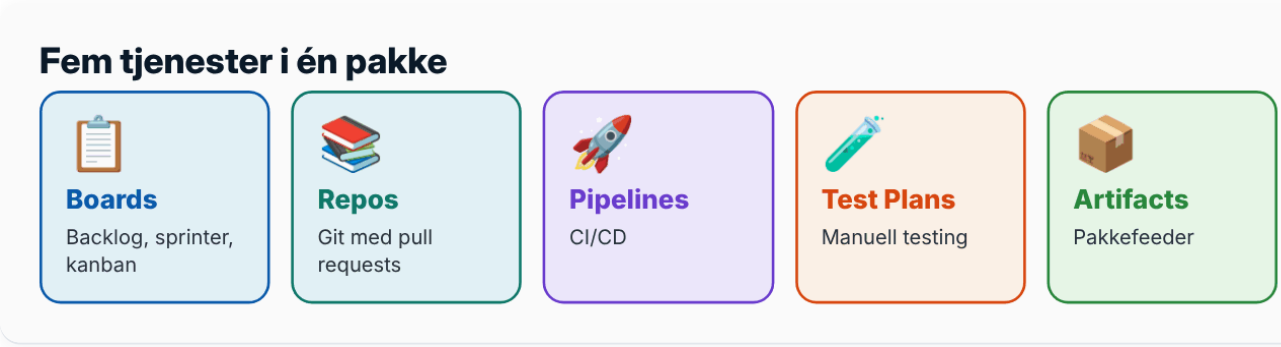
💰 **Kostnad:** Logger koster per GB. Samle det du faktisk bruker.

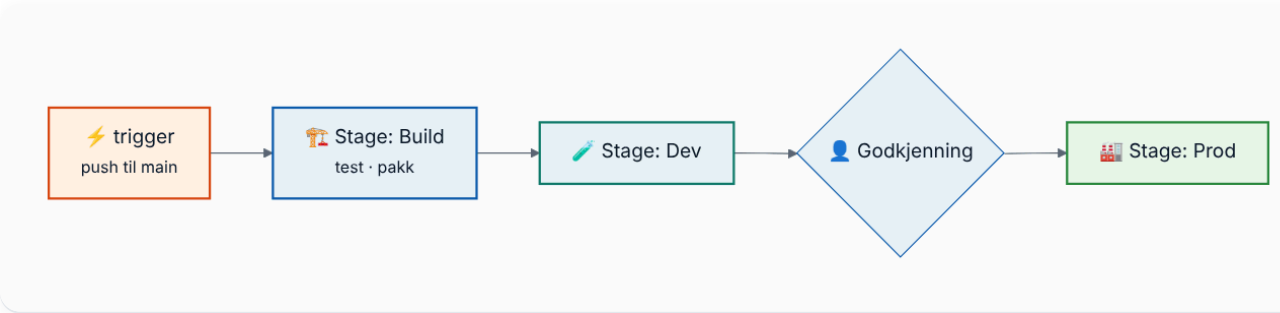
Azure Backup



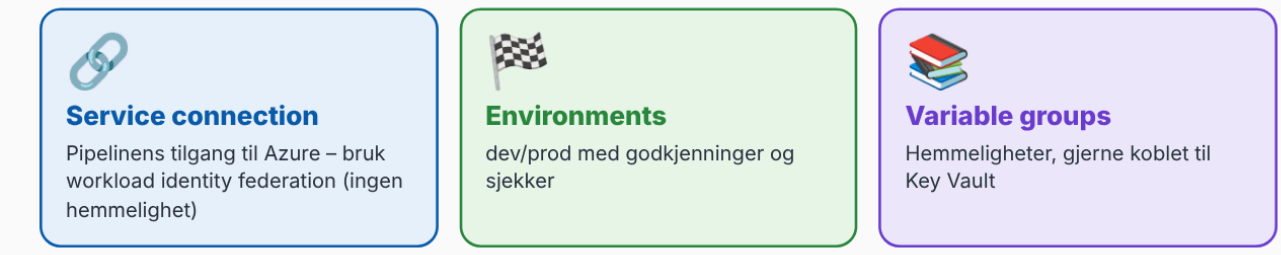
7.4 Azure DevOps og Bicep

Azure DevOps



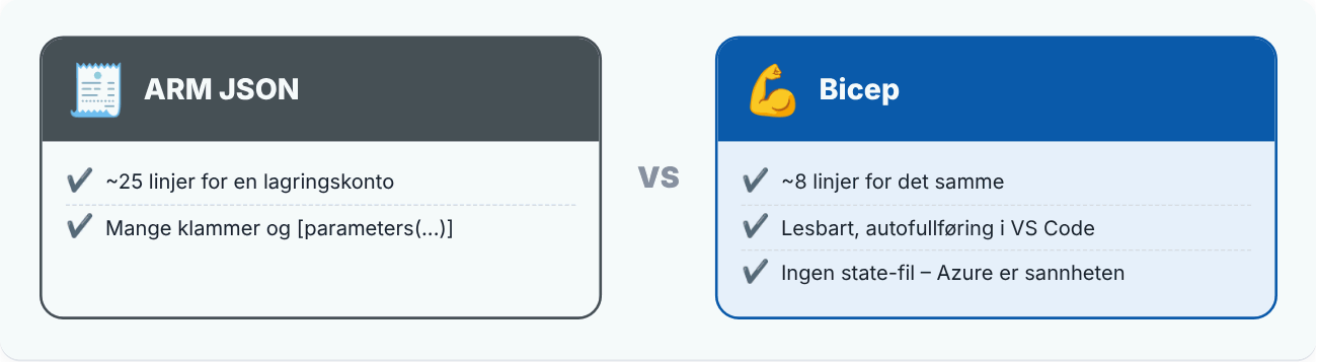


Strukturen i en Azure Pipeline



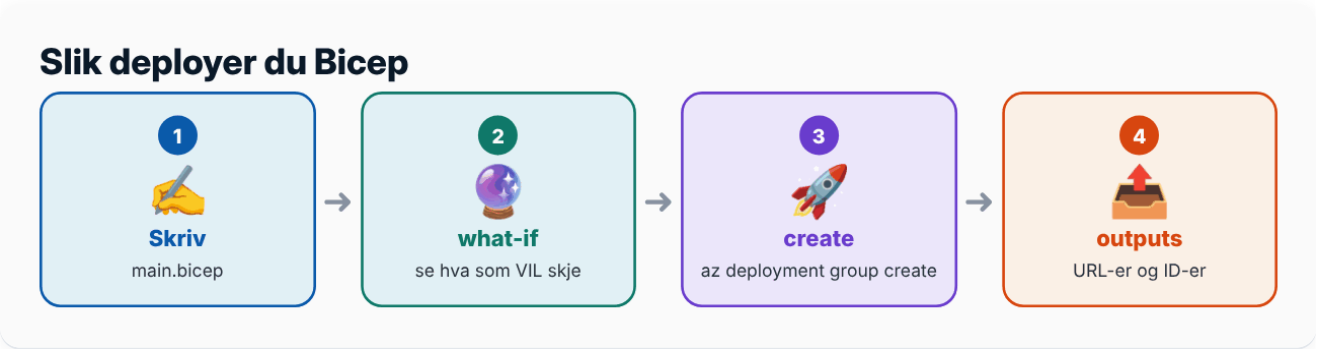
> 💡 **Azure DevOps vs. GitHub:** Samme prinsipper, ulik YAML. Du vil møte begge i norske bedrifter.

Bicep – infrastruktur som kode for Azure



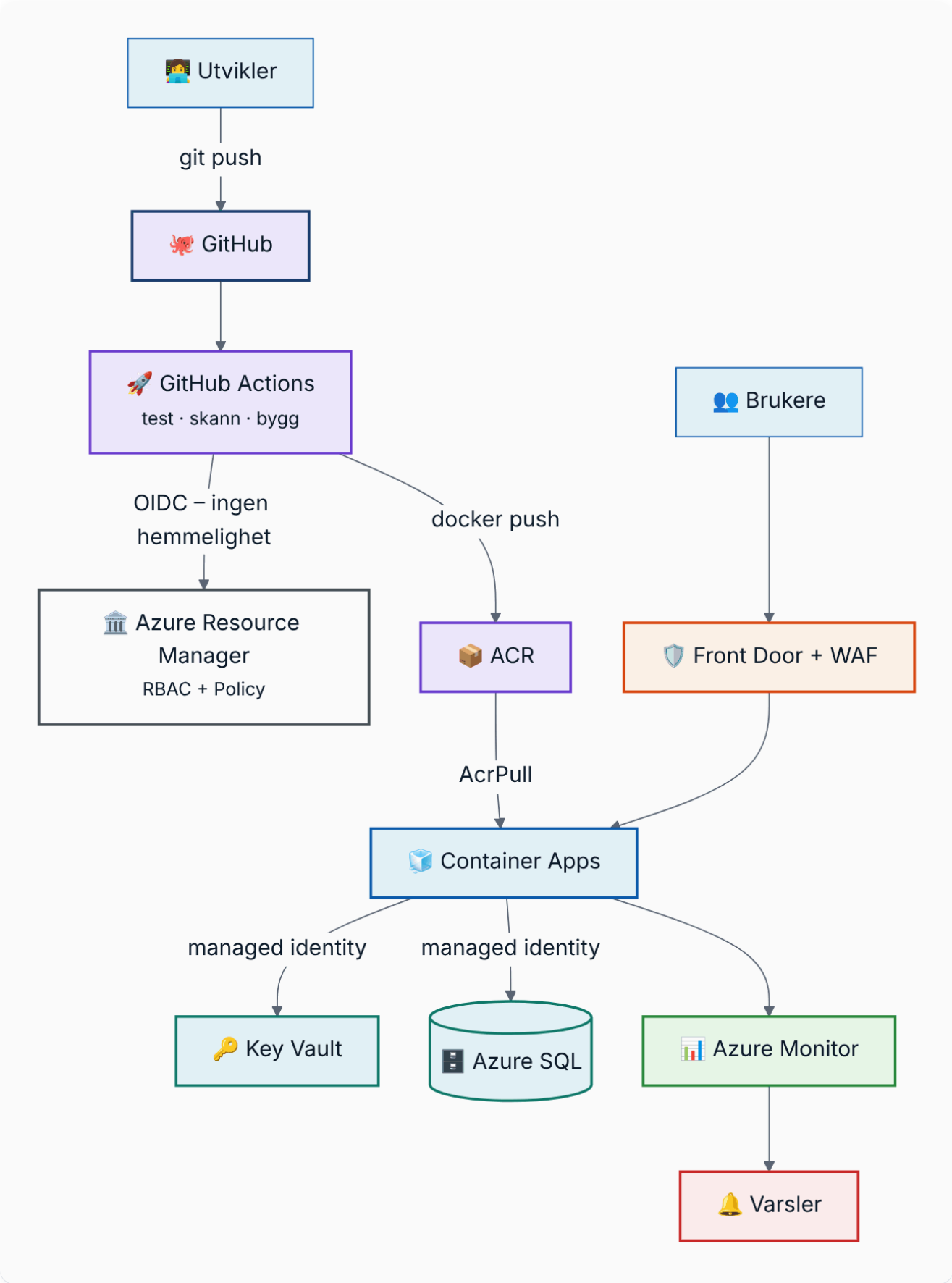

```
param location string = resourceGroup().location
param miljo string = 'dev'

resource st 'Microsoft.Storage/storageAccounts@2023-05-01' = {
  name: 'st${uniqueString(resourceGroup().id)}'
  location: location
  sku: { name: miljo == 'prod' ? 'Standard_ZRS' : 'Standard_LRS' }
  kind: 'StorageV2'
  properties: { minimumTlsVersion: 'TLS1_2', allowBlobPublicAccess: false }
}
```




```
az deployment group what-if -g rg-bicep -f main.bicep -p miljo=dev
az deployment group create  -g rg-bicep -f main.bicep -p miljo=dev
```

Referansearkitektur – alt sammen

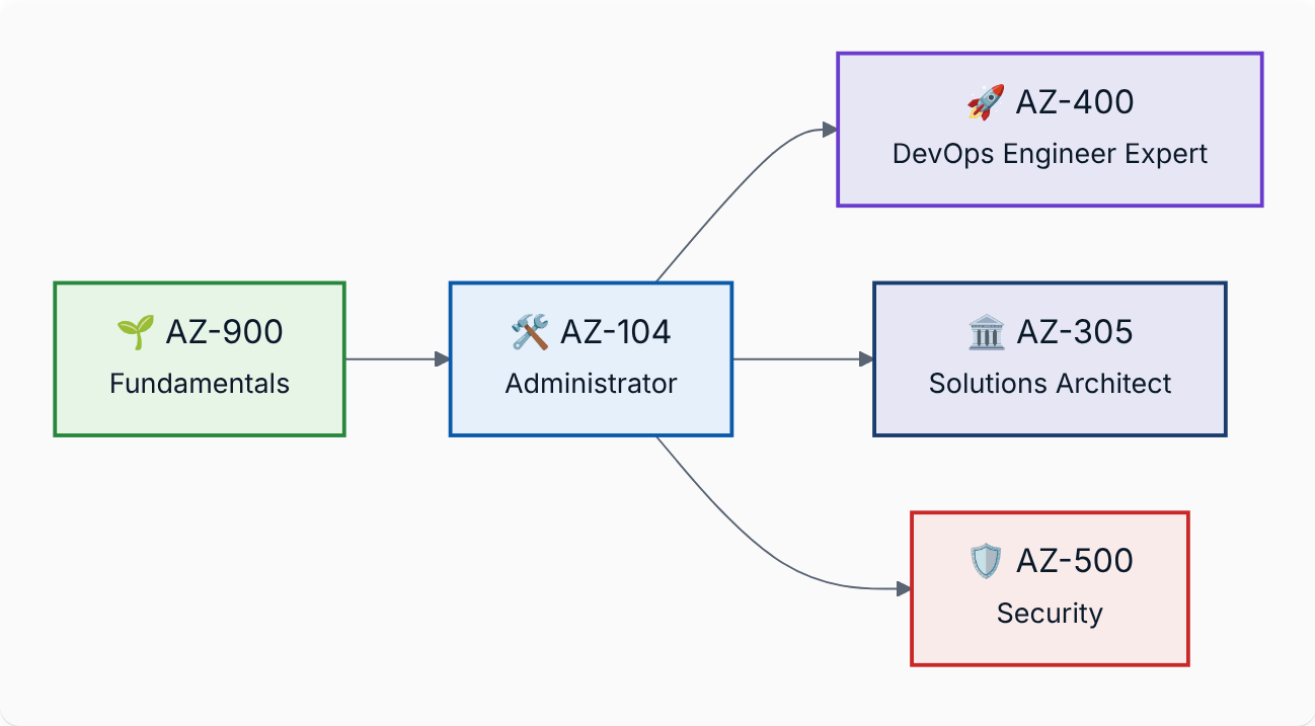


En moderne Azure-løsning – alt i del 7 i én figur

Del 7 – test deg selv

1. Subscription vs. resource group? → *Faktura/tilgangsgrense vs. mappe for ressurser.*
2. Stoppe ressurser utenfor Norway East? → *Azure Policy (Deny).*
3. Hvordan logger en app inn i Key Vault? → *Managed identity + «Key Vault Secrets User».*
4. NSG: tillat 22 på 200, avvis 22 på 150? → *Avvist – 150 kommer først.*
5. URL-ruting og WAF? → *Application Gateway (eller Front Door).*
6. Stopped vs. deallocated? → *Bare deallocated slutter å koste compute.*
7. Tåle sonefeil, bli i regionen? → *ZRS.*
8. Hvorfor ingen logger i Log Analytics? → *Mangler diagnostic setting.*

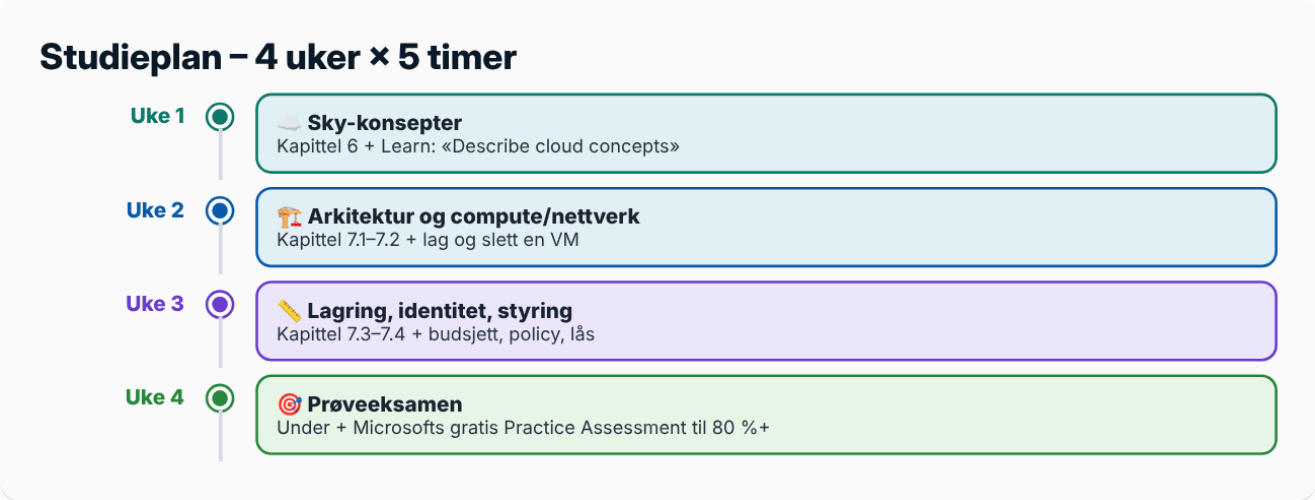
7.5 AZ-900 og AZ-104



Sertifiseringsveien i Azure (sjekk Microsoft Learn – porteføljen endres)

AZ-900 – Azure Fundamentals





Prøveeksamen AZ-900 (15 spørsmål)

1. Du betaler bare for det du bruker – hvilken egenskap? (a) elastisitet (b) forbruksbasert (c) HA
2. Flytte app med full OS-kontroll: (a) SaaS (b) PaaS (c) IaaS
3. Minst ansvar for kunden: (a) IaaS (b) PaaS (c) SaaS
4. Beskytte mot at et helt datasenter faller ut: (a) availability set (b) soner (c) større VM
5. Gruppere mange abonnementer for policy: (a) ressursgrupper (b) management groups (c) tags
6. Privat linje til Azure, ikke via internett: (a) VPN Gateway (b) ExpressRoute (c) peering
7. Billigst lagring, må hentes frem før lesing: (a) Cool (b) Cold (c) Archive
8. Krev MFA bare fra ukjent sted: (a) Policy (b) Conditional Access (c) RBAC
9. «Stol aldri, verifiser alltid»: (a) Defense in depth (b) Zero Trust
.0. Hindre både endring og sletting: (a) CanNotDelete (b) ReadOnly
.1. Estimer pris før du bygger: (a) Cost Management (b) Pricing Calculator
.2. Personlige spare- og sikkerhetsråd: (a) Advisor (b) Monitor (c) Arc
.3. Hendelser i Azure som rammer DINE ressurser: (a) Azure Status (b) Service Health
.4. Styre servere utenfor Azure fra Azure: (a) Arc (b) Migrate
.5. Alltid kundens ansvar: (a) fysisk sikkerhet (b) hypervisor (c) data og identiteter

Fasit: 1b · 2c · 3c · 4b · 5b · 6b · 7c · 8b · 9b · 10b · 11b · 12a · 13b · 14a · 15c

AZ-104 – Azure Administrator







Prøveeksamen AZ-104 (15 spørsmål)

1. Gi en gruppe rett til å administrere VM-er i én ressursgruppe – rolle og scope?
2. Sikre at ressurser uten taggen `kostnadssted` ikke kan lages?
3. Ekstern partner skal lese én blob i 24 timer – sikreste metode?

4. Lagring som tåler sonefeil, data skal bli i regionen?
5. Hva beskytter et availability set mot?
6. Teste ny versjon i prod-miljøet og bytte uten nedetid – funksjon og minste nivå?
7. App Service må nå en database i VNet via privat IP – hva konfigurerer du?
8. Kan `10.0.0.0/16` og `10.0.0.0/16` peeres?
9. Hvor mange VM-adresser i `10.1.5.0/28`?
10. Tving all trafikk ut via en brannmur-VM – hva lager du?
11. SSH til VM-er uten offentlig IP og uten VPN?
12. Helsesjekkene feiler, nginx kjører, NSG har DenyAll på 100 – årsak?
13. Hvilken regel tillot/blokkerte en pakke – verktøy?
14. E-post og SMS når en VM har vært nede i 5 min – to komponenter?
15. Gjenopprette flere VM-er i annen region med RPO på minutter?

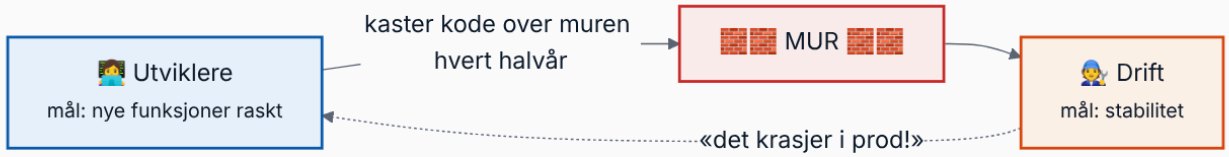
Fasit: 1 *Virtual Machine Contributor på ressursgruppen* · 2 *Azure Policy med Deny* · 3 *User delegation SAS, bare lese, 24 t* · 4 *ZRS* · 5 *Rack-/vertsfeil og vedlikehold i ett datasenter* · 6 *Deployment slots + swap, Standard* · 7 *VNet integration (+ private endpoint og privat DNS)* · 8 *Nei, overlapp* · 9 *11* · 10 *Route table med UDR 0.0.0.0/0 → virtual appliance* · 11 *Azure Bastion* · 12 *Regelen blokkerer AzureLoadBalancer-helsesjekkene* · 13 *Network Watcher IP flow verify* · 14 *Alert rule + action group* · 15 *Azure Site Recovery*

8 · DevOps Engineering

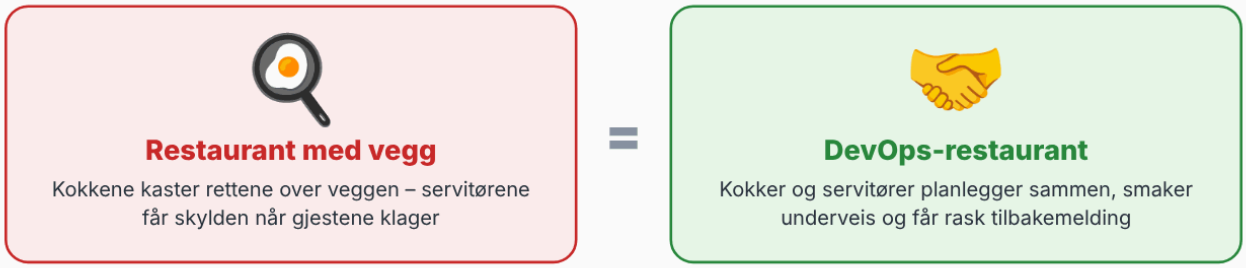
Kapittelet på én side



Muren som DevOps rev



Før DevOps – motstridende mål og skyld



💡 **Viktigste innsikt:** Fart og stabilitet er **ikke** motsetninger. Små, hyppige endringer er lettere å teste, forstå og rulle tilbake.

DevOps-kultur – CALMS



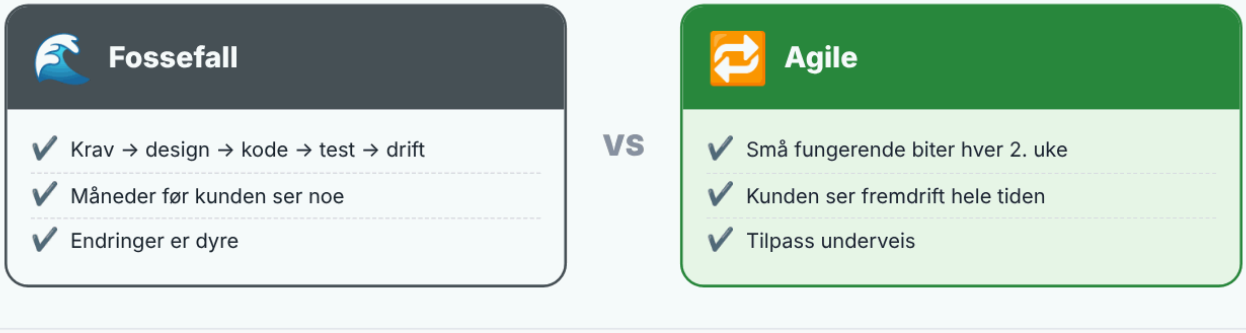
DORA-metrikkene – slik måler du DevOps

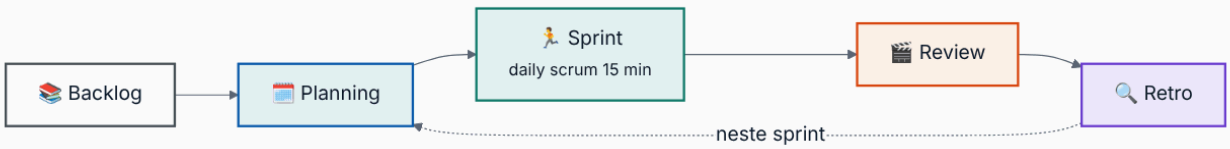


To måler fart, to måler stabilitet – de beste teamene er gode på begge

> 🧑‍⚖️ **Blameless:** Spør «hvordan lot systemet dette skje?», ikke «hvem gjorde feil?».

Agile og Scrum





En Scrum-sprint (ofte 2 uker)

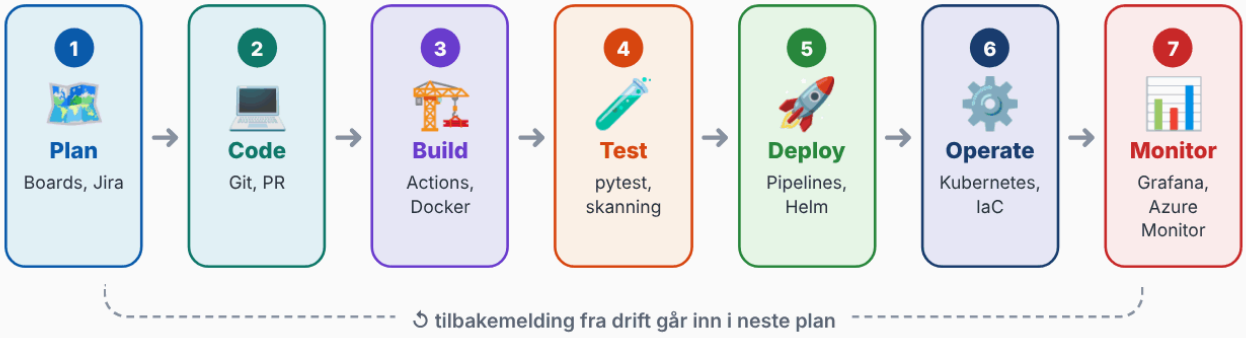
Rollene i Scrum



📋 **Kanban:** kontinuerlig flyt uten sprinter, med grense for pågående arbeid – populært i drifts- og plattformteam.

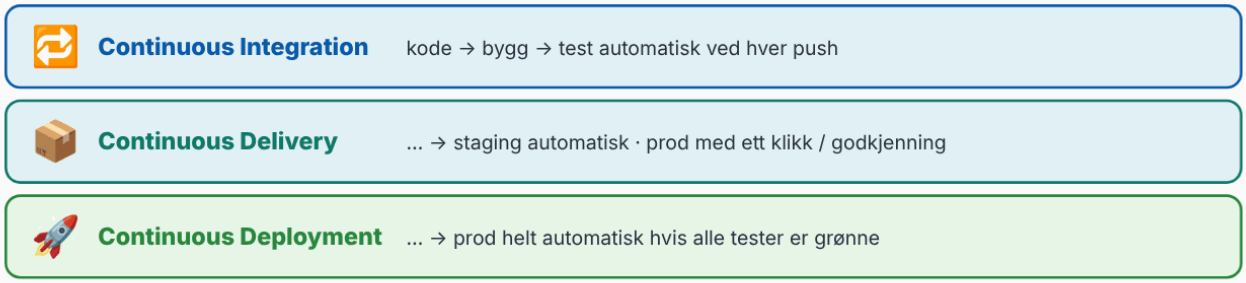
DevOps-livssyklusen

♾️ Uendelighetssløyfen



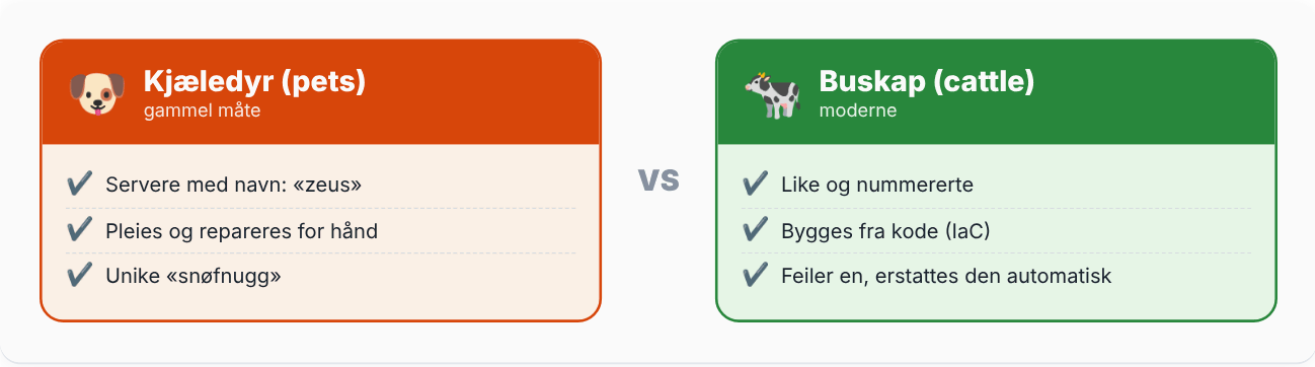
CI, Continuous Delivery og Continuous Deployment

Hvor langt går automatikken?



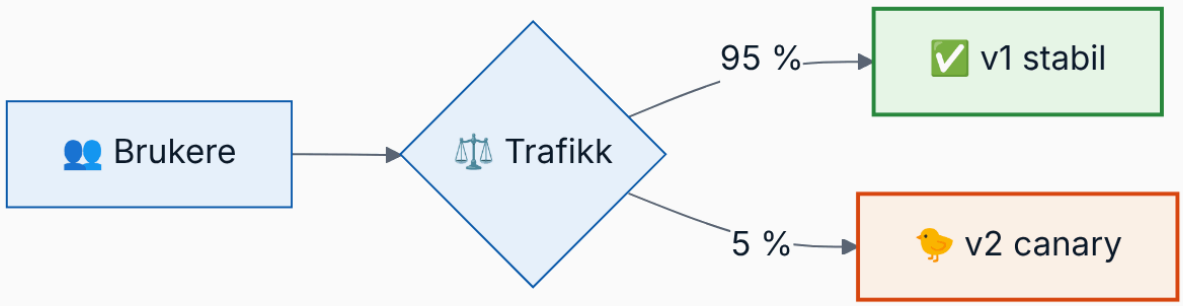
⚠️ **Vanlig feil:** «CD» betyr ikke alltid automatisk til prod. Banker har ofte Continuous **Delivery** med manuell godkjenning – helt legitimt.

Infrastruktur som buskap



Deploy-strategier





Canary – bare noen få brukere ser den nye versjonen først

Monitoring og observability

Observability – tre søyler

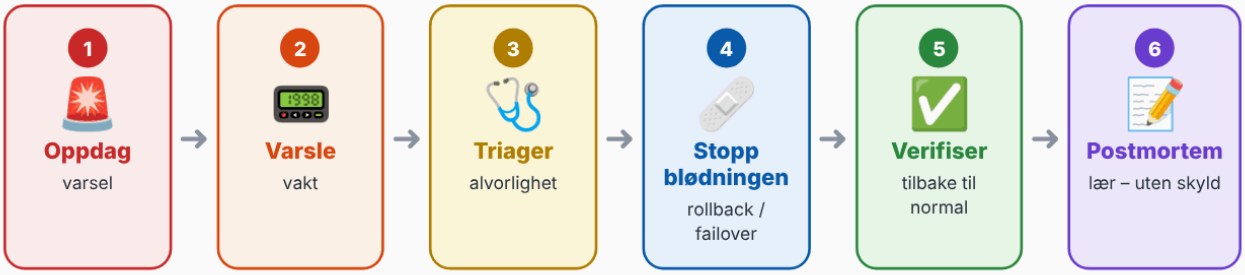


De fire gylne signalene



Hendelser og SRE

Hendelseshåndtering

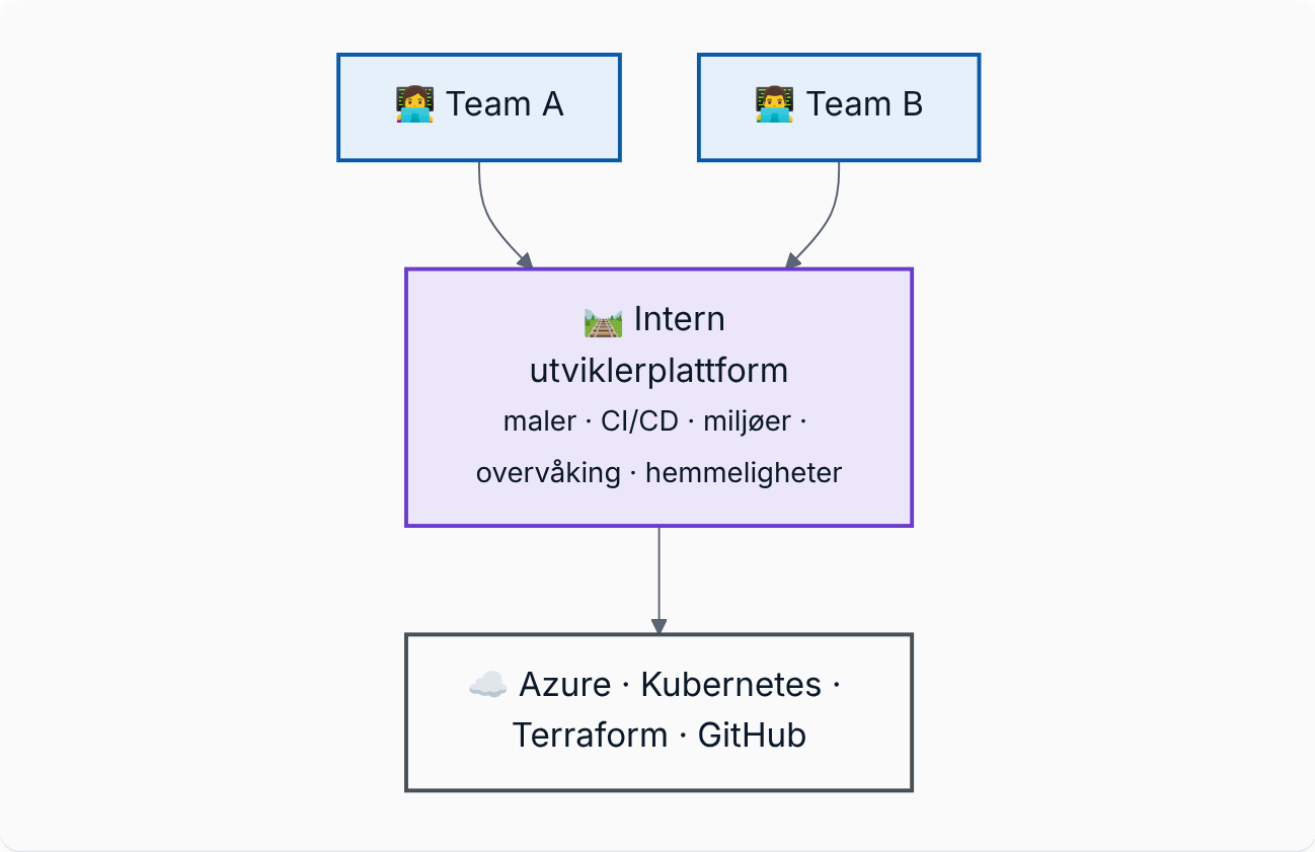


SRE-begrepene



💡 **Husk:** SLA ≤ SLO ≤ det systemet faktisk leverer. Er error budget brukt opp → prioriter stabilitet foran nye funksjoner.

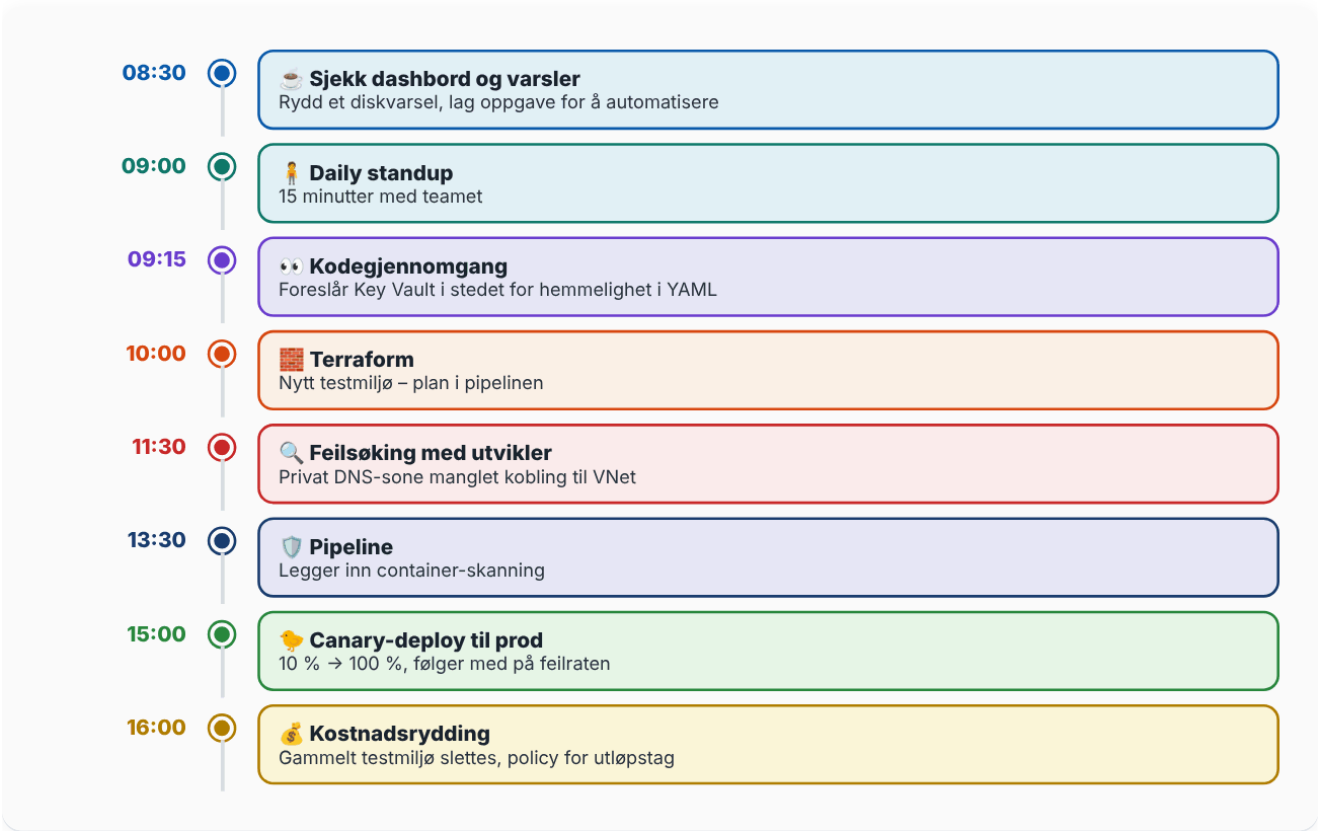
Platform Engineering og DevSecOps



En intern utviklerplattform gir produktteamene selvbetjening og «golden paths»



En dag som DevOps Engineer

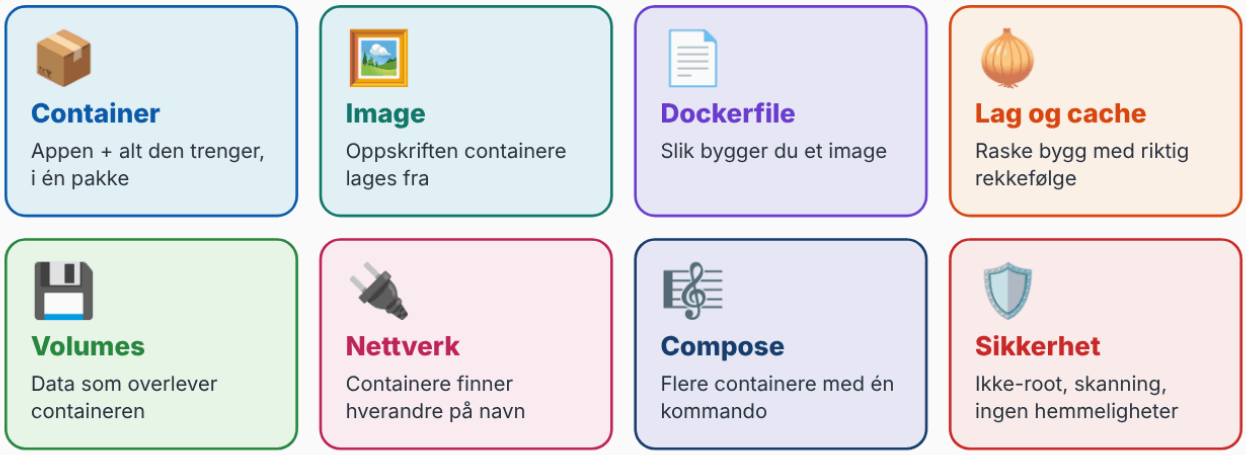


Test deg selv

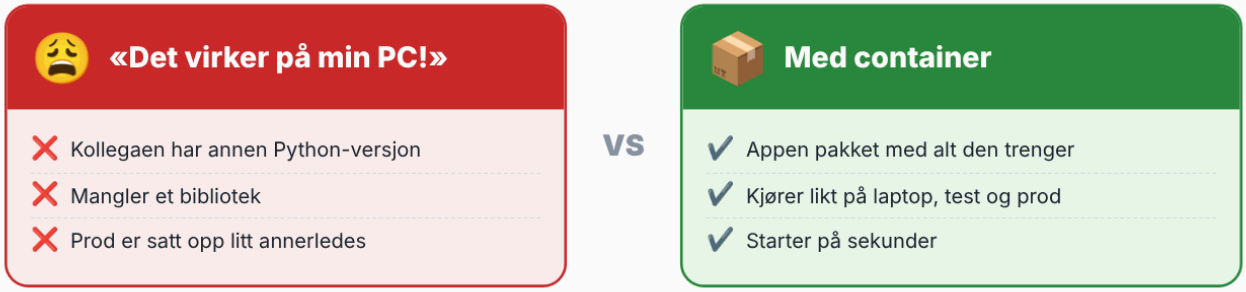
1. Hva står CALMS for? → *Culture, Automation, Lean, Measurement, Sharing.*
2. De fire DORA-metrikkene? → *Frekvens, ledetid, feilrate, gjenopprettingstid.*
3. Product Owner vs. Scrum Master? → *Hva vs. prosess.*
4. Delivery vs. Deployment? → *Manuell vs. automatisk til prod.*
5. SLO vs. SLA? → *Internt mål vs. kundeavtale.*
6. «Shift left»? → *Sikkerhet og testing tidligere.*

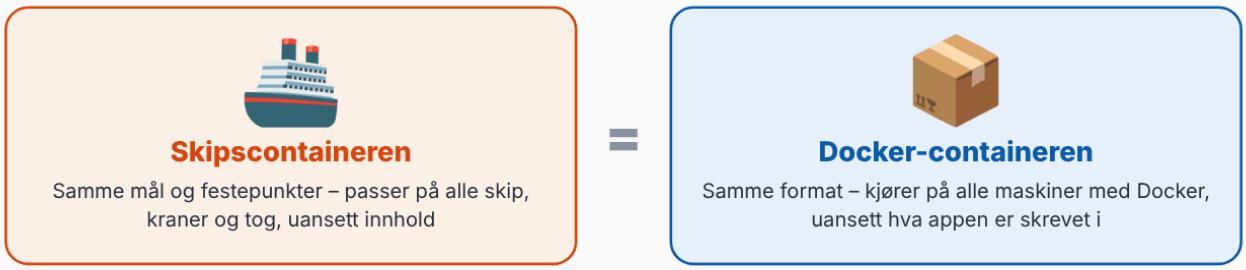
9 · Docker og containere

Kapittelet på én side



Hvorfor containere?





Varene → Appen og avhengighetene

Havnekranen → Docker Engine

Skipet → Serveren

Lastemanifestet → Dockerfile

Container vs. virtuell maskin

Virtuell maskin
hvert hus har egen grunnmur

- ✔ Eget operativsystem og kjerne
- ✔ Gigabyte stor
- ✔ Starter på minutter
- ✅ Sterk isolasjon

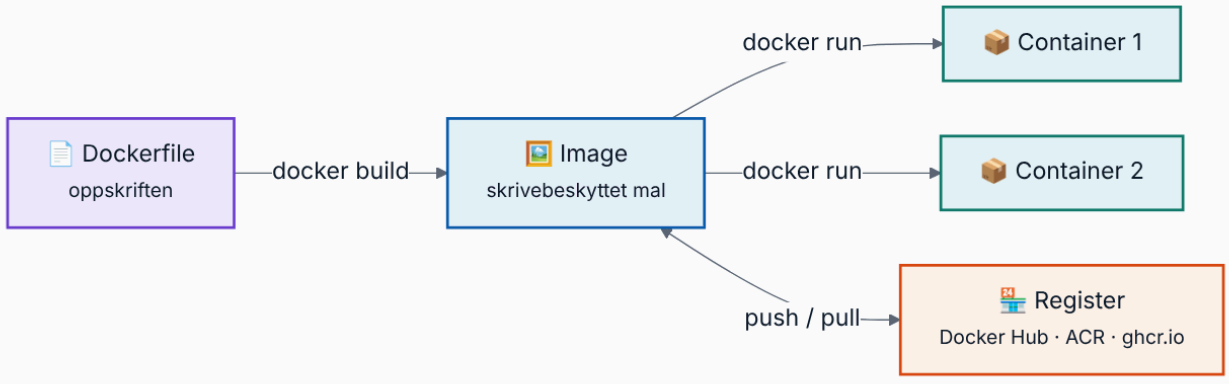
Container
leiligheter deler grunnmuren

- ✔ Deler vertens Linux-kjerne
- ✔ Megabyte liten
- ✔ Starter på sekunder
- ⚠ Svakere isolasjon

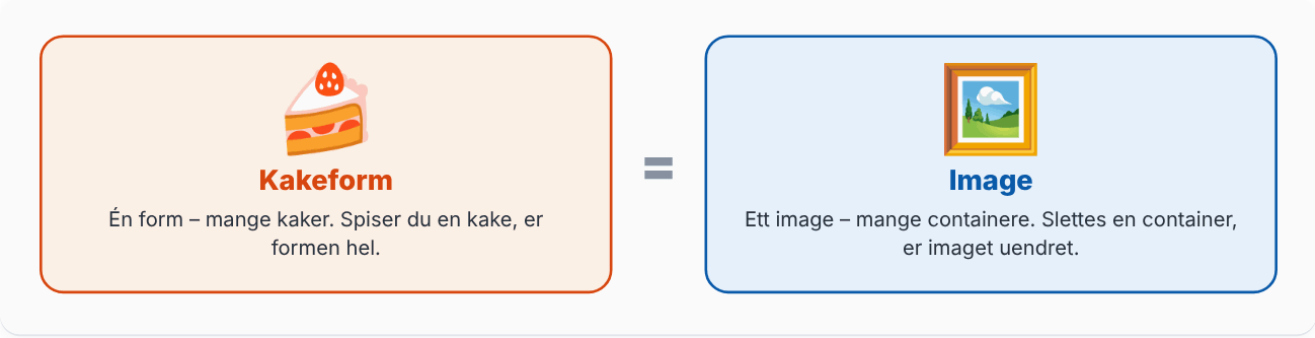
Containere på en server

App A · App B · App C	hver med sine biblioteker
Container-runtime	Docker / containerd
Verts-OS (Linux-kjerne)	namespaces isolerer · cgroups begrenser CPU og minne
Maskinvare (ofte en VM i skyen)	

Image og container



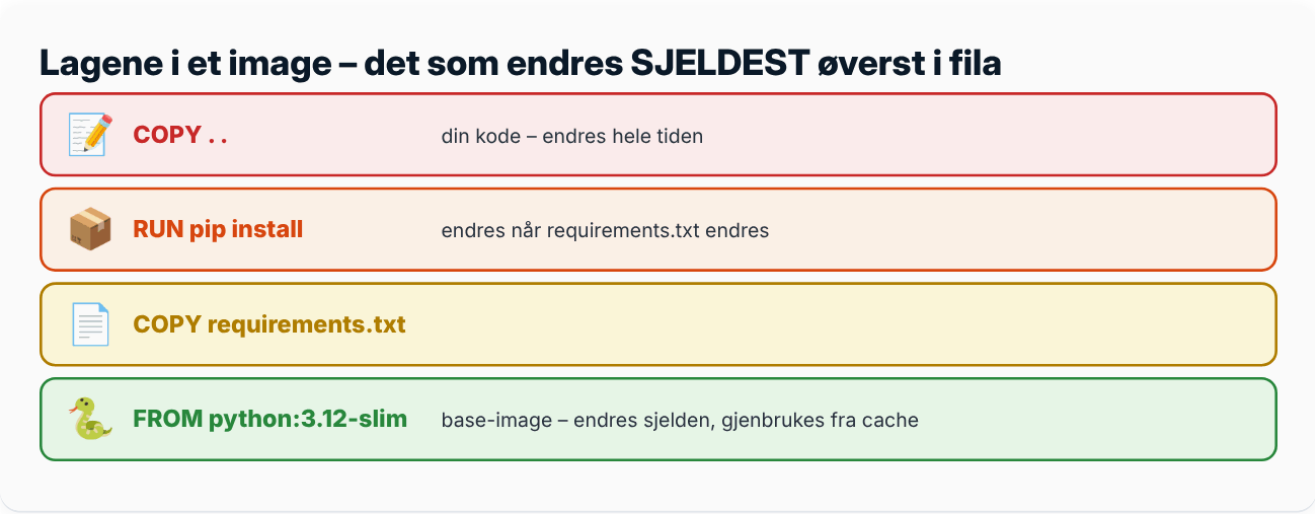
Fra oppskrift til kjørende containere



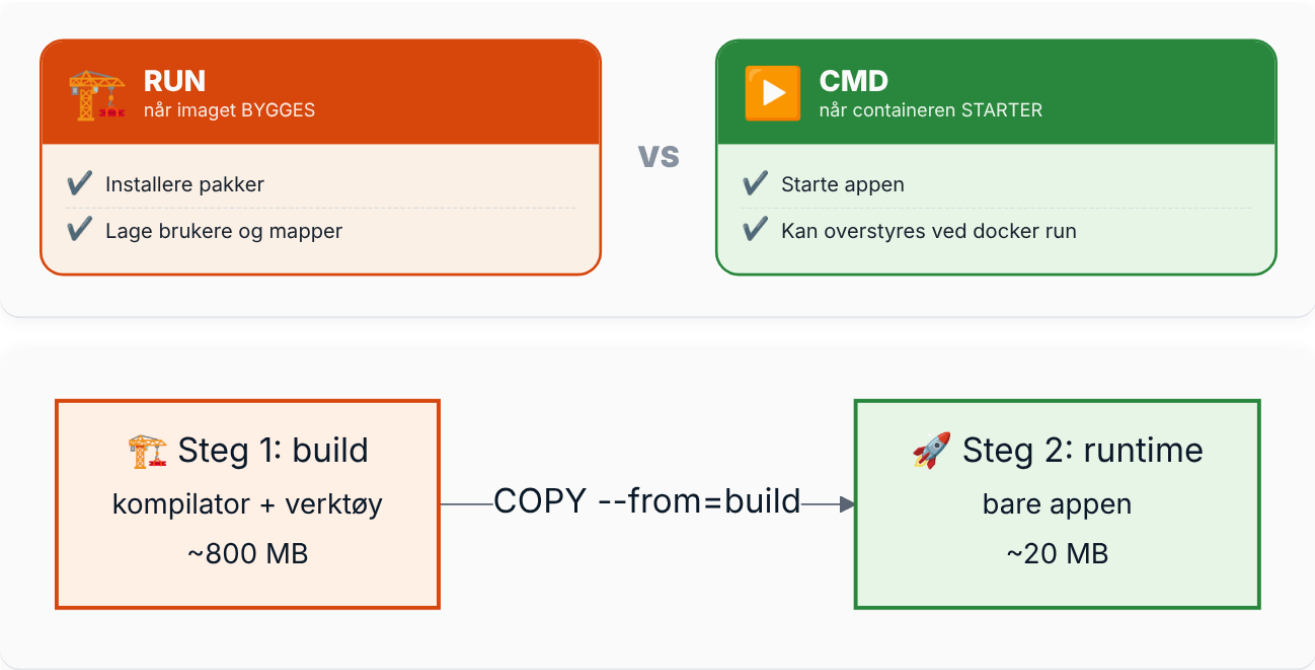

```
$ docker run -d -p 8080:80 --name web nginx:1.27     # -p VERT:CONTAINER
$ docker ps · docker logs -f web · docker exec -it web sh
$ docker stop web && docker rm web
```

⚠️ **Vanlig feil:** `:latest` i produksjon. Bruk faste versjoner – f.eks. commit-SHA som tag.

Dockerfile og lag

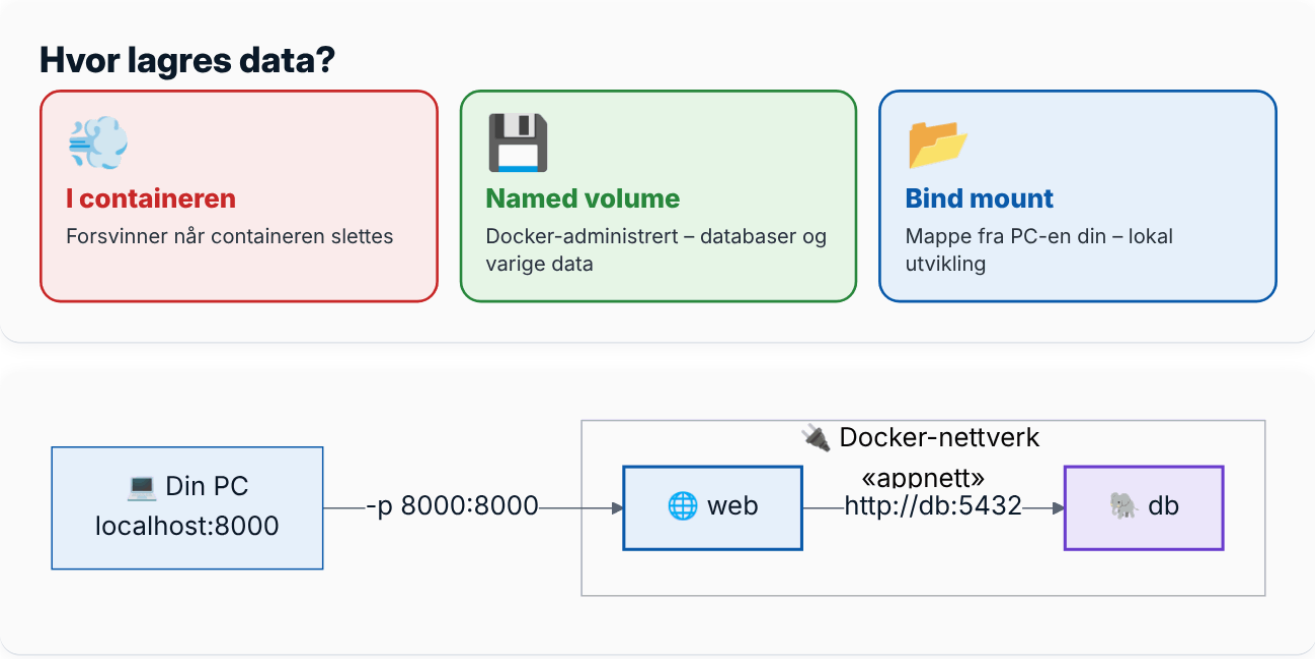



```
FROM python:3.12-slim                # fast, slank base
WORKDIR /app
COPY requirements.txt .              # avhengigheter først → cache
RUN pip install --no-cache-dir -r requirements.txt
COPY . .                             # så koden
RUN useradd --uid 10001 appuser
USER appuser                         # ikke root!
EXPOSE 8000                          # dokumentasjon – åpner ingenting
CMD ["gunicorn", "--bind", "0.0.0.0:8000", "app:app"]   # 0.0.0.0, ikke 127.0.0.1
```



Multi-stage build – bygg med alle verktøy, send bare resultatet videre

Volumes og nettverk

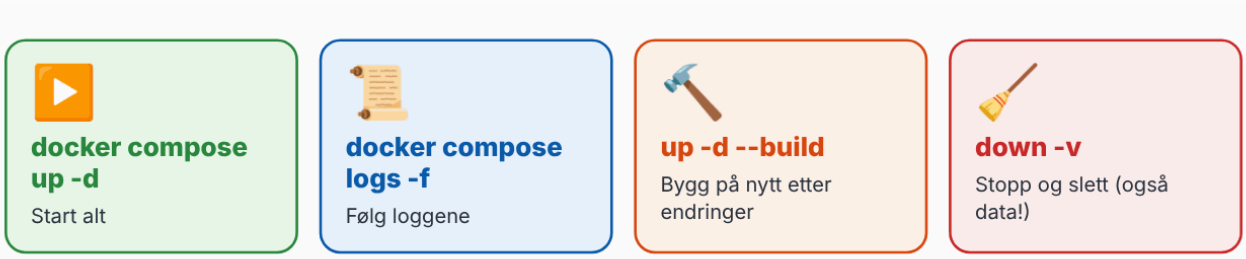


På samme Docker-nettverk når containere hverandre på navn

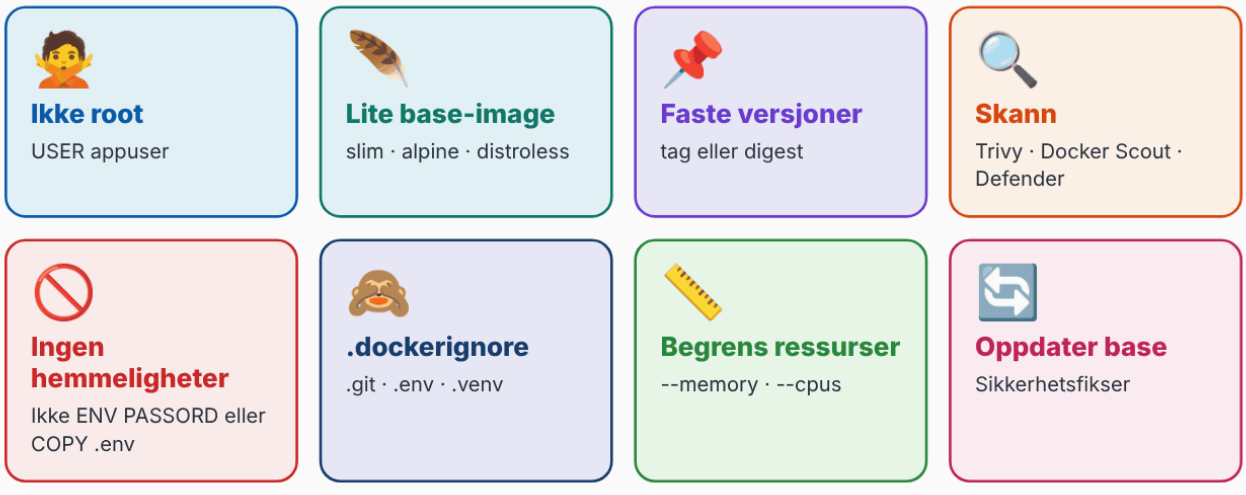
⚠️ **Vanlig feil:** Inne i en container betyr `localhost` containeren selv. Bruk tjenestenavnet (`db`).

Docker Compose

```yaml
services:
  web:
    build: .
    ports: ["8000:8000"]
    environment:
      DATABASE_URL: postgresql://app:${DB_PASSWORD}@db:5432/appdb
    depends_on:
      db: { condition: service_healthy }
  db:
    image: postgres:16
    environment: { POSTGRES_USER: app, POSTGRES_PASSWORD: "${DB_PASSWORD}", POSTGRES_DB:
appdb }
    volumes: [pgdata:/var/lib/postgresql/data]
    healthcheck: { test: ["CMD-SHELL", "pg_isready -U app"], interval: 5s }
volumes:
  pgdata:
```

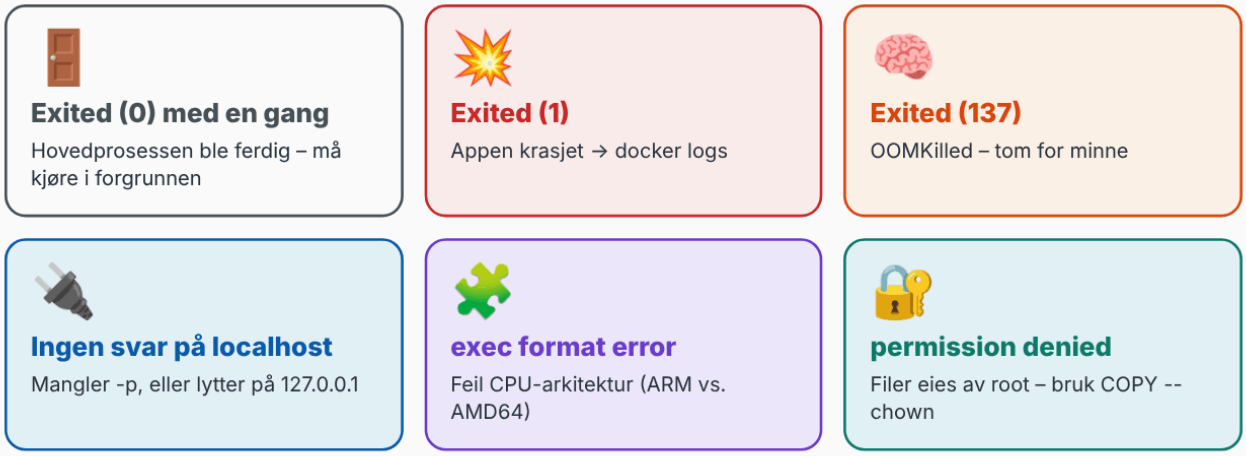


Sikkerhet



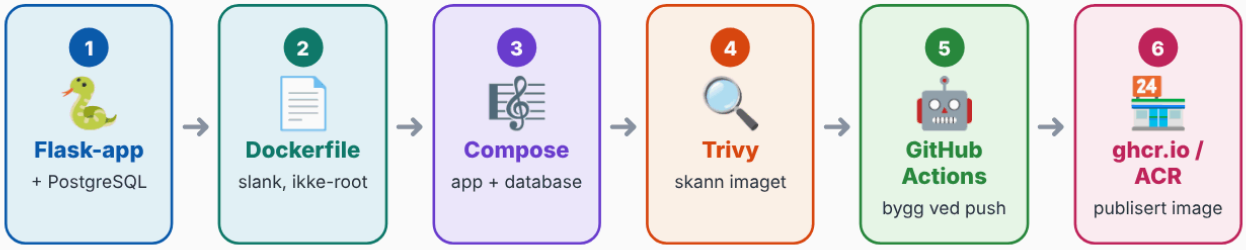
Feilsøking

Symptom → årsak



Prosjekt i korte trekk

Bygg, kjør og publiser



Test deg selv

1. Image vs. container? → *Mal vs. kjørende instans.*
2. Hva deler containere? → *Vertens kjerne.*
3. Hvorfor `requirements.txt` før koden? → *Lag-cache.*
4. Åpner `EXPOSE` porten? → *Nei – bruk* `-p`.
5. Hva betyr exit 137? → *Drept, ofte OOMKilled.*
6. Passord slettet i senere lag – borte? → *Nei, ligger i tidligere lag.*

10 · Kubernetes

Kapittelet på én side

Hvorfor?
Orkestrerer mange containere på mange maskiner

Arkitektur
Kontrollplan + noder

Pod og Deployment
Kjør og hold riktig antall

Service og Ingress
Stabil adresse og HTTP-ruting

ConfigMap og Secret
Konfigurasjon utenfra

Lagring
PVC og StorageClass

Autoskalering og Helm
HPA og pakker

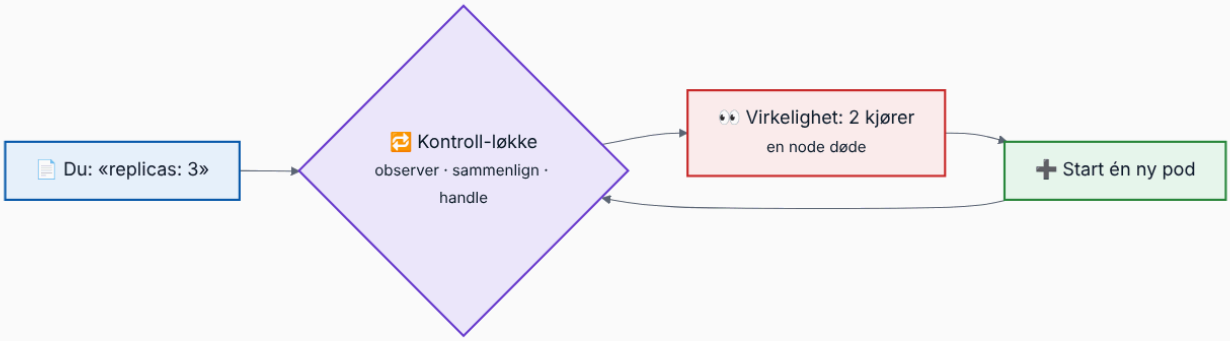
Sikkerhet og AKS
RBAC, nettverkspolicy, Azure

Hvorfor Kubernetes?

Dirigenten
Du gir noten. Dirigenten plasserer musikerne – besvimer en fiolinist, hentes en ny fra reservebenken.

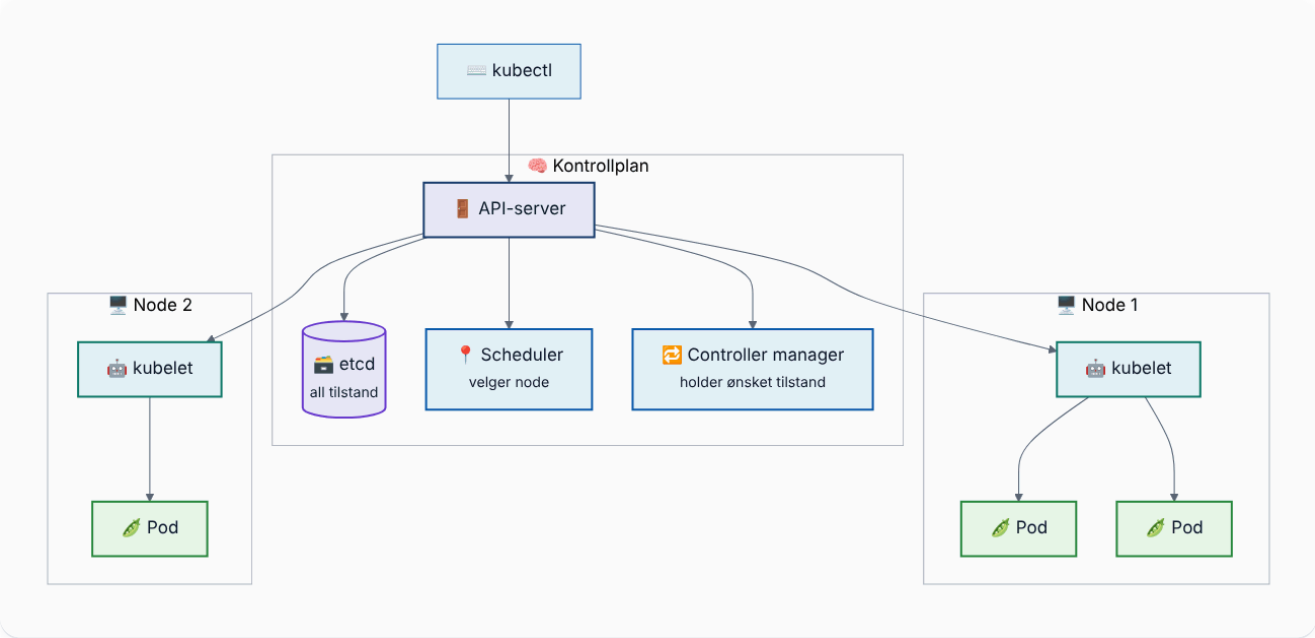
=

Kubernetes
Du sier «3 kopier av web:v2». Kubernetes plasserer dem – dør en, startes en ny automatisk.



Ønsket tilstand – Kubernetes jobber hele tiden for at virkeligheten skal matche

Arkitekturen



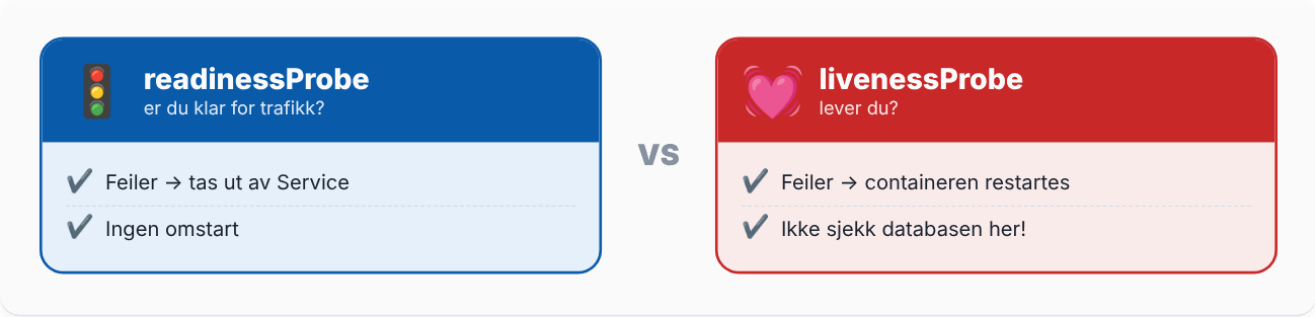
Et Kubernetes-cluster – i AKS drifter Microsoft kontrollplanet

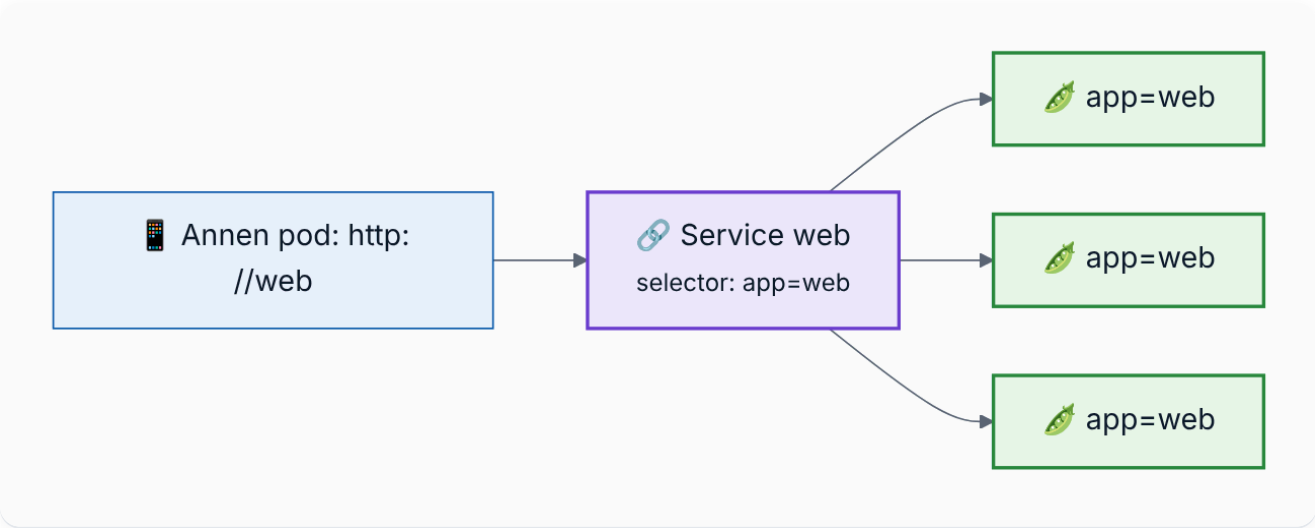
Pod, Deployment og Service

Byggeklossene – fra innerst til ytterst

Container	appen din
Pod	1+ containere med felles IP – minste enhet, dødelig
ReplicaSet	sørger for riktig antall pods
Deployment	ønsket tilstand + rullerende oppdateringer + rollback
Service	stabil IP/DNS-navn foran podene

```yaml
apiVersion: apps/v1
kind: Deployment
metadata: { name: web }
spec:
  replicas: 3
  selector: { matchLabels: { app: web } }
  template:
    metadata: { labels: { app: web } }          # MÅ matche selector
    spec:
      containers:
        - name: web
          image: nginx:1.27
          resources:
            requests: { cpu: 100m, memory: 128Mi }   # garantert
            limits: { memory: 256Mi }                 # over dette → OOMKilled
          readinessProbe: { httpGet: { path: /, port: 80 } }
```





Service finner podene via labels og fordeler trafikken

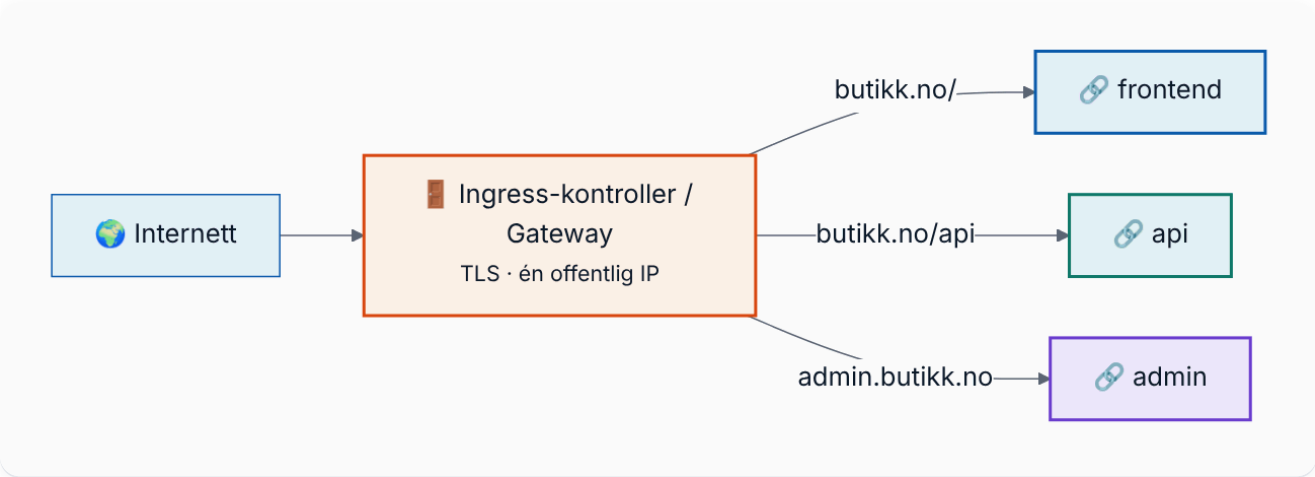
Service-typer

ClusterIP
Bare inne i clusteret (standard)

NodePort
Port på hver node – testing

LoadBalancer
Ekstern IP fra skyens lastbalanserer

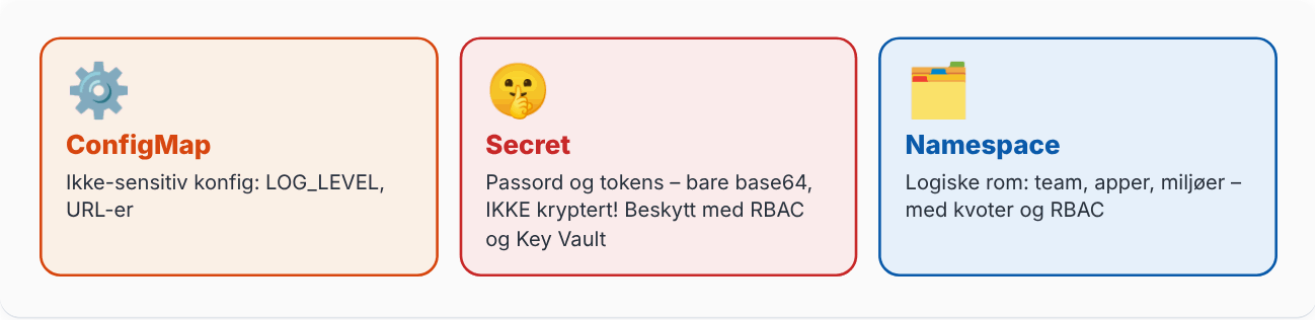
Ingress og Gateway API



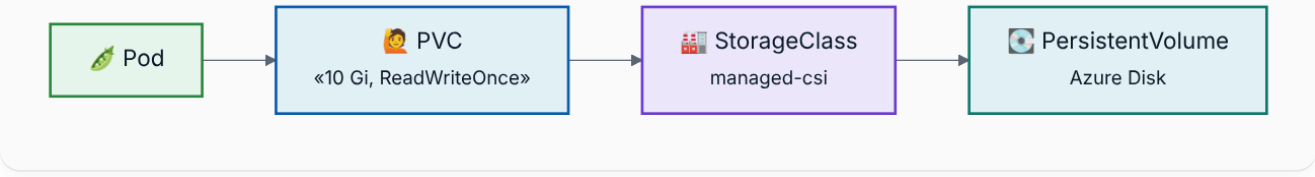
Én inngang – HTTP-ruting til mange tjenester

🆕 **Utvikling:** ingress-nginx-prosjektet er kunngjort pensjonert; **Gateway API** er etterfølgeren til Ingress. Lær begge – du møter Ingress i eksisterende clustere.

ConfigMap, Secret og Namespace

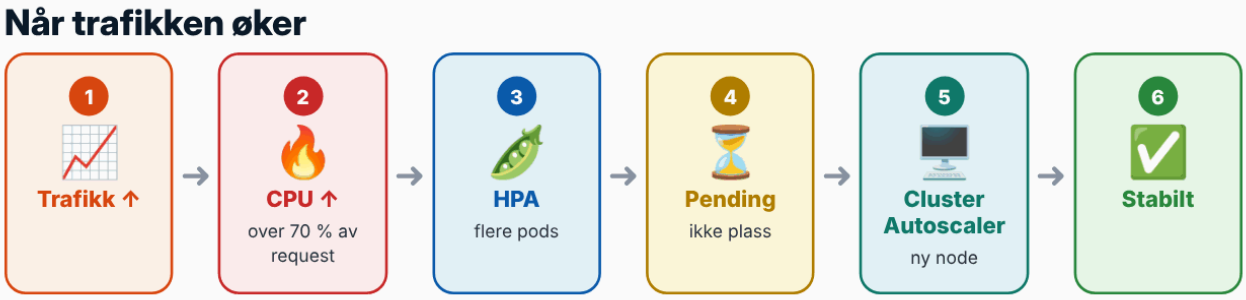


Lagring

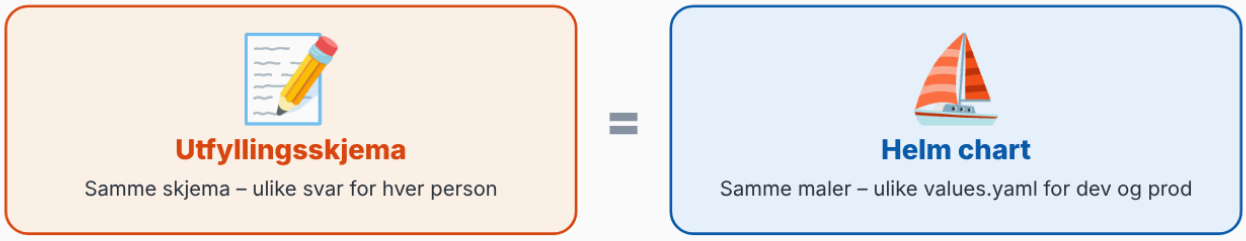


Appen ber om lagring – StorageClass lager en ekte Azure-disk automatisk

Autoskalering



Helm



Skjemaet → templates/

Svarene → values.yaml

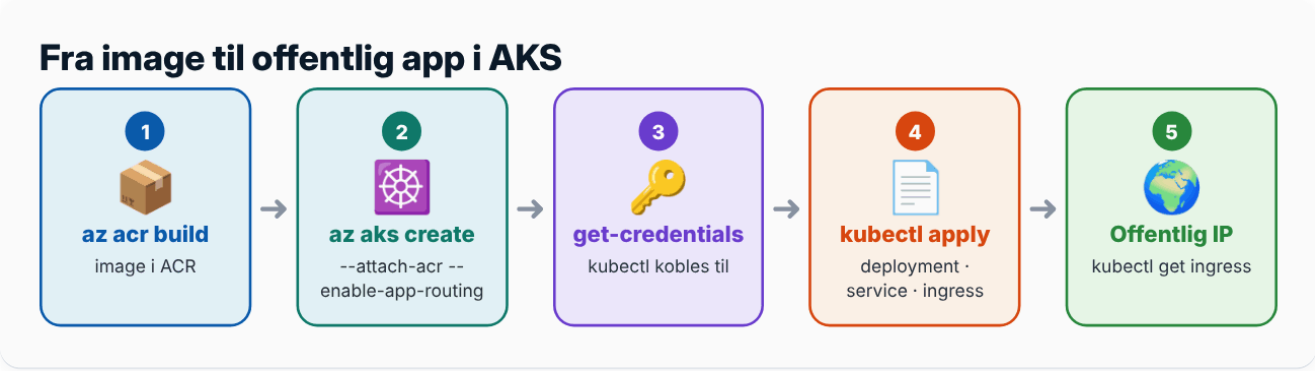
Utfylt skjema → Release

```
$ helm upgrade --install web ./minapp -f values-prod.yaml -n butikk
$ helm rollback web 1 -n butikk
```

Sikkerhet



AKS i praksis



💰 **Kostnad:** Nodene er VM-er. `az aks stop` – og slett clusteret samme dag.

Feilsøking



Test deg selv

1. Hva betyr deklarativt? → *Du beskriver ønsket tilstand, Kubernetes ordner resten.*
2. Hva lagres i etcd? → *Hele clusterets tilstand.*
3. Pod vs. Deployment? → *Én instans vs. ønsket antall + oppdateringer.*
4. Minne- vs. CPU-limit overskredet? → *Drept vs. strupet.*
5. Er Secrets kryptert? → *Nei, bare base64.*
6. Hva krever HPA? → *metrics-server og resource requests.*

11 · Infrastructure as Code

Kapittelet på én side

Hva er IaC?
Infrastruktur beskrevet i kode

Deklarativt
Beskriv sluttresultatet – idempotent

Terraform
init → plan → apply

State
Kobling mellom kode og virkelighet

Moduler
Gjenbrukbare byggeklosser

Bicep
Azures eget språk – uten state

Ansible
Konfigurer innsiden av servere

GitOps
Git er sannheten

Hvorfor IaC?

Klikk i portalen

- ❌ Husker ingen alle innstillingene
- ❌ Testmiljøet blir aldri helt likt
- ❌ Ingen historikk eller gjennomgang
- ❌ Vanskelig å bygge opp igjen

vs

Infrastruktur som kode

- ✔ Samme kode = samme resultat
- ✔ Git: hvem, hva, hvorfor
- ✔ Pull request før endring
- ✔ Gjenoppbygg alt på minutter

Matlaging uten oppskrift
Kan bli godt – men aldri likt to ganger, og ingen andre kan gjenta det

=

Oppskrift
Presis, delbar og forbedrebar – det er IaC

Deklarativt
HVA du vil ha

- ✔ «Det skal finnes et VNet med 3 subnett»
- ✔ Kjør to ganger → ingen endring (idempotent)
- ✔ Terraform · Bicep · Kubernetes YAML

Imperativt
HVORDAN – stegene

- ✔ «Kjør az network vnet create, så ...»
- ✔ Kjør to ganger → feil eller duplikater
- ✔ Bash-/Python-skript

Provisionering
lage infrastrukturen

- ✔ Terraform · Bicep · Pulumi
- ✔ VNet, VM, database, Key Vault

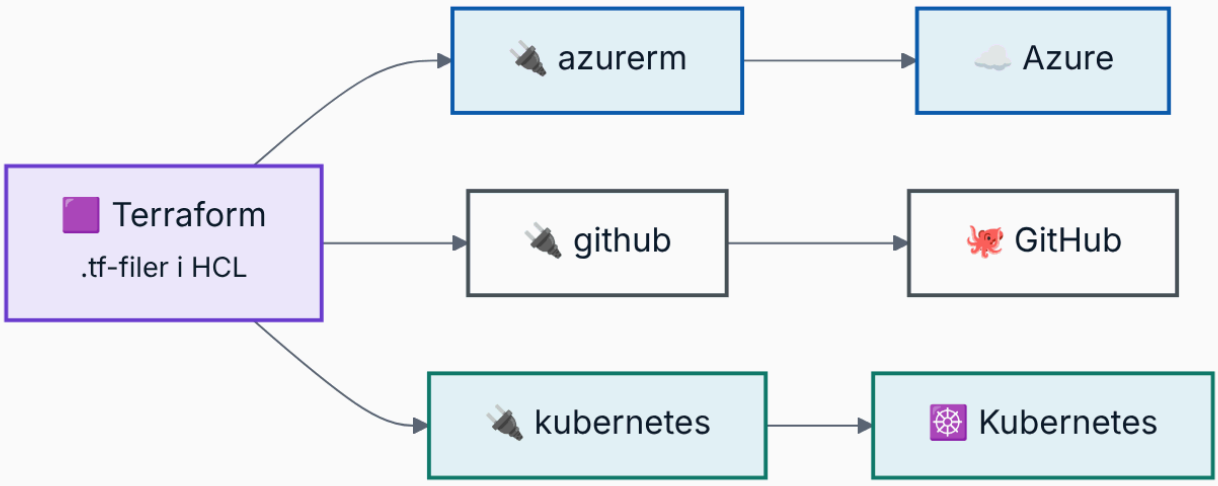
Konfigurasjonsstyring
konfigurere innsiden

- ✔ Ansible · cloud-init
- ✔ Pakker, filer, brukere, tjenester

Terraform

Terraform-arbeidsflyten

1. **Skriv .tf** — providers, resources
2. **init** — last ned providers, koble state
3. **plan** — + lages · ~ endres · − slettes
4. **apply** — gjør endringene
5. **destroy** — slett alt



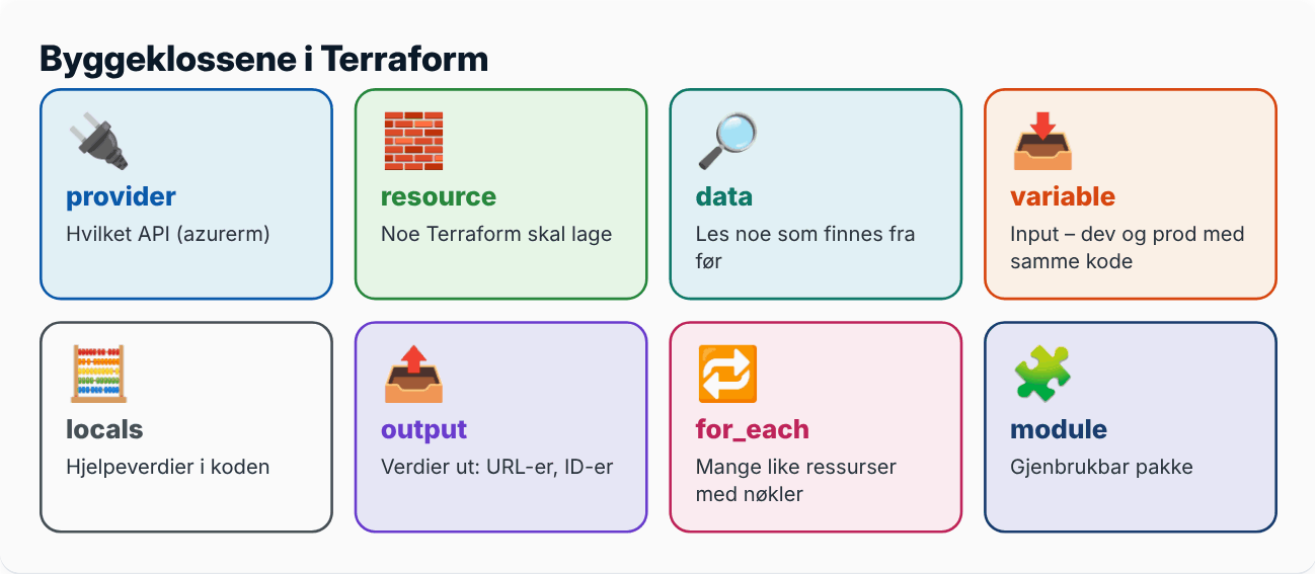
Terraform snakker med API-ene via providers

```
terraform {
  required_providers {
    azurerm = { source = "hashicorp/azurerm", version = "~> 4.0" }
  }
}
provider "azurerm" {
  features {}
  subscription_id = var.subscription_id
}

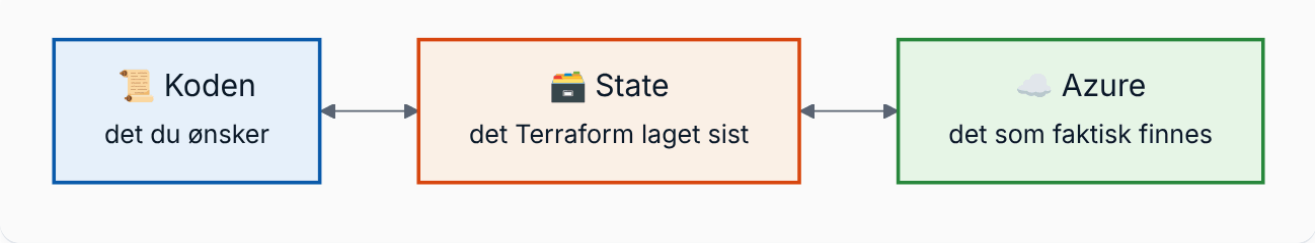
resource "azurerm_resource_group" "main" {        # type + lokalt navn
  name     = "rg-${var.miljo}"
  location = var.location
}

resource "azurerm_virtual_network" "main" {
  name                = "vnet-${var.miljo}"
  resource_group_name = azurerm_resource_group.main.name   # referanse = avhengighet
  location            = azurerm_resource_group.main.location
  address_space       = ["10.0.0.0/16"]
}

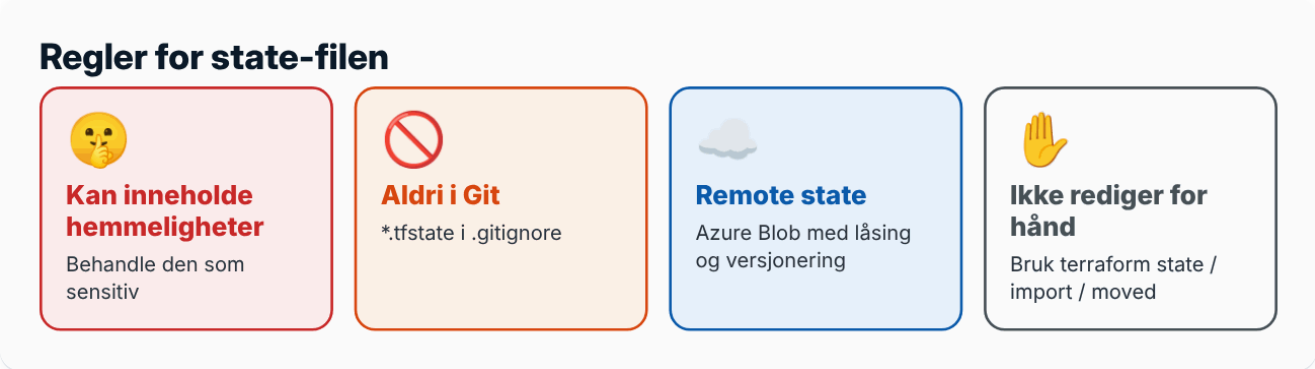
output "vnet_id" { value = azurerm_virtual_network.main.id }
```

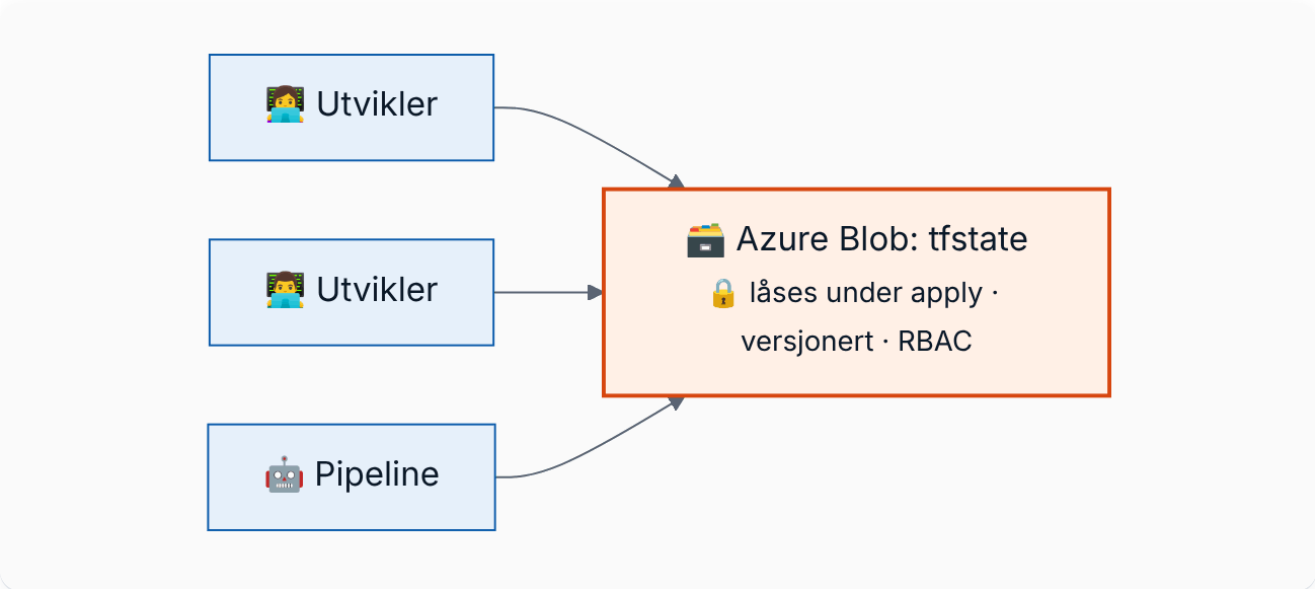


State



terraform plan sammenligner tre ting

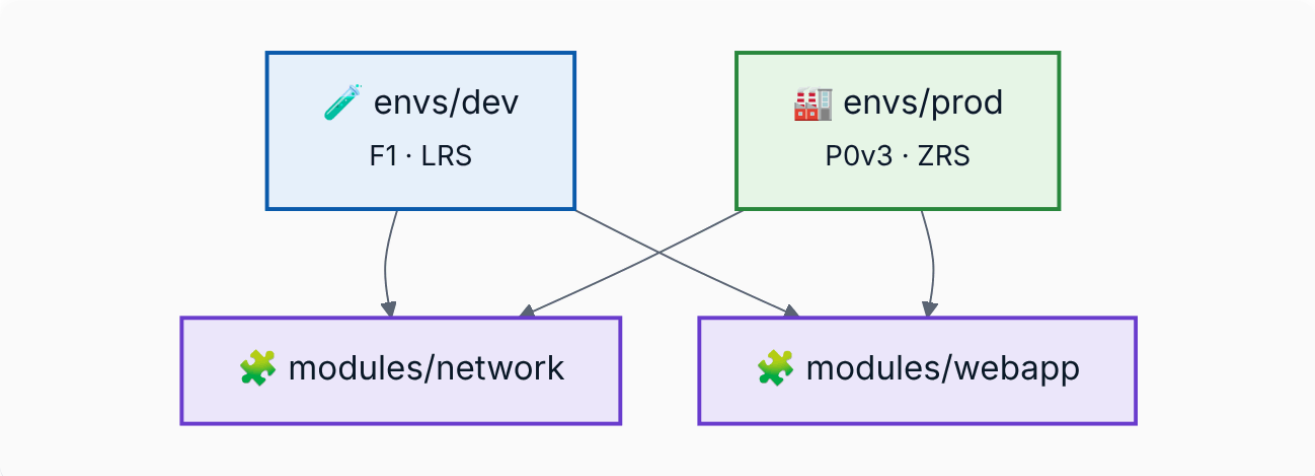




Remote state – hele teamet og pipelinen deler én låst state

> ⚠️ **Les planen!** `-/+` betyr **slett og lag på nytt** – på en database kan det bety datatap.

Moduler



Samme moduler – forskjellige verdier per miljø

Bicep vs. Terraform

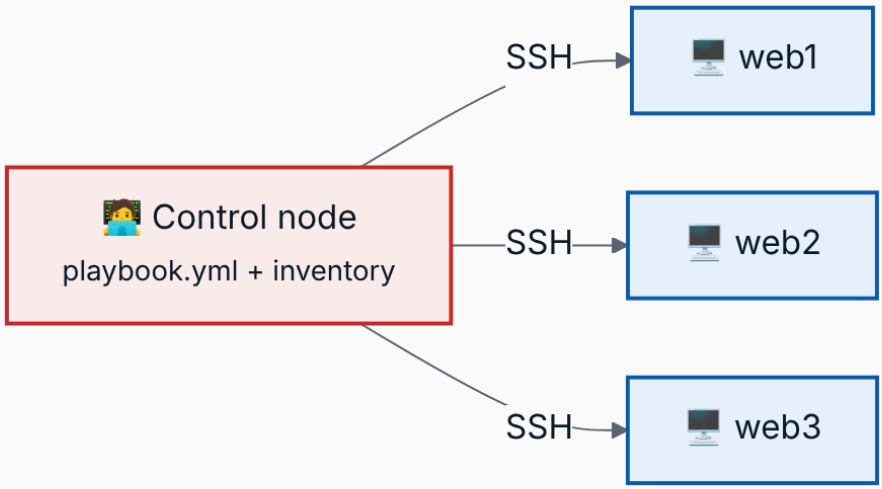
💪 Bicep

- ✔️ Bare Azure
- ✔️ Ingen state-fil
- ✔️ Nye Azure-funksjoner med en gang
- ✔️ what-if

🟪 Terraform

- ✔️ Mange plattformer
- ✔️ State-fil som må beskyttes
- ✔️ Svært vanlig i jobbannonser
- ✔️ plan

Ansible

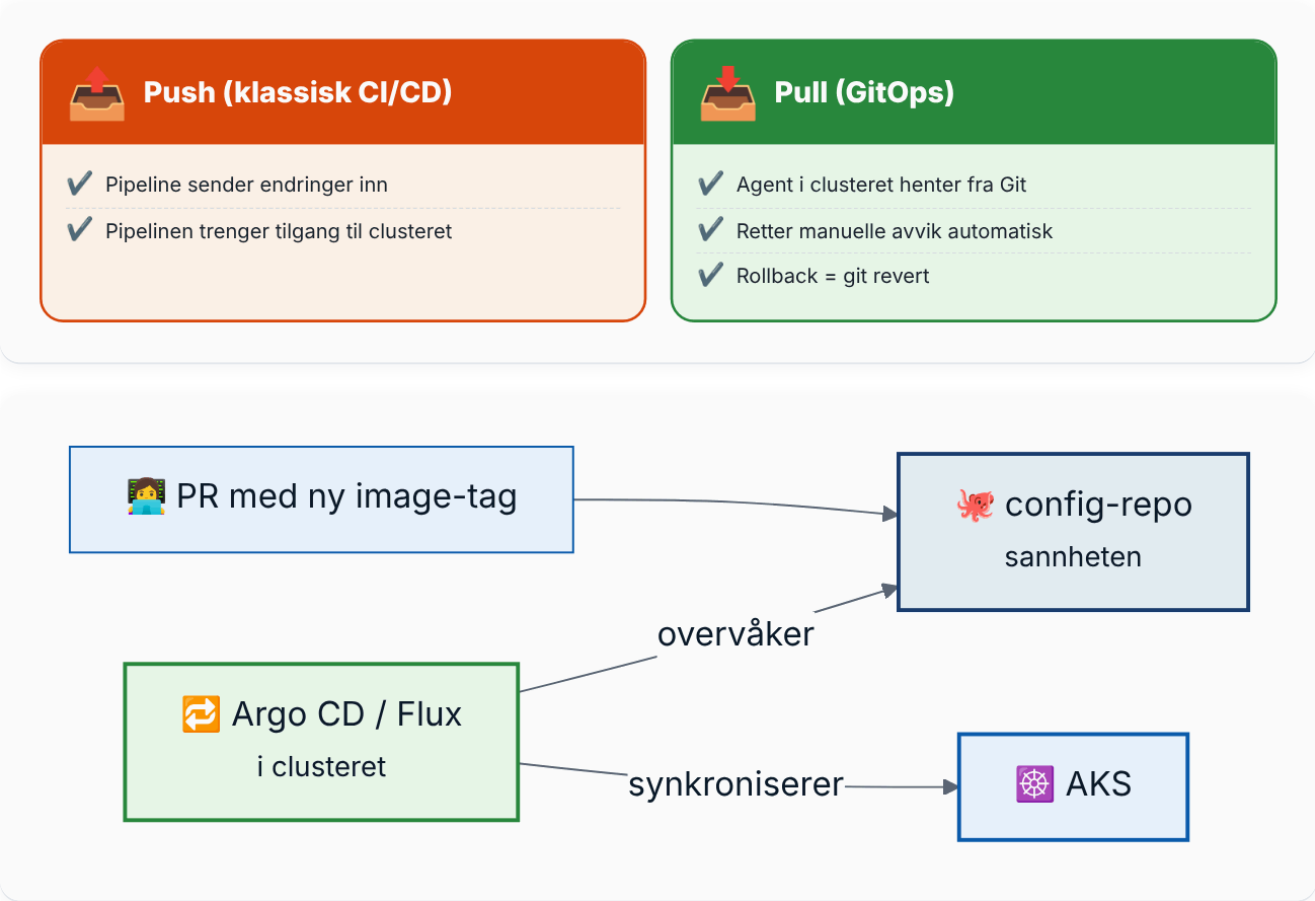


Ansible er agentløst – kobler til over SSH og gjør serverne like

```yaml
- name: Konfigurer webservere
  hosts: web
  become: true
  tasks:
    - name: Installer nginx
      ansible.builtin.apt: { name: nginx, state: present, update_cache: true }
    - name: Sørg for at nginx kjører
      ansible.builtin.service: { name: nginx, state: started, enabled: true }
```

💡 Kjør playbooken to ganger: andre gang står det `changed=0` – det er idempotens.

GitOps



GitOps med Argo CD eller Flux

Test deg selv

1. Hva er idempotens? → *Samme kode flere ganger = samme resultat.*
2. Hva gjør `terraform plan`? → *Viser hva som vil skje, uten å gjøre det.*
3. Hvorfor ikke state i Git? → *Hemmeligheter og ingen låsing.*
4. Hva betyr `-/+`? → *Slett og lag på nytt.*
5. Hvorfor trenger ikke Bicep state? → *Azure er sannheten.*
6. Push vs. pull i GitOps? → *Pipeline sender inn vs. agent henter selv.*

12 · CI/CD og automatisering

Kapittelet på én side

Pipeline
Samlebåndet fra commit til drift

Testing
Testpyramiden

GitHub Actions
Workflows, jobs, steps

Azure Pipelines og Jenkins
Samme prinsipper

Artefakter
Bygg én gang, deploy mange ganger

OIDC
Til Azure uten hemmeligheter

Rollback
Planlagt og testet

Blue-green og canary
Trygg utrulling

Pipelinen

Samlebånd i bilfabrikk
Hver stasjon gjør én ting. Feiler kvalitetskontrollen, stopper båndet før bilen når kunden.

=

CI/CD-pipeline
Hvert steg sjekker noe. Feiler en test, stopper pipelinen før koden når produksjon.

Fra commit til produksjon



En god pipeline er ...

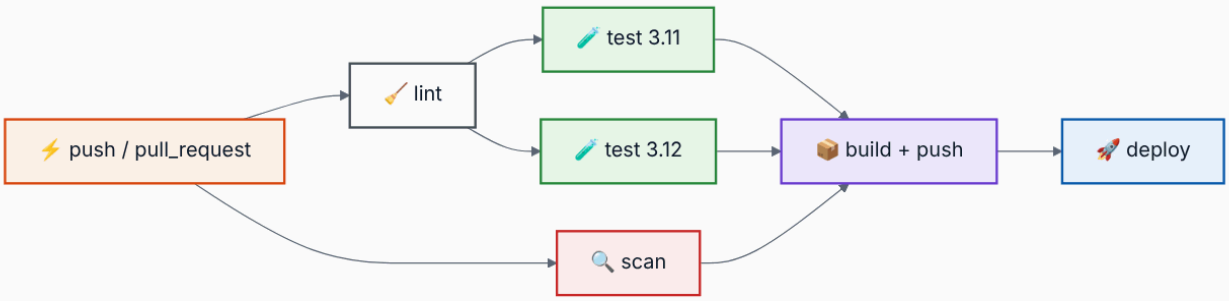
- ⚡ **Rask** — Svar på minutter
- 🎯 **Pålitelig** — Ingen tilfeldige feil
- 📦 **Bygg én gang** — Samme artefakt i alle miljøer
- 📄 **Kode** — YAML i repoet, gjennomgått
- 🔐 **Sikker** — Minste privilegium, ingen hemmeligheter i klartekst
- 🔍 **Sporbar** — Hvilken commit kjører i prod?

Testpyramiden

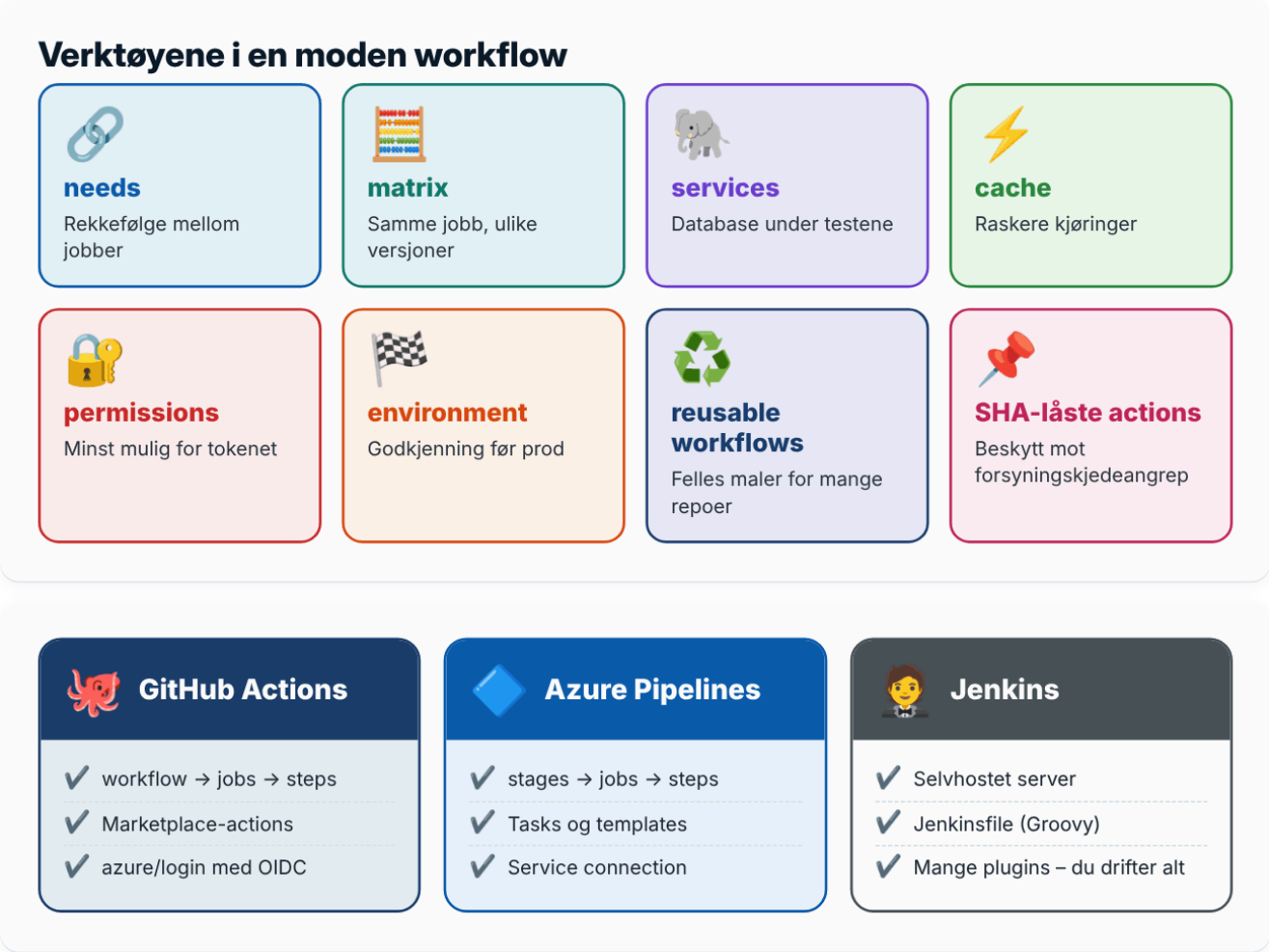


I tillegg: linting først, sikkerhetsskanning i CI, smoke test etter hver deploy

GitHub Actions



Jobber kjører parallelt – needs gir rekkefølge

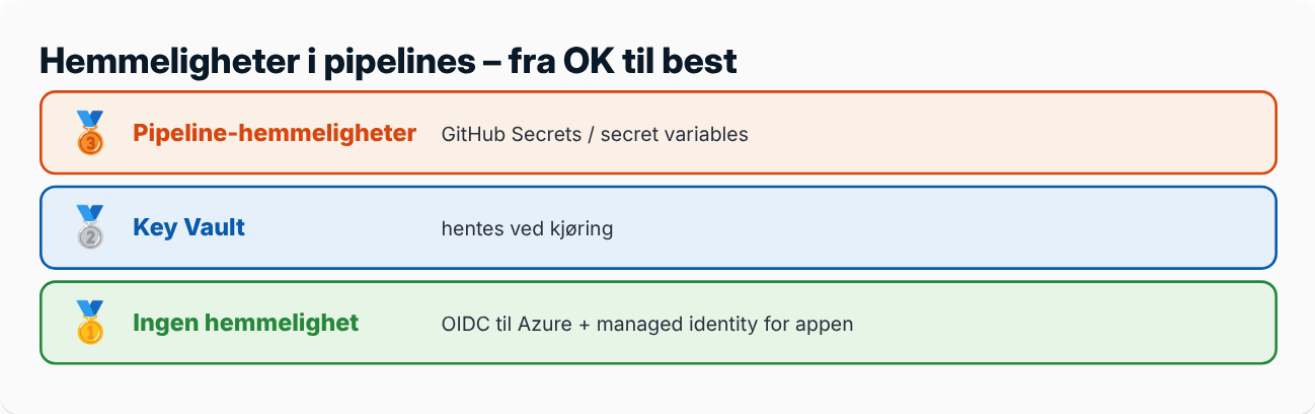


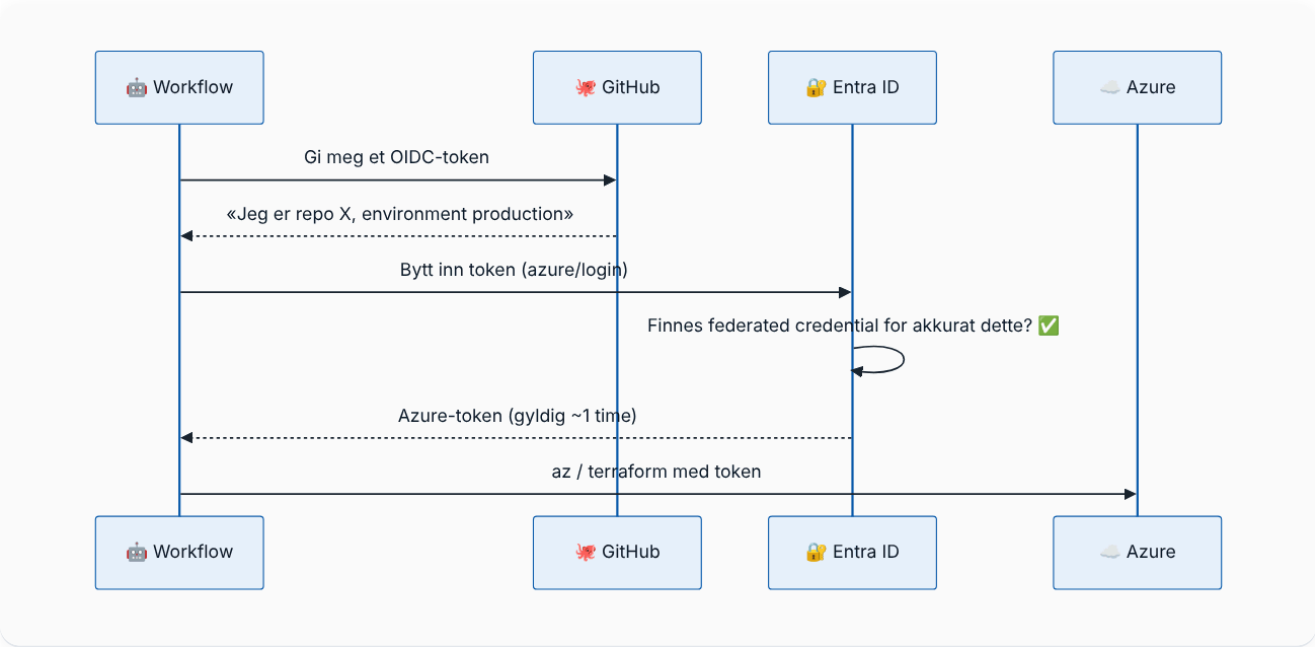
Artefakter



Samme image hele veien – bare konfigurasjonen endres

Hemmeligheter og OIDC





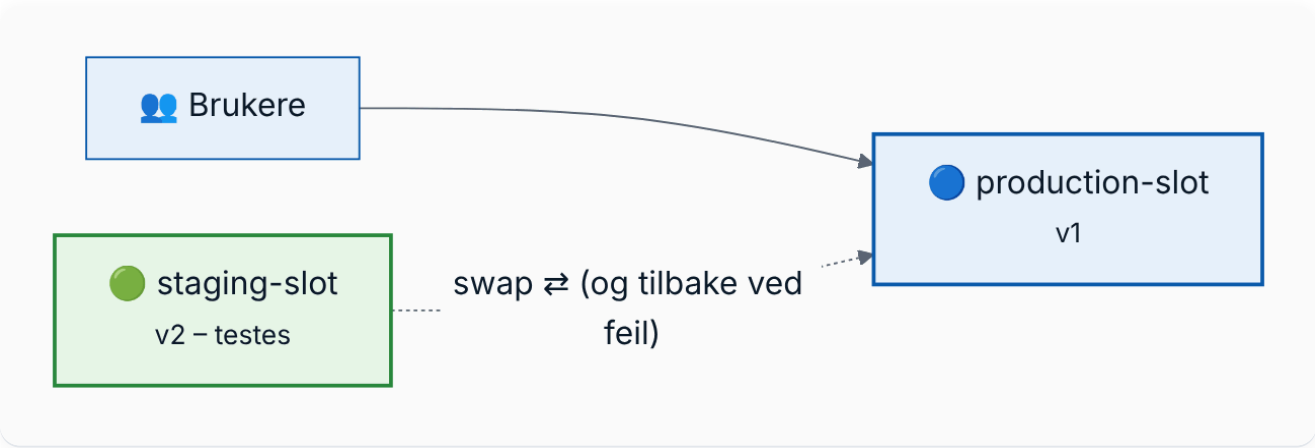
OIDC – GitHub beviser hvem den er, Azure gir et kortlevd token

```yaml
permissions:
  id-token: write          # nødvendig for OIDC
  contents: read
jobs:
  deploy:
    environment: production
    runs-on: ubuntu-latest
    steps:
      - uses: azure/login@v2
        with:
          client-id: ${{ vars.AZURE_CLIENT_ID }}
          tenant-id: ${{ vars.AZURE_TENANT_ID }}
          subscription-id: ${{ vars.AZURE_SUBSCRIPTION_ID }}
```

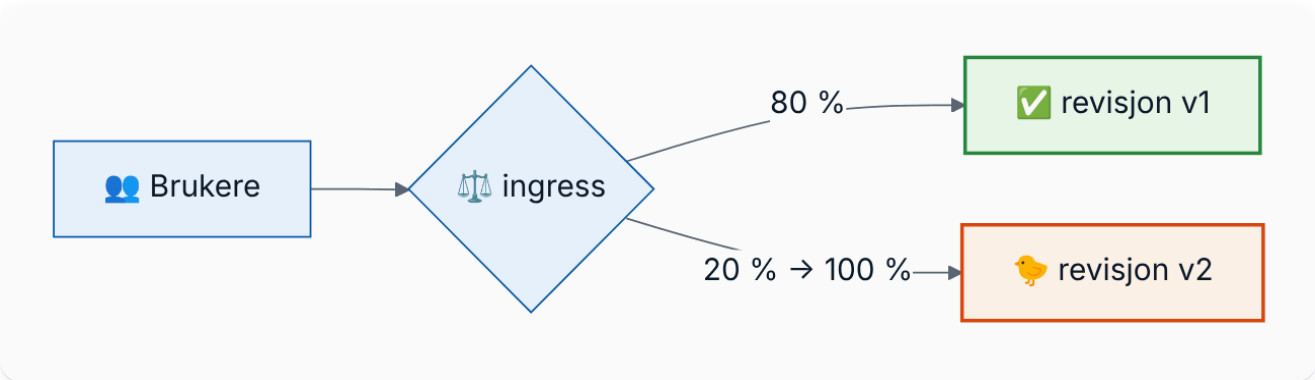
⚠️ `AADSTS70021` = subject i federated credential matcher ikke (f.eks. `repo:eier/repo:environment:production`).

Rollback, blue-green og canary



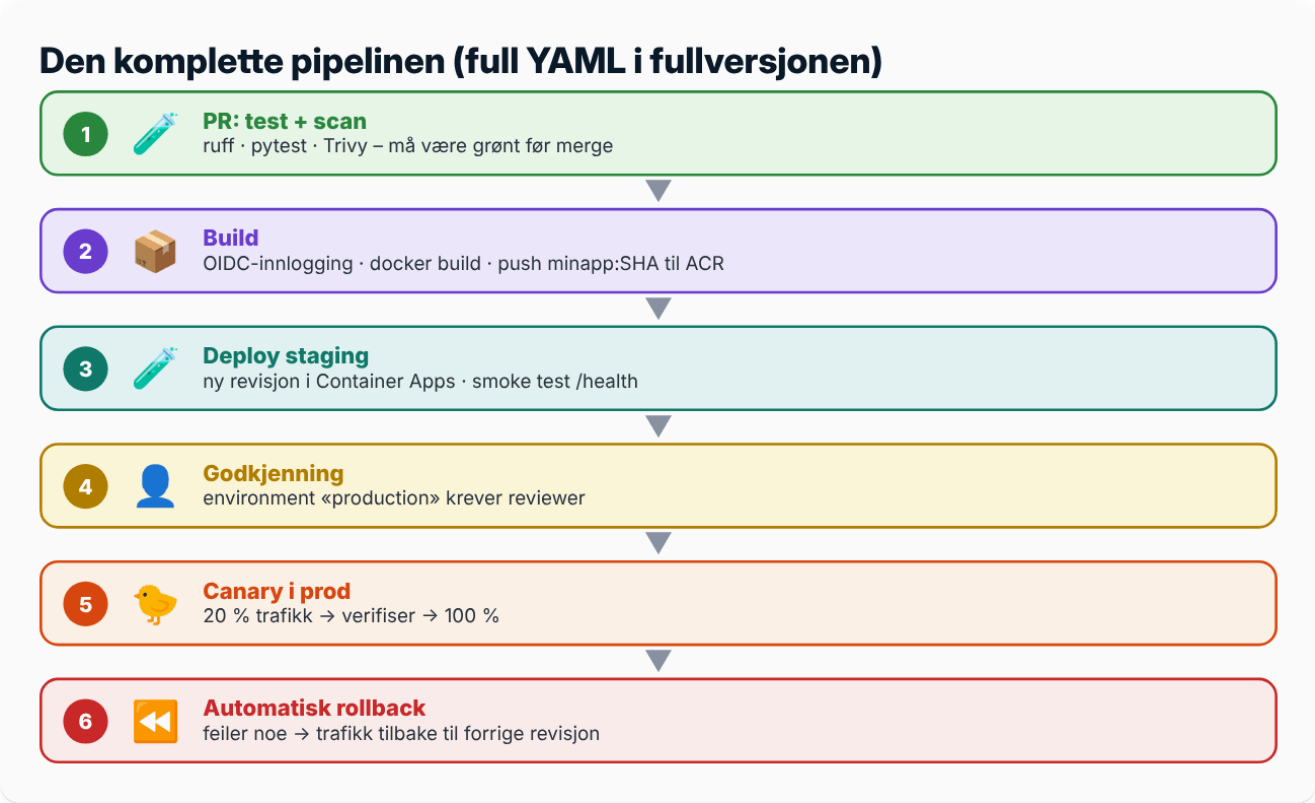


Blue-green med App Service – deploy til staging, test, swap



Canary med Container Apps-revisjoner

Hele pipelinen – kode til produksjon



Test deg selv

1. «Bygg én gang»? → *Samme artefakt i alle miljøer.*
2. `needs` og `matrix`? → *Rekkefølge og varianter.*
3. Hvorfor SHA-låste actions? → *Tagger kan flyttes.*
4. Hvilken permission krever OIDC? → `id-token: write`.
5. Blue-green vs. canary? → *Alt på én gang vs. gradvis.*
6. Hva er en smoke test? → *Rask sjekk at tjenesten lever etter deploy.*

13 · Sikkerhet og monitoring

Kapittelet på én side



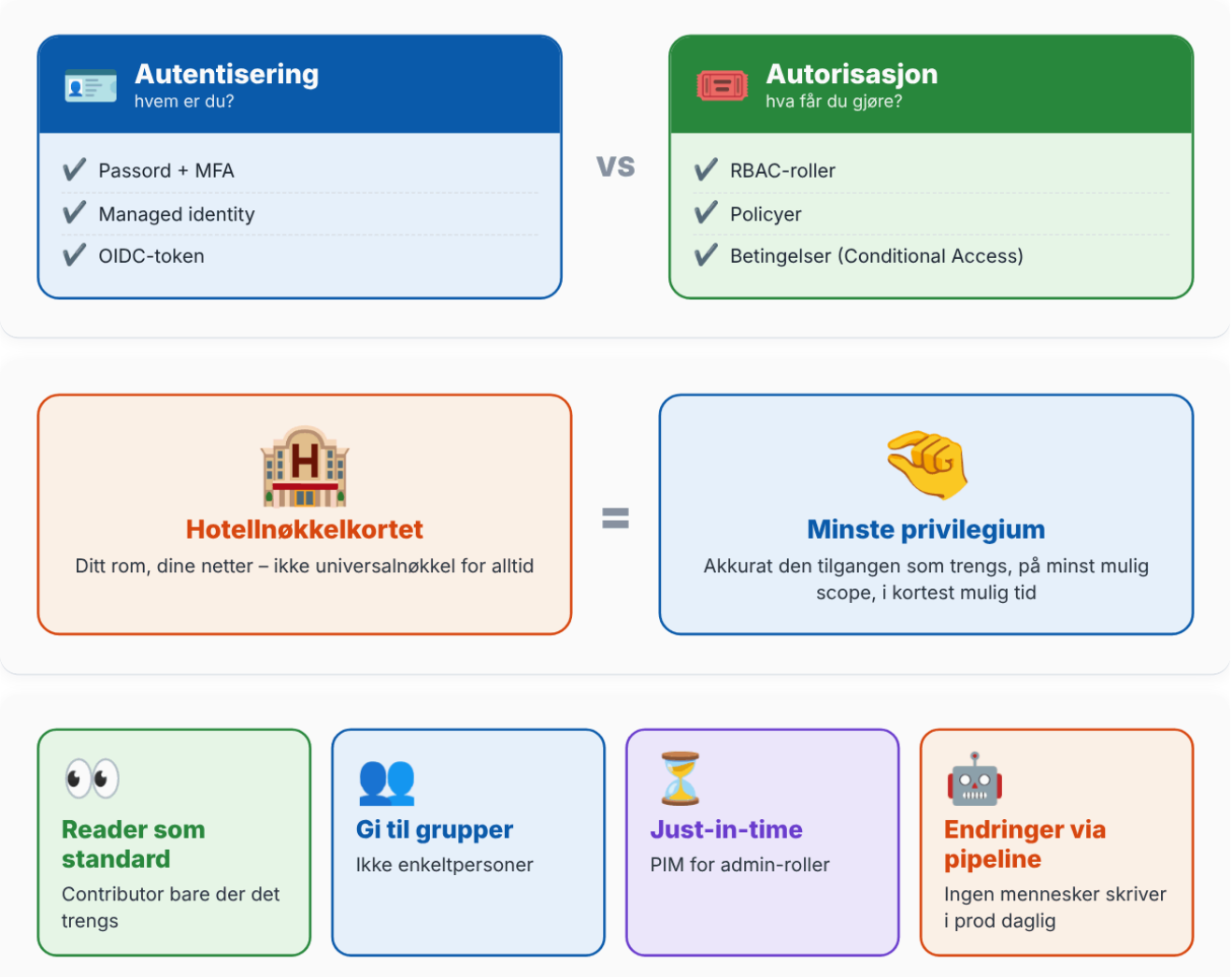
🏢 **Viktig innsikt:** De fleste alvorlige hendelser i skyen skyldes grunnleggende feil – lekkede nøkler, for brede tilganger, åpen lagring, manglende MFA. God grunnhygiene stopper det meste.

Del A – Sikkerhet

CIA-triaden

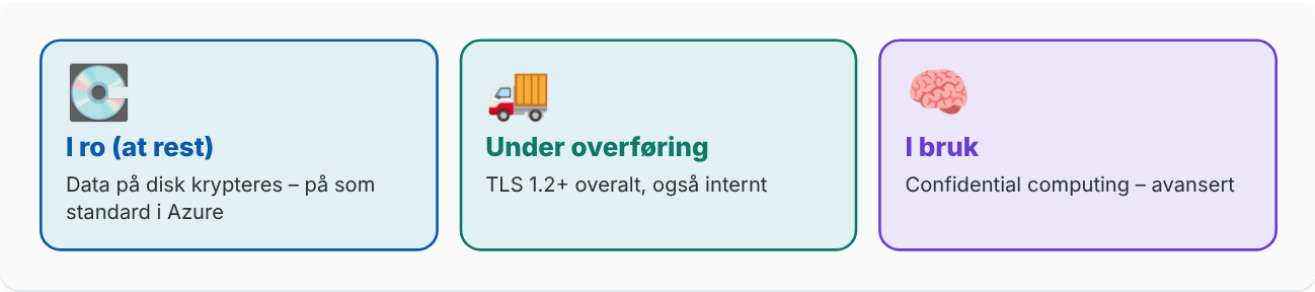


IAM og minste privilegium



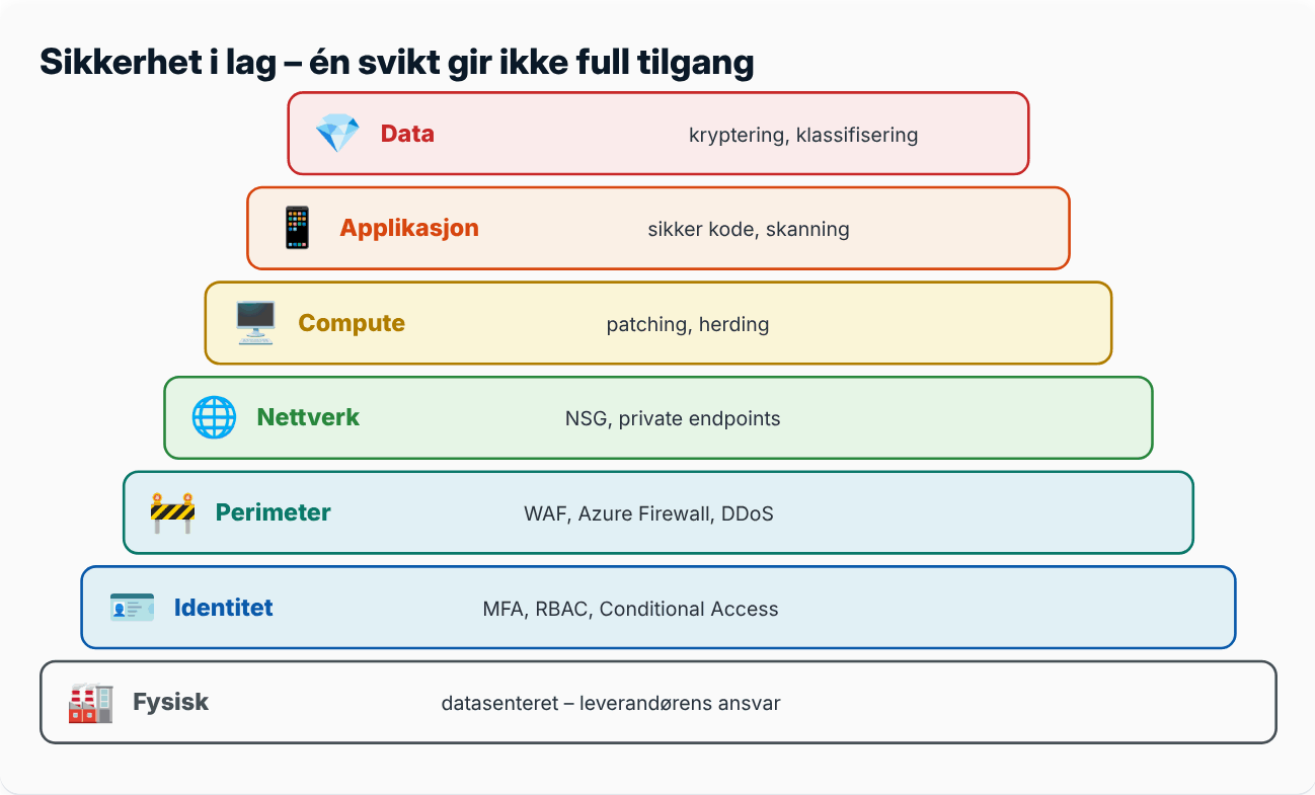
Hemmeligheter og kryptering

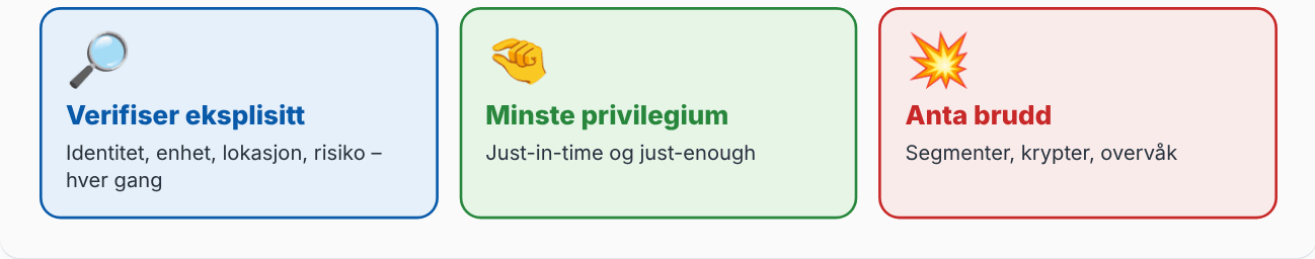


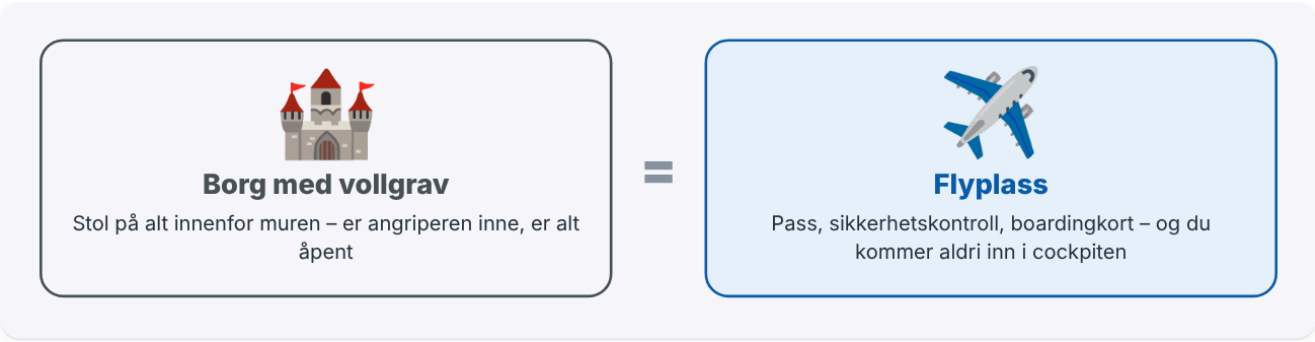


⚠️ **Base64 er ikke kryptering.** Alle kan dekode det.

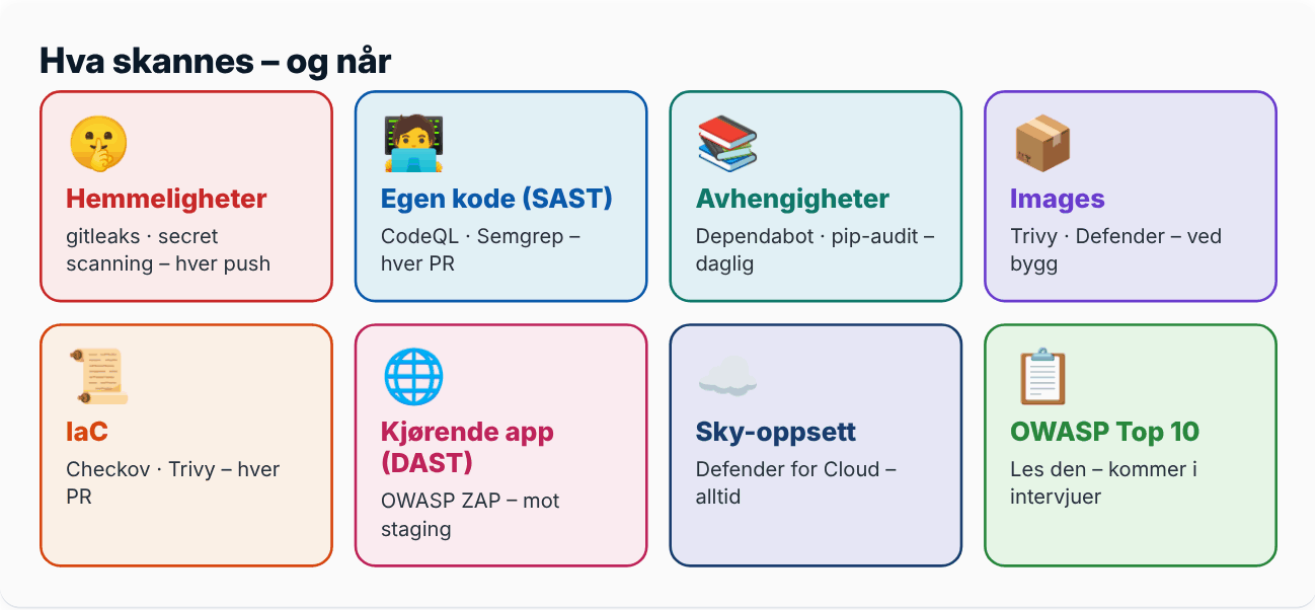
Defense in depth og Zero Trust

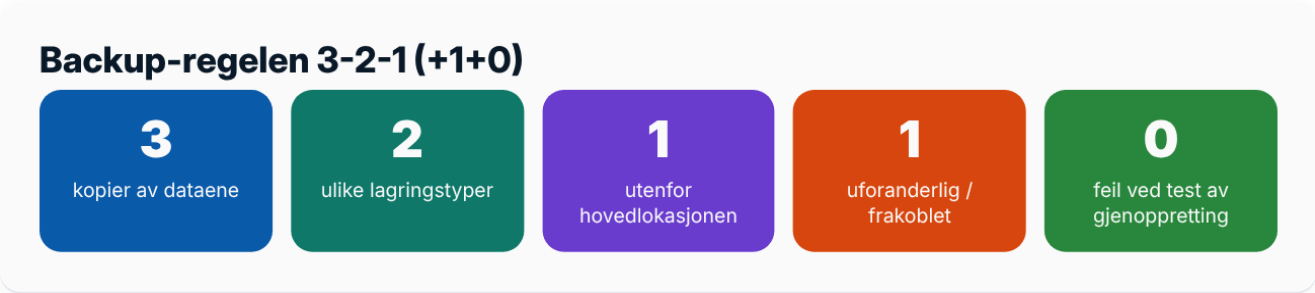




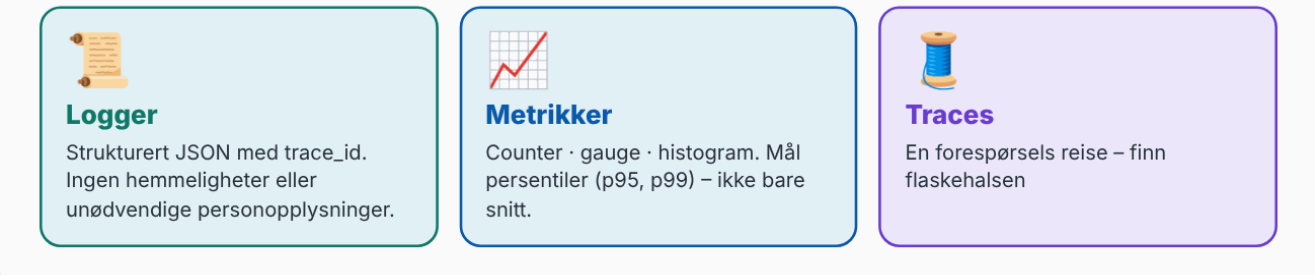


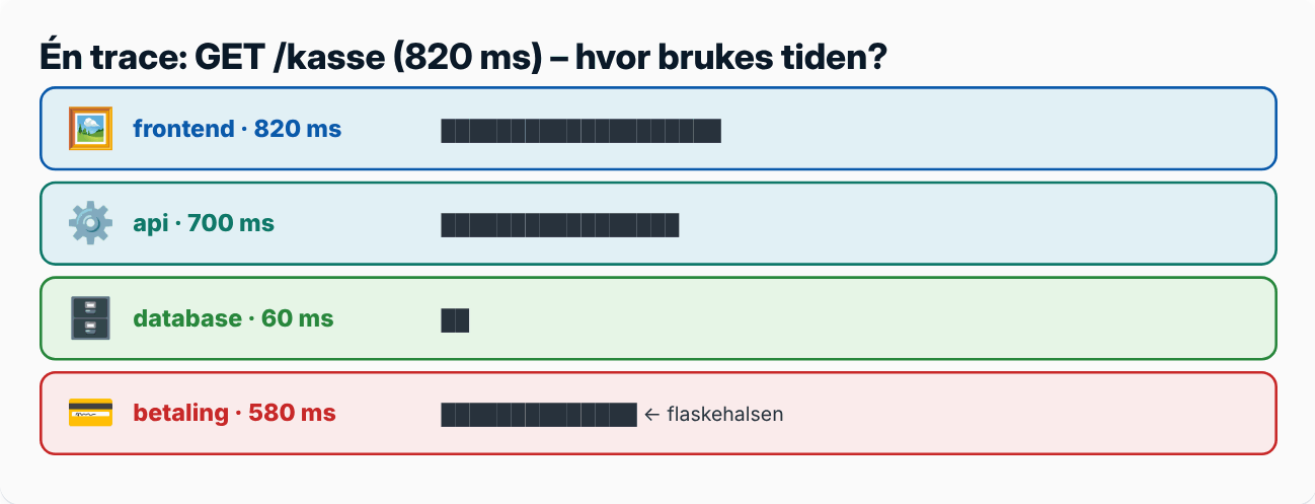
Skanning og backup



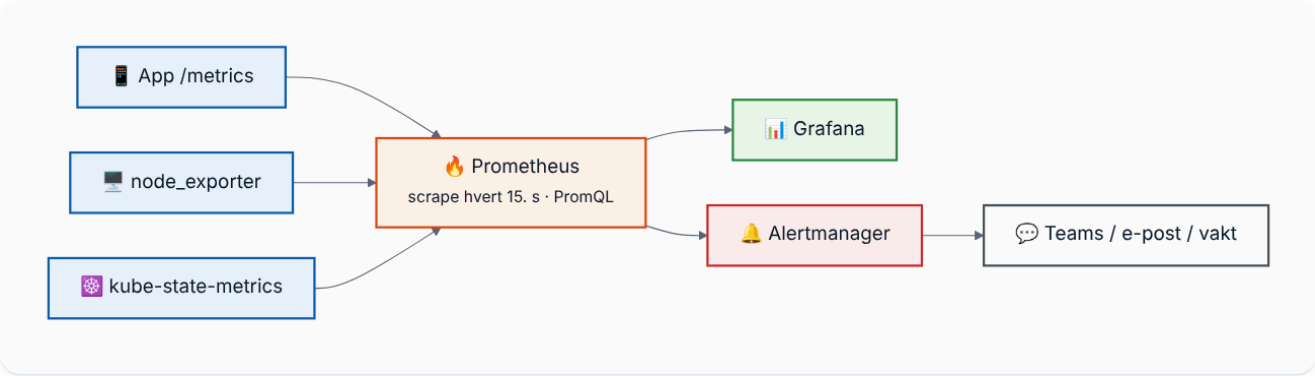


Del B – Monitoring og observability





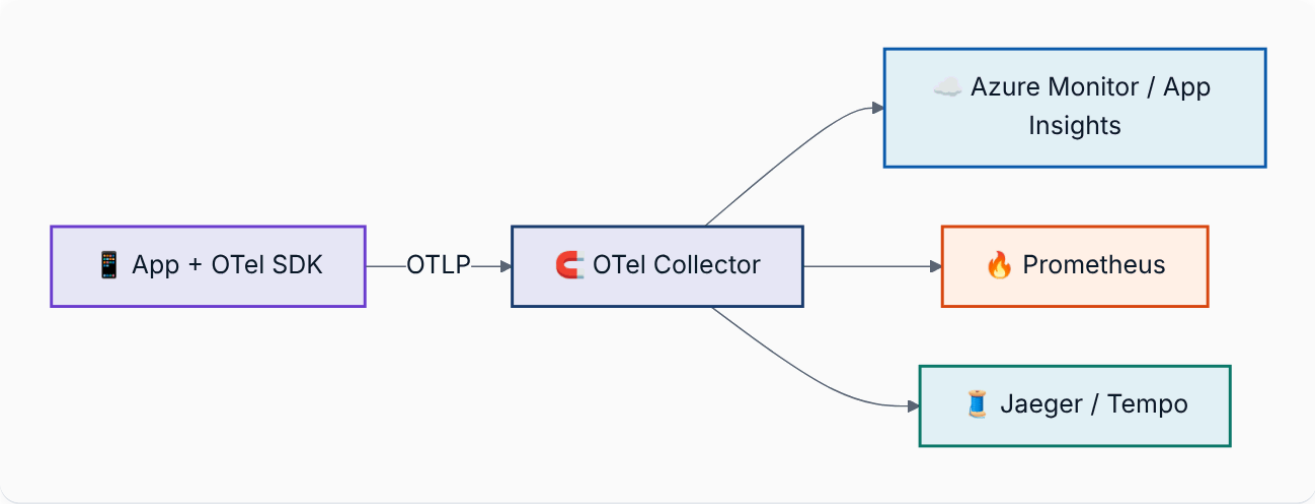
Prometheus og Grafana



Prometheus henter (pull) metrikker og varsler – Grafana viser dem

```
sum(rate(http_requests_total{status=~"5.."}[5m])) / sum(rate(http_requests_total[5m]))   # feilrate
```

OpenTelemetry og Azure Monitor



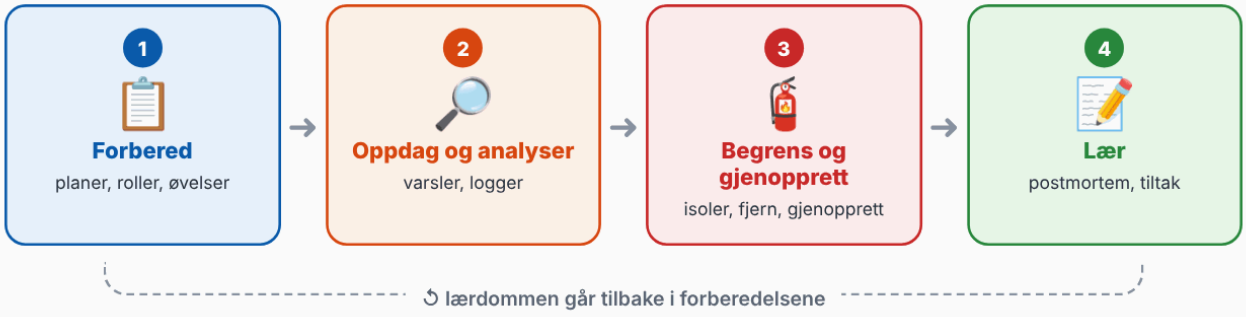
Instrumenter én gang med OpenTelemetry – send hvor du vil

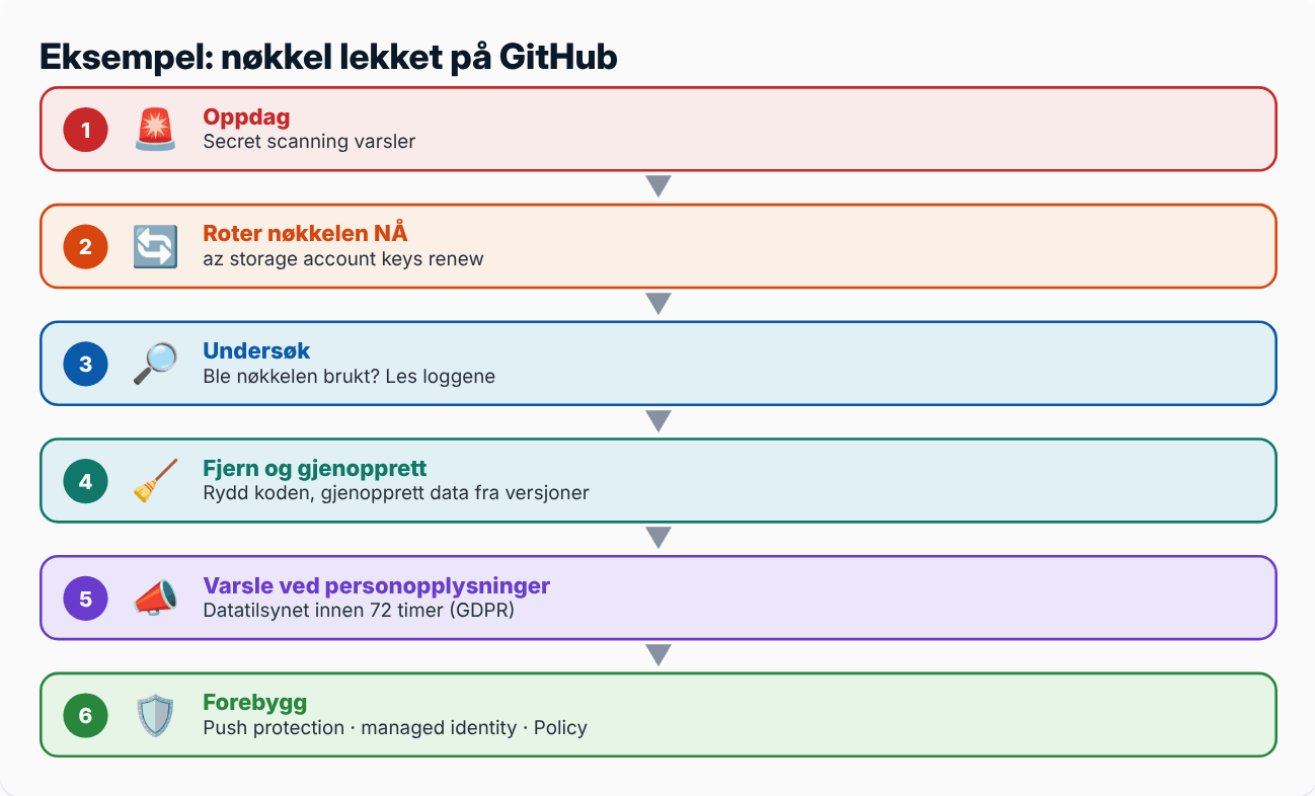
Gode varsler



Del C – Hendelser og feilsøking

Hendelseshåndtering (NIST)







Test deg selv

1. Tre prinsipper i Zero Trust? → *Verifiser eksplisitt, minste privilegium, anta brudd.*
2. Autentisering vs. autorisasjon? → *Hvem du er vs. hva du får.*

3. Hvorfor managed identity? → *Ingen hemmelighet å lekke.*
4. Hva er 3-2-1? → *3 kopier, 2 medier, 1 utenfor.*
5. Push eller pull i Prometheus? → *Pull.*
6. Første steg ved lekket hemmelighet? → *Roter den.*

14 · AI-infrastruktur og MLOps

Kapittelet på én side

AI-infrastruktur
Plattformen for å trene, kjøre og overvåke modeller

MLOps
DevOps for kode + data + modeller

Trening vs. inferens
Lære vs. svare

CPU vs. GPU
Eksperter vs. en hær av regnemaskiner

Servering
Modellen som API

Overvåking
Teknisk helse + modellkvalitet

Azure ML og Foundry
Plattformer i Azure

Karrierevei
Fra DevOps til MLOps

Hva er annerledes med AI?

Vanlig webapp

- ✔ CPU og litt RAM
- ✔ Image på ~100 MB
- ✔ Feil = krasj eller feilkode
- ✔ Moderat kostnad

AI-arbeidslast

- ✔ Ofte GPU med mye GPU-minne
- ✔ Modeller fra MB til hundrevis av GB
- ✔ «Stille feil»: svarer, men dårligere
- ✔ GPU-timer kan være svært dyre

Kafé
En kaffemaskin og en kasse

=

AI-plattform
Industrikjøkken med spesialovner (GPU), råvarelager (data), oppskrifter som forbedres (modeller) og kvalitetskontroll

MLOps

DevOps

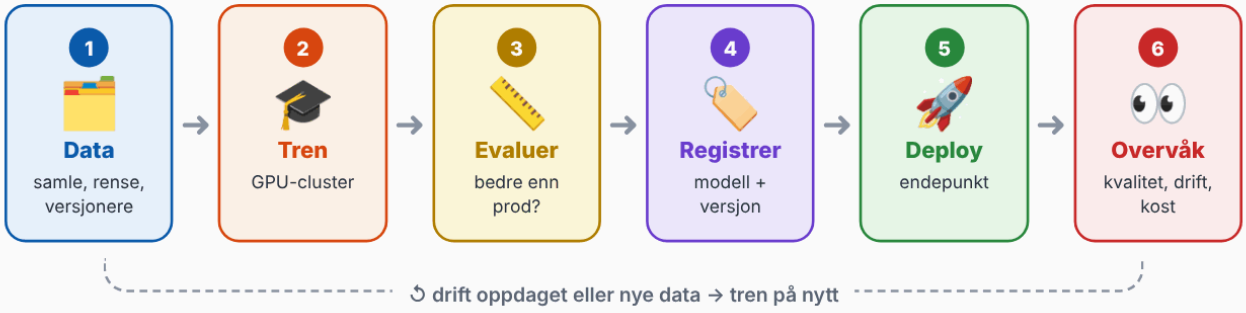
- ✔ Versjonerer kode
- ✔ Tester koden
- ✔ CI/CD
- ✔ Overvåker oppetid og feil

vs

MLOps

- ✅ data, modeller og eksperimenter
- ✅ datavalidering og modellevaluering
- ✅ CT: kontinuerlig trening
- ✅ modellkvalitet og drift

Livssyklusen til en modell



Trening vs. inferens

Trening
modellen lærer

- ✔ Sjelden – timer til uker
- ✔ Mange kraftige GPU-er
- ✔ Batch-jobb
- ✔ Store, sjeldne utgifter

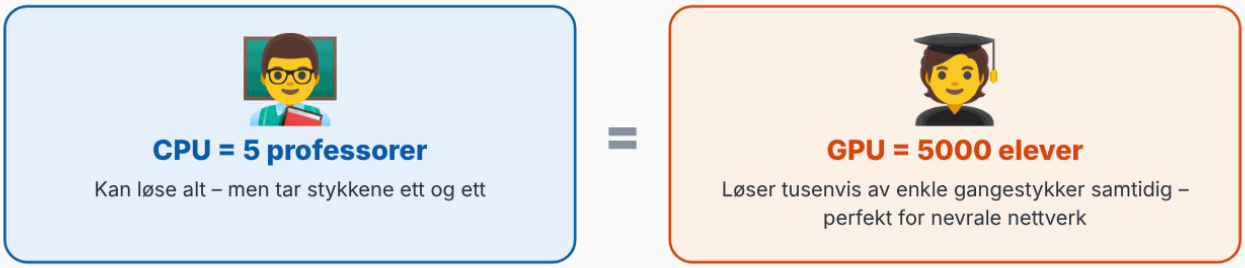
Inferens
modellen svarer

- ✔ Hele tiden – per forespørsel
- ✔ CPU for små, GPU for store
- ✔ Tjeneste (API)
- ✔ Løpende kostnad som vokser med bruk

Tre måter å kjøre AI i skyen

🔌 **Ferdige API-er**	Azure AI-tjenester, modeller i Microsoft Foundry – minst drift, betal per token
🧰 **ML-plattform (PaaS)**	Azure Machine Learning – tren og deploy med plattformens verktøy
🖥️ **Egen infrastruktur**	GPU-VM-er, AKS med GPU-noder, vLLM – mest kontroll, mest drift

CPU vs. GPU



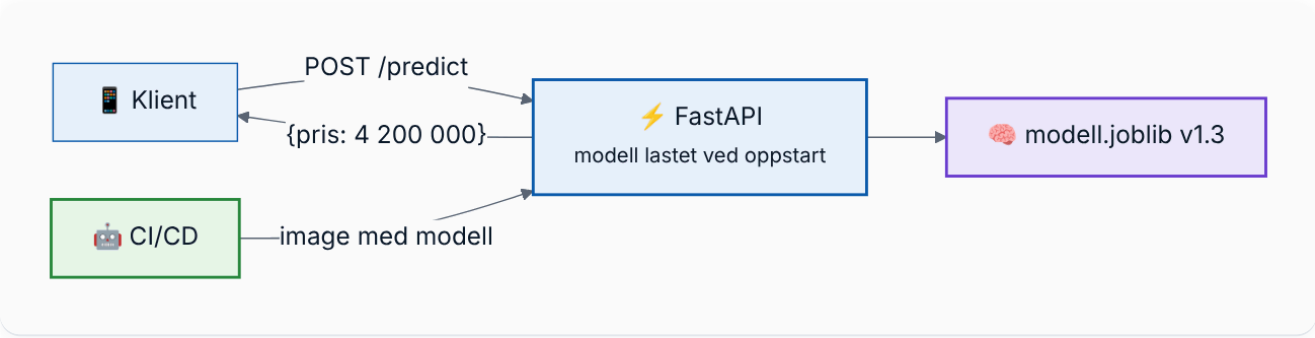
Tommelfingerregel: GPU-minne for modellvektene (16-bit ≈ 2 GB per milliard parametere)

~14 GB	~16 GB	~4 GB	~140 GB
7B-modell, 16-bit	8B-modell, 16-bit	8B-modell, 4-bit kvantisert	70B-modell, 16-bit → flere GPU-er

Pluss ekstra minne til kontekst (KV-cache). Kvantisering sparer minne mot litt lavere kvalitet.

> 💰 **Kostnad:** GPU-VM-er er blant de dyreste ressursene og krever ofte kvoteøkning. Øv på CPU, små modeller og API-er med lave volumer.

Servering av modeller



En modell som API – samme DevOps-teknikker som før

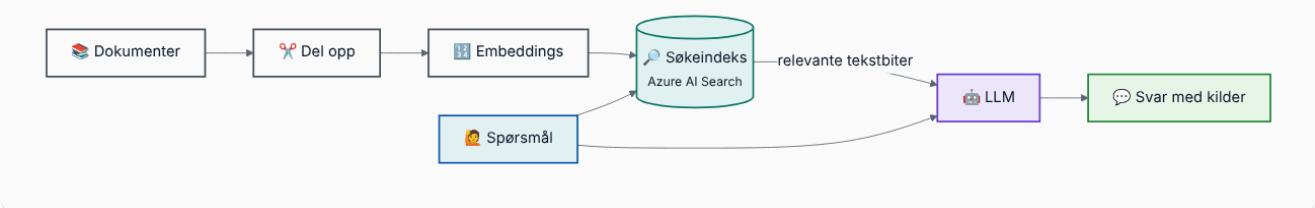
Serveringsmønstre

⚡ Sanntid	🌙 Batch	🌊 Strømming	📷 Edge
API svarer per forespørsel – svindelsjekk på 50 ms	Score millioner av rader i natt	Hendelser fra en kø	På enheten – kamera, mobil

Verktøy for servering

⚡ FastAPI	🚄 vLLM	🔺 Triton	☸ KServe
Enkle modeller som API	Rask servering av LLM-er	Mange modelltyper på GPU	Modellservering på Kubernetes

RAG – svar fra dine egne dokumenter



Retrieval-Augmented Generation

> 🔐 **Viktig:** Brukeren skal bare få svar fra dokumenter hen har tilgang til.

Pipelines, versjonering og overvåking

Alt som må versjoneres

Kode
Git

Data
Datasett-versjoner

Modell
Modellregister (MLflow, Azure ML)

Miljø
Container-image

Eksperimenter
Parametere og metrikker

LLM-apper
Prompter og evalueringssett

Hva overvåkes?

Teknisk
latency, feil, GPU-bruk

Data drift
input endrer seg fra treningsdataene

Concept drift
verden endrer seg – samme input, nytt riktig svar

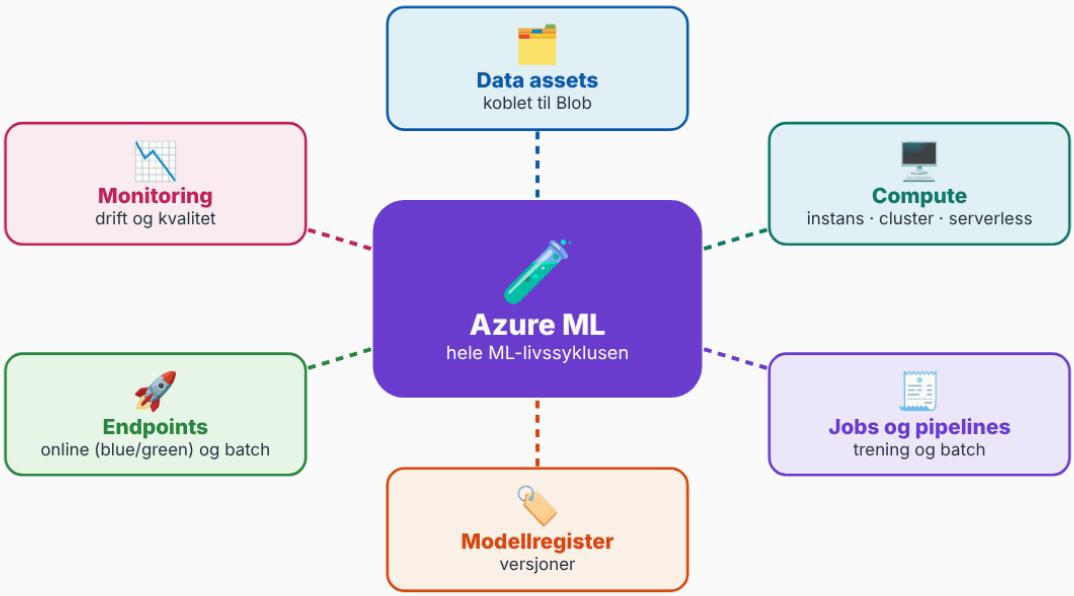
Kvalitet
nøyaktighet mot fasit

LLM
relevans, groundedness, skadelig innhold

Kostnad
per forespørsel og per token

Azure ML og Microsoft Foundry

Azure Machine Learning-workspace



Ferdige AI-tjenester i Azure

Foundry Models / Azure OpenAI
Språkmodeller via API

AI Search
Søk og vektorsøk – RAG

Speech
Tale ↔ tekst

Document Intelligence
Data fra fakturaer og skjemaer

Vision
Bildeanalyse og OCR

Translator
Oversettelse

Content Safety
Skadelig innhold og prompt-angrep

API Management
AI-gateway: token-grenser per team

AI-sikkerhet og kostnader

Nye trusler med AI

Prompt injection
«Ignorer instruksjonene og ...» – behandle all input som upålitelig

Datalekkasje
Sensitiv info i svar – tilgangsstyring i RAG

Overprivilegerte agenter
Minste privilegium og godkjenning

Usikker output
Svar brukt i SQL/shell/HTML – valider

Forsyningskjede
Modeller fra ukjente kilder

Denial of wallet
Dyre forespørsler – rate limiting og budsjett

Spar penger på AI

Minste modell som holder

Korte prompter
Begrens output

Caching
Også semantisk

Batch
Ikke-hastende jobber

Skaler til null
GPU-noder

Spot
Avbrytbar trening

Tag og mål
Kostnad per team

Budsjetter
Særlig for eksperimenter

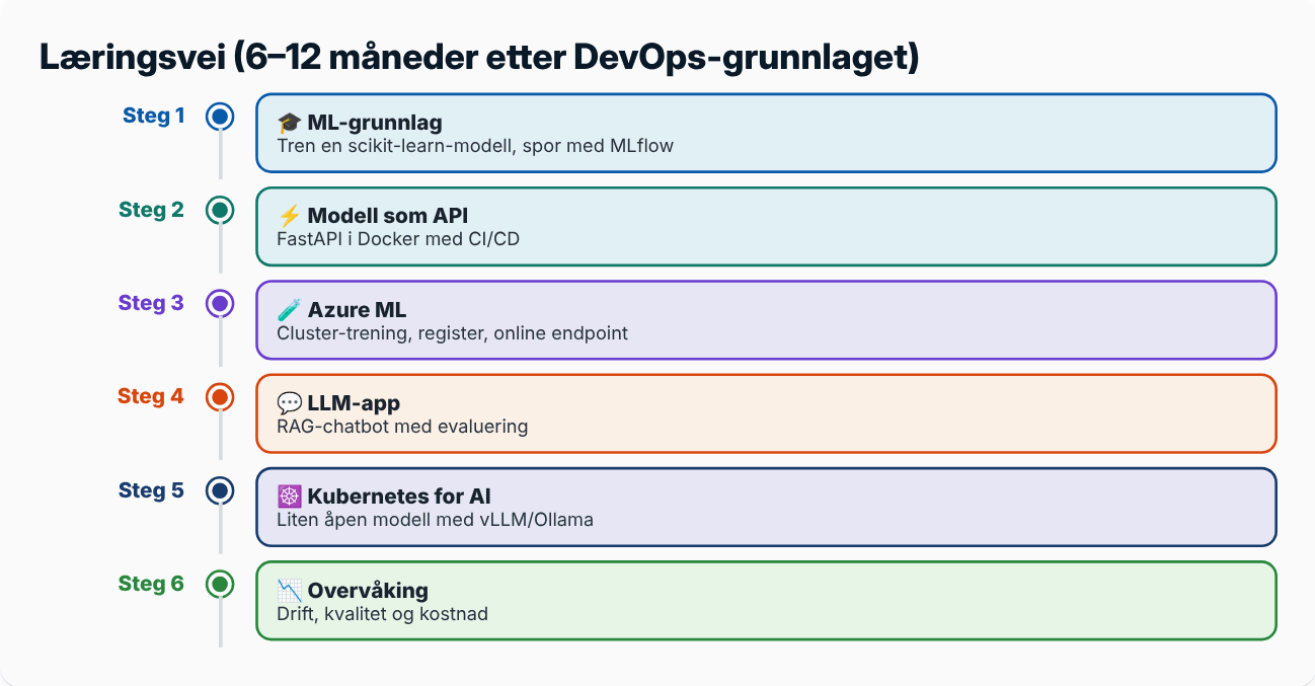
Fra DevOps til MLOps

Det du allerede har

- ✔ Linux, nettverk, sky
- ✔ Docker og Kubernetes
- ✔ Terraform og CI/CD
- ✔ Overvåking, sikkerhet, kostnad

Det du lærer i tillegg

- ➕ ML-grunnlag og evaluering
- ➕ pandas, scikit-learn, litt PyTorch
- ➕ MLflow og Azure ML
- ➕ GPU-er, LLM-er, RAG, vLLM



> 💡 **Husk:** Bedrifter trenger langt flere som kan drifte AI **pålitelig, sikkert og kostnadseffektivt** enn som kan finne opp nye modeller.

Test deg selv

1. DevOps vs. MLOps? → *MLOps versjonerer og overvåker også data og modeller.*
2. Trening vs. inferens? → *Lære vs. svare.*
3. Hvorfor GPU? → *Tusenvis av parallelle regneoperasjoner.*
4. Hva er data drift? → *Produksjonsdata skiller seg fra treningsdataene.*
5. Hva er RAG? → *Hent relevante dokumentbiter og gi dem til LLM-en.*
6. Hva er prompt injection? → *Input som prøver å overstyre instruksjonene.*

15 · Tolv prosjekter

Hvert prosjekt vises som et **prosjektkort** og en **arkitekturtegning**. Full steg-for-steg-oppskrift med all kode finnes i fullversjonen, kapittel 15.

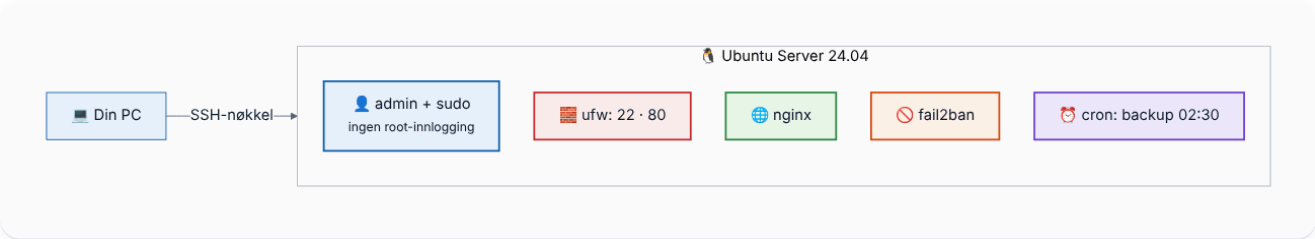
Prosjektkartet – fra enkel til avansert

- 🐧 **1 · Linux-server** ★☆☆☆☆ · 4–6 t · gratis
- 🐍 **2 · Kostnadsjeger** ★★☆☆☆ · 4–6 t · gratis
- 🖥️ **3 · Azure-VM med kode** ★★☆☆☆ · 3–4 t · øre
- 🐙 **4 · GitHub-portefølje** ★★☆☆☆ · 3–4 t · gratis
- 🐳 **5 · Docker-app** ★★★☆☆ · 4–6 t · gratis
- 🌐 **6 · Nettside på Azure** ★★★☆☆ · 3–5 t · gratis
- 🚀 **7 · CI/CD til App Service** ★★★☆☆ · 6–8 t · lav
- 🟪 **8 · Terraform + pipeline** ★★★★☆ · 8–12 t · lav
- ☸️ **9 · Kubernetes + Helm** ★★★★☆ · 8–12 t · gratis lokalt
- 🏆 **10 · Komplett plattform** ★★★★★ · 25–40 t · lav–moderat
- 🤖 **11 · MLOps** ★★★★☆ · 10–15 t · lav
- 🚨 **12 · Observability + game day** ★★★★☆ · 8–10 t · gratis

Regler for alle sky-prosjekter

- 📁 **Én ressursgruppe** per prosjekt
- 🏷️ **Tags** prosjekt · eier · utløper
- 🔔 **Budsjettvarsel** før du starter
- 🧹 **Rydd samme dag** az group delete
- 📝 **README underveis** ikke etterpå

1 · Linux-server



Herdet Ubuntu-server

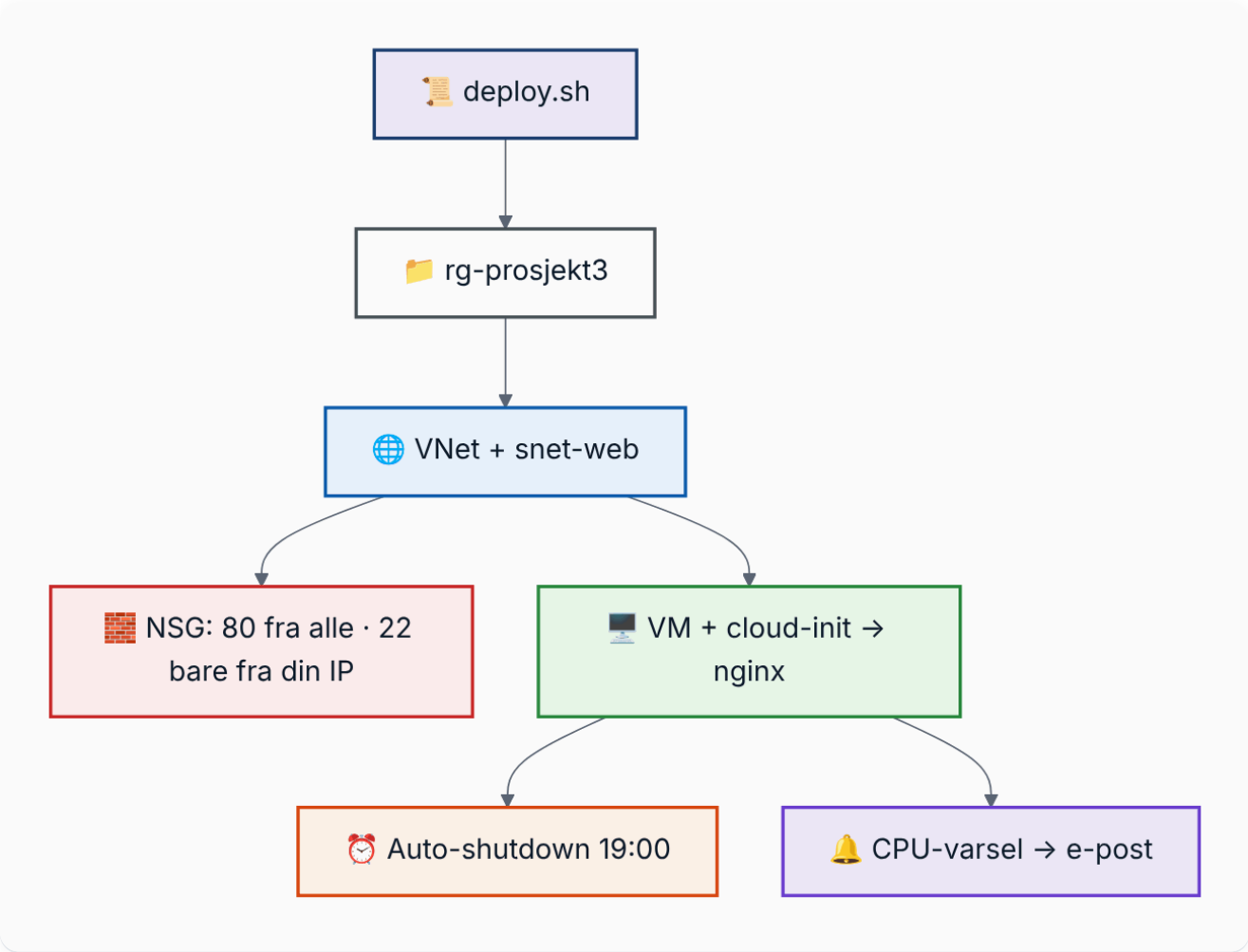
Du lærer: SSH-herding, brukere, brannmur, nginx, automatiske oppdateringer, backup med cron. **CV:** «Installerte og herdet Ubuntu Server med nøkkelbasert SSH, ufw, fail2ban og automatisert backup.»

2 · Kostnadsjeger i Python



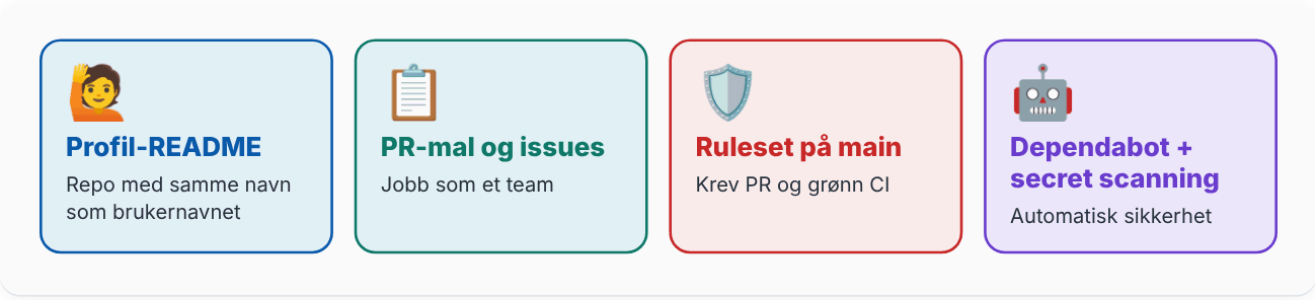
Du lærer: Azure SDK, argparse, logging, tester med pytest. **CV:** «Python-verktøy som finner ressurser uten eier- og utløpstagger for å redusere skykostnader.»

3 · Azure-VM med kode

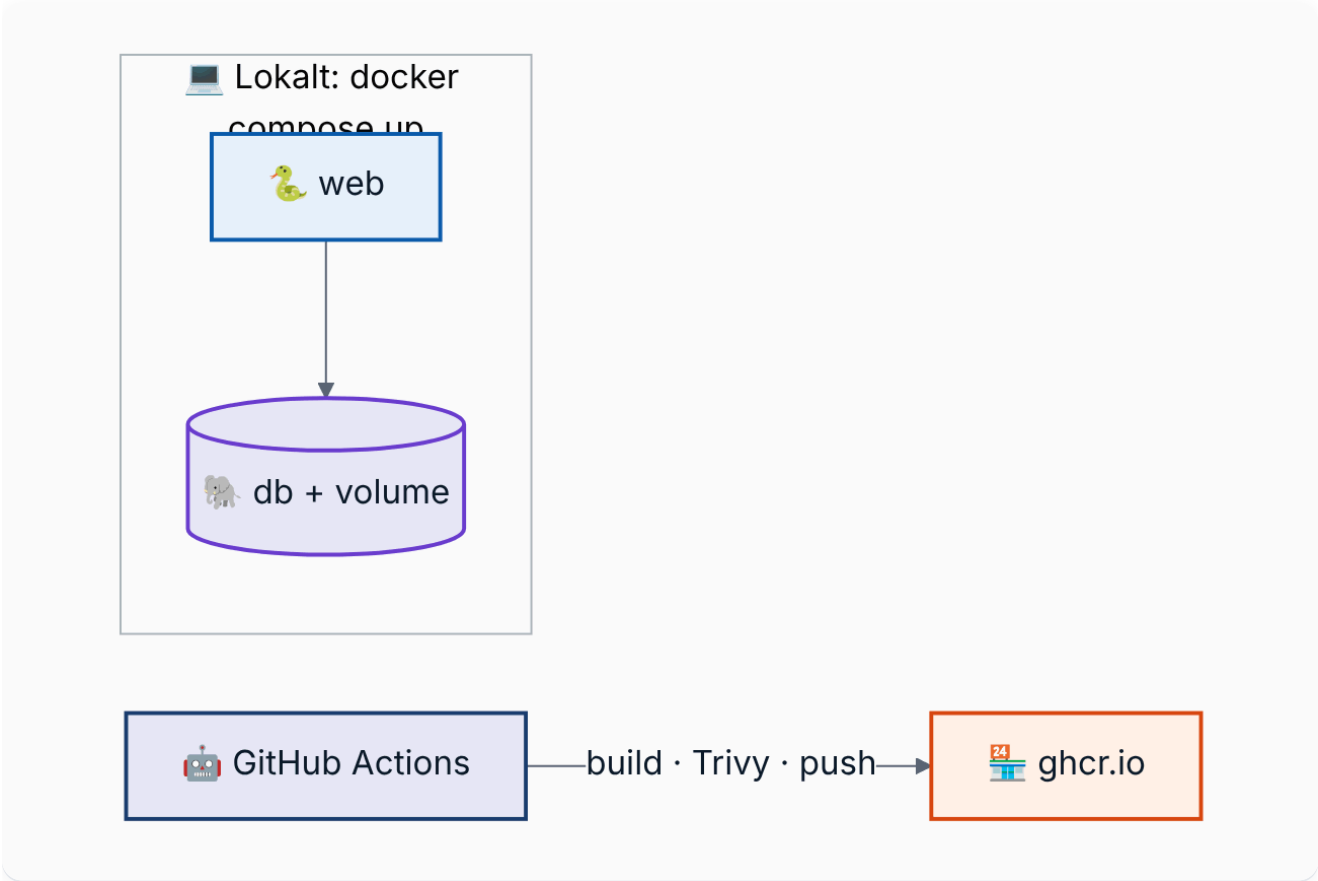


Alt laget med ett skript – og slettet med ett

4 · GitHub-portefølje

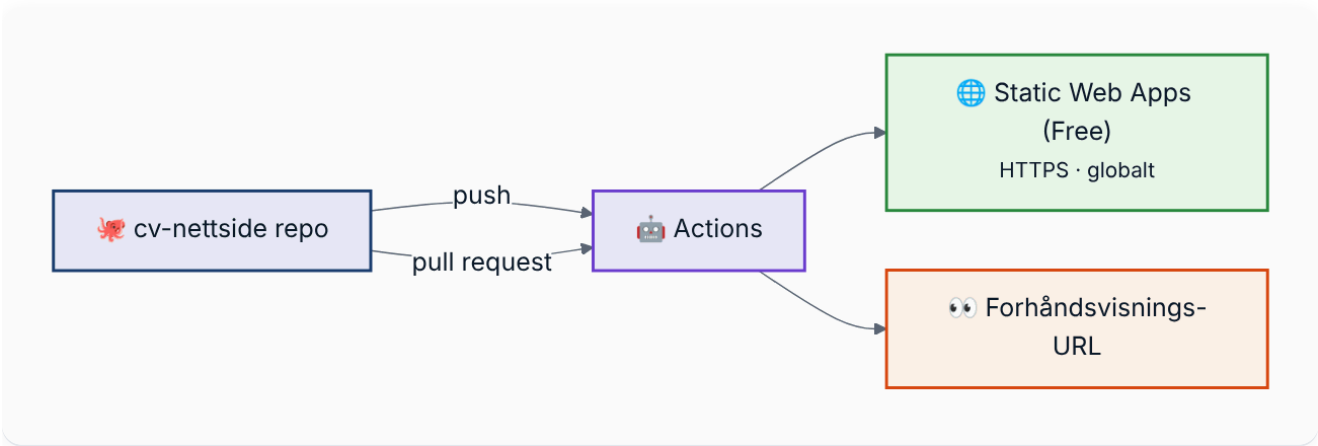


5 · Docker-app



Flask + PostgreSQL – bygget, skannet og publisert automatisk

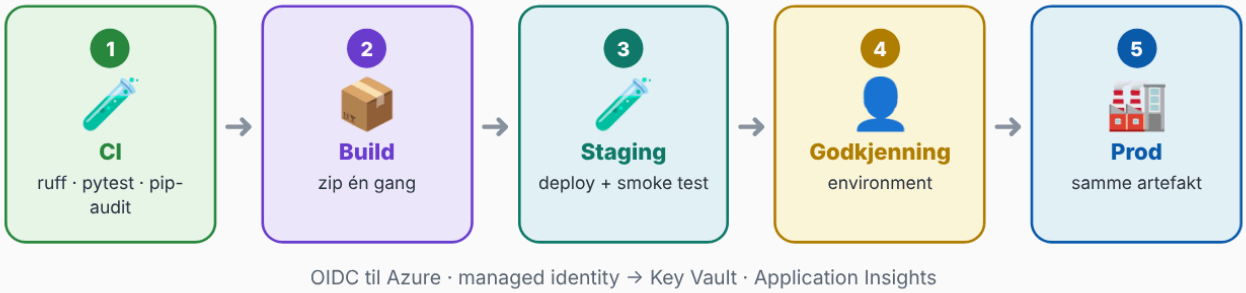
6 · Nettside på Azure Static Web Apps



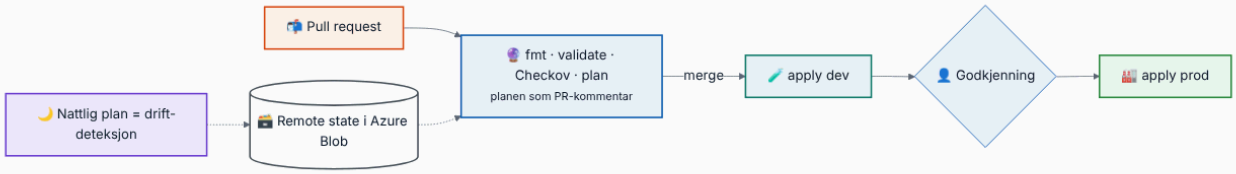
Push til main = ny versjon · PR = egen forhåndsvisning

Tips: Bruk den som din egen CV-nettside – og lenk til den øverst i CV-en.

7 · CI/CD til App Service

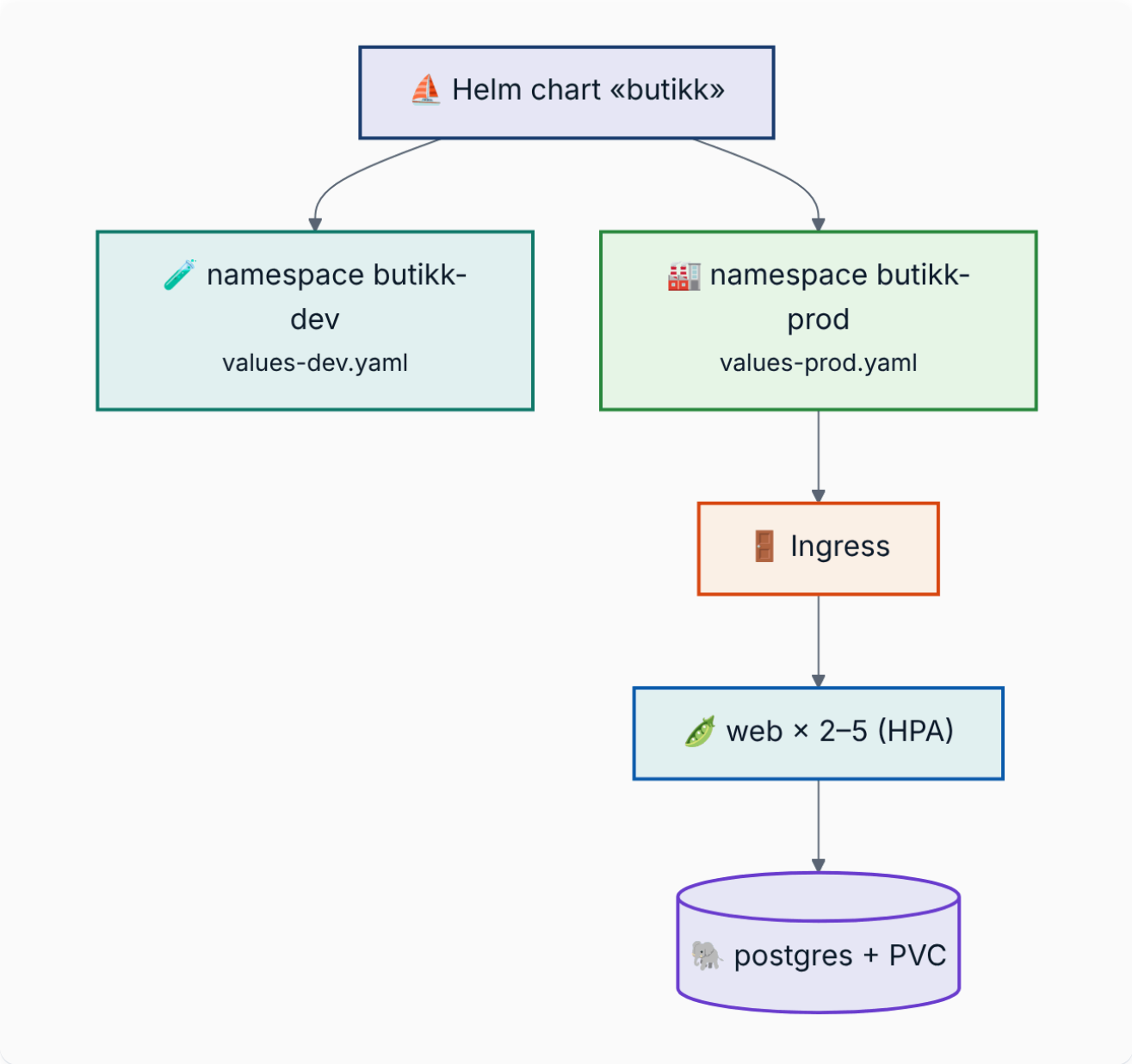


8 · Terraform med pipeline



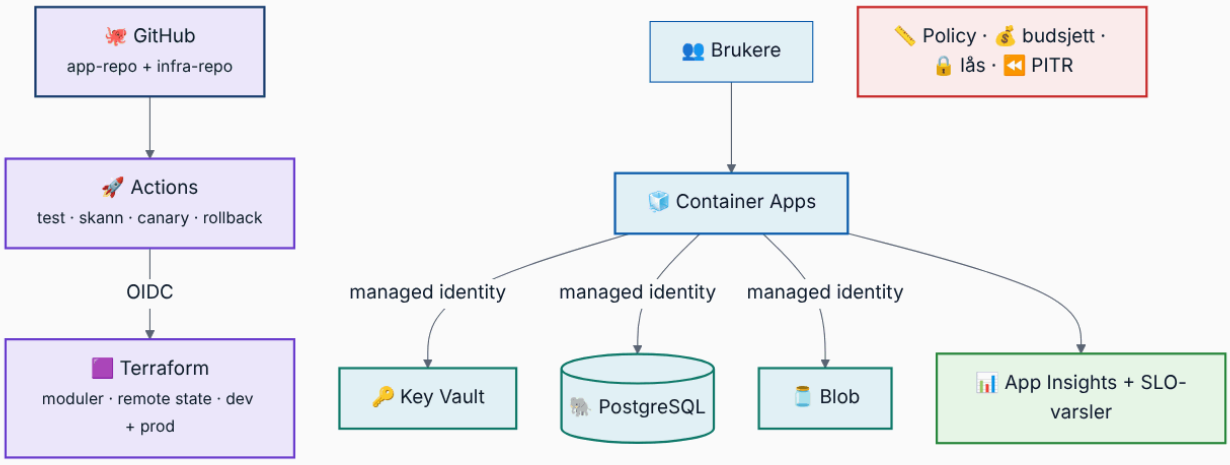
Infrastrukturendringer går bare via pull request

9 · Kubernetes med Helm



Samme chart – dev og prod med ulike values

10 · Komplett plattform – sluttprosjektet



Porteføljens hovedprosjekt

Sju faser

Fase 1 — **Design**
ADR-er, SLO-er, arkitekturtegning

Fase 2 — **IaC**
Moduler, remote state, policy, budsjett

Fase 3 — **App**
Passordløs tilgang, OpenTelemetry

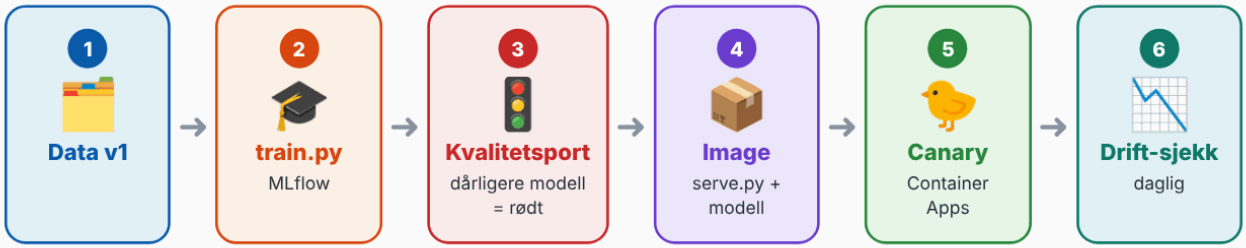
Fase 4 — **CI/CD**
Canary, rollback, skanning

Fase 5 — **Drift**
Dashboard, varsler, runbooks

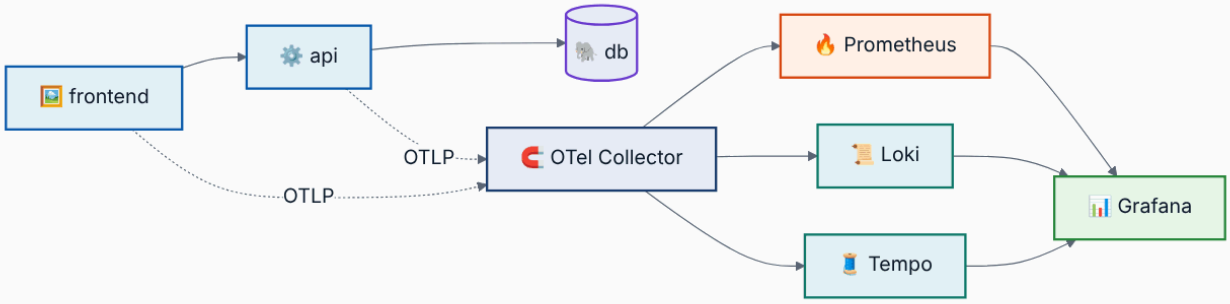
Fase 6 — **Robusthet**
Test PITR og full gjenoppbygging

Fase 7 — **Presentasjon**
README, demovideo, opprydding

11 · MLOps



12 · Observability og game day



Full observability lokalt – og en kollega som «ødelegger» noe i hemmelighet

Slik presenterer du prosjektene

Fest 4–6 repoer
De beste øverst på GitHub-profilen

README-mal
Problem · arkitektur · teknologier · kjør · hva jeg lærte

Tallfest
«image fra 900 til 60 MB» · «gjenoppbygd på 18 min»

STAR i intervju
Situasjon · oppgave · handling · resultat

Vis feilsøkingen
Ærlige problemer du løste imponerer mest

Ingen hemmeligheter
Ingen .env, state eller nøkler i repoet

Vedlegg · Hurtigreferanser, sertifiseringer og læringsplan

A · De viktigste begrepene på ett oppslag



B · Hurtigreferanser

Linux

- pwd · ls -la · cd
- cp · mv · rm -r · mkdir -p
- cat · less · tail -f
- grep -rin · find
- chmod 640 · chown
- systemctl status/restart
- journalctl -u x -f
- df -h · free -h · top
- ss -tlnp · ip addr

Git

- git status · diff
- git add -p · commit -m
- git switch -c branch
- git pull · push
- git merge · rebase
- git restore · revert
- git stash · reflog
- git log --oneline --graph

Docker

- docker build -t navn:tag .
- docker run -d -p 8080:80
- docker ps -a · logs -f
- docker exec -it x sh
- docker inspect · stats
- docker compose up -d --build
- docker compose down -v
- docker system prune

kubectl

- kubectl get pods -A
- kubectl describe pod x
- kubectl logs x --previous
- kubectl apply -f fil.yaml
- kubectl rollout undo deploy/x
- kubectl port-forward svc/x 8080:80
- kubectl top pods
- helm upgrade --install

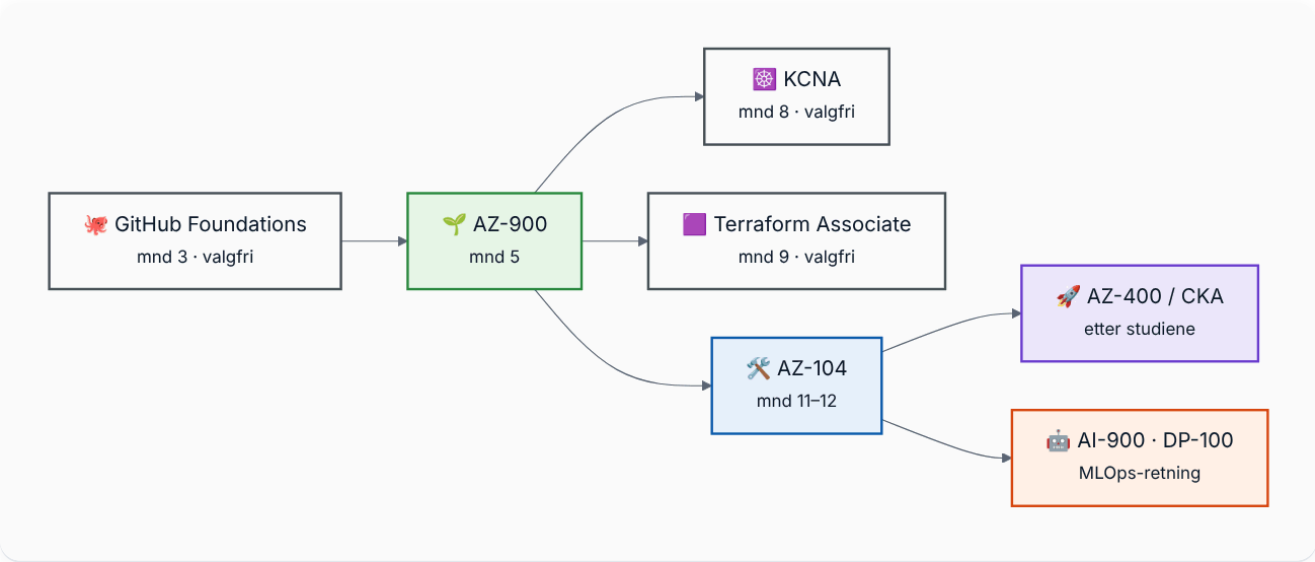
Azure CLI

- az login · az account show
- az group create / delete
- az vm create · deallocate
- az webapp up · log tail
- az keyvault secret set/show
- az role assignment create
- az deployment group what-if
- --query "[].name" -o tsv

Terraform

- terraform init
- terraform fmt · validate
- terraform plan -out=tfplan
- terraform apply tfplan
- terraform output -raw x
- terraform state list
- terraform destroy

C · Sertifiseringer



Forslag til rekkefølge – sjekk alltid Microsoft Learn og andre utstedere for gjeldende eksamener

> 💡 Sertifiseringer åpner dører – men **prosjekter og forståelse** gir jobben. Kombiner dem.

D · Kilder

Offisiell dokumentasjon slik den var kjent per **oktober 2026** – sjekk alltid den nyeste versjonen:

Microsoft Learn
learn.microsoft.com/azure

Sertifiseringer
learn.microsoft.com/credentials

Docker
docs.docker.com

Kubernetes
kubernetes.io/docs

Terraform
developer.hashicorp.com/terraform

GitHub
docs.github.com

Prometheus · Grafana
prometheus.io · grafana.com/docs

OpenTelemetry
opentelemetry.io

DORA
dora.dev

Google SRE
sre.google/books (gratis)

OWASP
owasp.org · genai.owasp.org

roadmap.sh
veikart for DevOps

Bøker

The Phoenix Project
DevOps som roman – start her

The DevOps Handbook
Praksisen bak

Accelerate
Forskningen bak DORA

Terraform: Up & Running
IaC i dybden

E · 12-måneders læringsplan

Ett år – fra nybegynner til jobbklar (ca. 8–12 timer i uken)

Mnd 1 — 🌍 **IT og nettverk**
Kap 1–2 · subnetting · forklar «hva skjer når jeg åpner en nettside»

Mnd 2 — 🐧 **Linux**
Kap 3 · bare terminal i en uke · Prosjekt 1

Mnd 3 — 🐍 **Python og Git**
Kap 4–5 · Prosjekt 4 · GitHub-profil

Mnd 4 — ☁️ **Cloud og Azure-grunnmur**
Kap 6–7.1 · budsjettvarsel · Prosjekt 2

Mnd 5 — 🎓 **Azure-tjenester**
Kap 7.2–7.5 · Prosjekt 3 · TA AZ-900

Mnd 6 — ♾️ **DevOps og Bicep**
Kap 8 og 7.4 · Prosjekt 6 · oppdater CV

Mnd 7 — 🐳 **Docker**
Kap 9 · Prosjekt 5 · start å søke internship

Mnd 8 — ☸️ **Kubernetes**
Kap 10 · Prosjekt 9

Mnd 9 — 🟪 **Infrastructure as Code**
Kap 11 · Prosjekt 8 (start)

Mnd 10 — 🚀 **CI/CD + AZ-104-lab**
Kap 12 · Prosjekt 7 og 8 · søk aktivt

Mnd 11 — 🛡️ **Sikkerhet og monitoring**
Kap 13 · Prosjekt 12 · AZ-104-laber

Mnd 12 — 🏆 **AI + sluttprosjekt**
Kap 14 · Prosjekt 10 · TA AZ-104

Ukerytme

📖 **Mandag · 1,5 t**
Les kapittelet

⌨️ **Onsdag · 2 t**
Øvelser i terminal og Azure

🛠️ **Lørdag · 3–4 t**
Prosjekt

🔁 **Søndag · 1 t**
Quiz, notater, oppdater GitHub

☀️ Sommerjobb / internship
søk fra måned 7–10

- ✔️ 3–5 prosjekter på GitHub
- ✔️ AZ-900
- ✔️ Linux, Git, Docker, Azure-grunnlag

👷 Deltidsjobb i drift/support
når som helst fra måned 3

- ✔️ Linux og nettverk
- ✔️ Vilje til å lære
- ✔️ Praktisk erfaring er gull

🎯 Trainee / junior DevOps
siste studieår

- ✔️ Sluttprosjektet
- ✔️ AZ-104
- ✔️ 6–10 prosjekter, gjerne praksis

Sjekkliste hver måned

📖 **Lest og tatt quiz**

⌨️ **Gjort øvelsene selv**
ikke bare lest

📂 **Prosjekt på GitHub**
med «hva jeg lærte»

🧹 **Ryddet i Azure**
sjekket Cost Management

📄 **Oppdatert CV/LinkedIn**

🎯 **Hva var vanskeligst?**
plan for å styrke det

Tre ting å huske

🧱 **Grunnmuren betyr mest**
Linux, nettverk og «hvorfor» gjør alt annet lettere

📂 **Bygg, dokumenter, del**
Ett ekte prosjekt slår ti kurs du bare har sett

🧹 **Rydd opp i skyen**
Hver gang

Lykke til med studiene – og velkommen til DevOps! 🚀